

Java Enterprise Handbook

Node.js → Enterprise Java Transition Guide for FAANG & Global Banks

For: Senior Backend Engineers transitioning from Node.js/MERN to Enterprise Java
Level: Intermediate → Advanced → Elite
Focus: FAANG interviews · Enterprise architecture · Production engineering · AI-era readiness

Who This Handbook Is For

You are a production-level backend engineer with 5+ years of Node.js experience. You understand distributed systems, cloud infrastructure, and real-world engineering. You are **not** a beginner programmer — you are a professional making a strategic career pivot.

This handbook does **not** treat you like one. Every concept is mapped from what you already know, explained through the lens of enterprise architecture, and benchmarked against what actually matters at top-tier engineering organizations.

Navigation

#	Section	Priority	Interview Weight
1	Java Fundamentals	★★★★★	High
2	JVM Deep Understanding	★★★★★	Very High
3	Enterprise Java Ecosystem	★★★★★	Very High
4	Production Backend Engineering	★★★★★	High
5	Databases & Persistence	★★★★★	High
6	Distributed Systems	★★★★★	Very High
7	Concurrency & Multithreading	★★★★★	Very High
8	Cloud & DevOps Integration	★★★★☆	Medium
9	Security	★★★★☆	High
10	System Design	★★★★★	Very High
11	Interview Preparation	★★★★★	Core
12	AI Era Engineering	★★★★☆	Growing
13	Real Enterprise Engineering	★★★★☆	Medium
14	Node.js → Java Mapping	★★★★★	Foundational
15	Practical Learning & Projects	★★★★★	Portfolio

Resources

- [Java Syntax Cheatsheet](#)
- [Spring Boot Cheatsheet](#)
- [Topic Priority Matrix](#)
- [6-Month Mastery Roadmap](#)
- [Interview Prep Sequence](#)
- [Enterprise Engineering Checklist](#)
- [Java vs Node.js Comparison Table](#)
- [Production Architecture Examples](#)
- [Common Mistakes to Avoid](#)

Core Philosophy

What Elite Engineers Know That Average Engineers Don't

1. The "Why" before the "How"

Enterprise Java isn't just syntax. It's a set of architectural decisions made for scale, team size, long-term maintainability, and enterprise contracts. Every pattern — DI, AOP, JPA — exists to solve a problem at scale that 10-person startups never face.

2. The JVM is the Moat

Java's true competitive advantage is not syntax — it's the JVM. 30 years of GC tuning, JIT optimization, and battle-hardened thread safety. When you understand the JVM, you understand *why* Java dominates banks and FAANG infrastructure.

3. Type Safety is Architecture

Coming from JavaScript, you'll initially feel constrained by types. Within 3 months, you'll realize strong typing **is** your architecture documentation, your compiler-checked contracts, and your first line of defense in multi-team codebases.

4. Spring is not "just a framework"

Spring Boot is the lingua franca of enterprise Java. Understanding Spring's bean lifecycle, dependency injection container, AOP proxy model, and autoconfiguration mechanism is non-negotiable at HSBC, Goldman Sachs, or JP Morgan.

5. Concurrency is a First-Class Citizen

Node.js is single-threaded by design. Java is multi-threaded by default. This changes everything — from how you model request handling to how you think about shared state. JVM concurrency primitives are tested heavily at FAANG.

The Transition Mindset

Node.js World		Java World
npm	→	Maven / Gradle
package.json	→	pom.xml / build.gradle
Express / Fastify	→	Spring MVC / Spring WebFlux
Mongoose / TypeORM	→	Hibernate / Spring Data JPA
dotenv	→	application.properties / Vault
async/await	→	CompletableFuture / Project Reactor
EventEmitter	→	Spring Events / Kafka
PM2 / cluster	→	JVM threads / thread pools
Node process	→	JVM (Heap + Stack + Metaspace)
JavaScript prototype	→	Java class hierarchy
TypeScript interfaces	→	Java interfaces + generics

Quick Start Learning Path

Week 1-2: Java Language Fluency

- Read Section 1 (Java Fundamentals) and Section 14 (Node.js mapping)
- Code along: rewrite a small Node.js REST API in Java
- Focus: syntax, OOP, generics, streams, lambdas

Week 3-4: JVM + Spring Foundation

- Read Section 2 (JVM) and Section 3 (Spring ecosystem)
- Build a CRUD Spring Boot app with JPA
- Focus: DI container, bean lifecycle, request lifecycle

Week 5-8: Production Patterns

- Read Sections 4, 5, 6, 7
- Build a microservice with Kafka, retry, circuit breaker
- Focus: resilience, transactions, concurrency

Week 9-12: System Design + Security

- Read Sections 9, 10
- Design banking-grade payment service
- Focus: architecture decisions, trade-offs, security model

Week 13-20: Interview + Enterprise Depth

- Read Sections 11, 12, 13, 15
- Build 2 portfolio projects
- Mock interviews: LLD, HLD, behavioral

Month 5-6: Mastery Sprint

- Contribute to open-source Spring projects
- Study real-world production incidents
- Target: confident at senior/staff level interviews

What Companies Actually Test

FAANG (Google, Amazon, Meta, Apple, Netflix)

- **Algorithms + Data Structures** (LeetCode medium-hard)
- **System Design** (HLD at massive scale)
- **JVM internals** at senior/staff level
- **Concurrency patterns**
- **Behavioral** (leadership principles)

Global Banks (HSBC, Goldman, JP Morgan, Morgan Stanley)

- **Spring ecosystem depth** (DI, security, data)
- **Transaction management** (isolation levels, distributed tx)
- **Enterprise patterns** (event sourcing, CQRS, saga)
- **Security** (OAuth2, JWT, OWASP)
- **Compliance-grade logging and auditing**
- **Performance under load**

Product MNCs (Stripe, Atlassian, Uber, Adobe)

- API design quality
 - Microservices patterns
 - Kafka / event-driven depth
 - Observability
 - Resilience engineering
-

How to Use This Handbook

1. **Read actively** — every section has "what elite engineers think about this" callouts
 2. **Code along** — every code example should be typed, not copy-pasted
 3. **Map everything** — use Section 14 as your bridge before every new concept
 4. **Think aloud** — practice explaining every concept to an imaginary interviewer
 5. **Build, don't just read** — Section 15 has projects; build them
-

This handbook is a living document. Revisit sections as you gain practical experience — the depth you extract will increase over time.

Section 1: Java Fundamentals

Node.js Engineer's Lens: Java is TypeScript taken to its logical extreme — fully compiled, deeply typed, with a runtime (JVM) that has been battle-hardened for 30 years. If you can write TypeScript confidently, Java syntax will feel familiar within 2 weeks.

1.1 Java Syntax Basics

The Compilation Model

```
Node.js:  source.js → V8 JIT → machine code (at runtime)
Java:    source.java → javac → bytecode (.class) → JVM → JIT → machine code
```

Java compiles to **bytecode** (platform-independent), not machine code directly. The JVM interprets and JIT-compile it at runtime. This is why "write once, run anywhere" is real.

Hello World — What It Actually Means

```
public class HelloWorld {           // Every .java file = one public class
    public static void main(String[] args) { // Entry point (JVM calls this)
        System.out.println("Hello, World!"); // System.out = stdout stream
    }
}
```

Mapping from Node.js: - `public class HelloWorld` → like a named module export - `public static void main` → like `index.js` entry point - `System.out.println` → `console.log` - No semicolons are optional — they are **mandatory**

The `public static void main` Contract

This is the JVM-defined entry contract. `static` means the JVM calls it without instantiating the class. This is why your Spring Boot apps have:

```
@SpringBootApplication
public class MyApp {
    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args); // Hands off to Spring container
    }
}
```

1.2 Variables and Data Types

Primitive Types (Stack-allocated, value semantics)

Java Type	Size	Range	Node.js Equivalent
<code>byte</code>	8-bit	-128 to 127	Number (no equiv)
<code>short</code>	16-bit	-32768 to 32767	Number
<code>int</code>	32-bit	~±2.1 billion	Number
<code>long</code>	64-bit	~±9.2 quintillion	BigInt

Java Type	Size	Range	Node.js Equivalent
float	32-bit	IEEE 754	—
double	64-bit	IEEE 754	Number
char	16-bit	Unicode char	—
boolean	—	true/false	Boolean

Critical production insight: In banking systems, **never use float or double for money**. Use `BigDecimal` :

```
// WRONG – floating point precision issues
double price = 0.1 + 0.2; // 0.30000000000000004

// CORRECT – exact decimal arithmetic
BigDecimal price = new BigDecimal("0.1").add(new BigDecimal("0.2"));
// Result: 0.3 (exact)
```

Reference Types (Heap-allocated, reference semantics)

```
String name = "Alice";           // Object on heap, variable holds reference
int[] numbers = {1, 2, 3};       // Array object on heap
List<String> list = new ArrayList<>(); // Collection on heap
```

The **null** problem — Java's billion-dollar mistake:

```
String name = null;
name.length(); // NullPointerException – crashes the JVM thread

// Modern Java: Use Optional (like Node's optional chaining but explicit)
Optional<String> safeName = Optional.ofNullable(name);
safeName.map(String::length).orElse(0); // Returns 0 if null
```

var — Java 10+ Type Inference

```
var list = new ArrayList<String>(); // Compiler infers ArrayList<String>
var map = new HashMap<String, Integer>(); // Inferred, not dynamic like JS
```

`var` is **still statically typed** — the compiler infers the type at compile time. This is NOT JavaScript's dynamic typing.

String — More Than You Think

```
// String is immutable in Java – like Python strings
String s1 = "hello";
String s2 = s1.toUpperCase(); // Returns NEW string, s1 unchanged

// String pool – interning for memory efficiency
String a = "hello"; // Stored in string pool
String b = "hello"; // Reuses same pool object
System.out.println(a == b); // true (same reference from pool)
System.out.println(a.equals(b)); // true (content comparison)

// ALWAYS use .equals() for string comparison, NEVER ==
String input = getUserInput(); // New String object
input == "expected" // WRONG – compares references
input.equals("expected") // CORRECT – compares content
```

StringBuilder for performance (critical in hot paths):

```
// Node.js: strings are immutable, JS engines optimize concatenation
// Java: explicit StringBuilder is required for performance in loops

// WRONG – creates N intermediate string objects in loop
String result = "";
for (int i = 0; i < 10000; i++) {
    result += i; // Each += creates a new String object
}

// CORRECT
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 10000; i++) {
    sb.append(i);
}
String result = sb.toString();
```

1.3 Object-Oriented Programming

Classes and Objects

```
// Equivalent to a TypeScript class
public class BankAccount {
    // Fields (instance variables) – private by default in good design
    private final String accountId; // final = cannot be reassigned
    private BigDecimal balance;
    private final String owner;

    // Constructor
    public BankAccount(String accountId, String owner, BigDecimal initialBalance) {
        this.accountId = accountId;
        this.owner = owner;
        this.balance = initialBalance;
    }

    // Method
    public void deposit(BigDecimal amount) {
        if (amount.compareTo(BigDecimal.ZERO) <= 0) {
            throw new IllegalArgumentException("Deposit amount must be positive");
        }
        this.balance = this.balance.add(amount);
    }

    // Getter
    public BigDecimal getBalance() {
        return balance;
    }

    // toString – like Node's inspect/toString
    @Override
    public String toString() {
        return String.format("BankAccount{id=%s, owner=%s, balance=%s}",
            accountId, owner, balance);
    }
}
```

The Four Pillars — Enterprise Application

1. Encapsulation — hide implementation details

```
// Bad: public fields – any code can mutate state
public class Order {
    public List<OrderItem> items; // Any class can call items.clear()!
}

// Good: controlled access
public class Order {
    private final List<OrderItem> items = new ArrayList<>();
}
```

```

    public void addItem(OrderItem item) {
        validateItem(item);
        items.add(item);
        recalculateTotal();
    }

    public List<OrderItem> getItems() {
        return Collections.unmodifiableList(items); // Read-only view
    }
}

```

2. Inheritance — extend behavior

```

public abstract class BaseEntity {
    private Long id;
    private Instant createdAt;
    private Instant updatedAt;
    // Common JPA fields – every entity inherits these
}

public class User extends BaseEntity {
    private String email;
    private String passwordHash;
}

```

3. Polymorphism — same interface, different behavior

```

// Strategy pattern – common in enterprise payment systems
public interface PaymentProcessor {
    PaymentResult process(PaymentRequest request);
}

public class StripeProcessor implements PaymentProcessor {
    @Override
    public PaymentResult process(PaymentRequest request) {
        // Stripe-specific implementation
    }
}

public class BraintreeProcessor implements PaymentProcessor {
    @Override
    public PaymentResult process(PaymentRequest request) {
        // Braintree-specific implementation
    }
}

// Runtime polymorphism – client doesn't know which processor
public class PaymentService {
    private final PaymentProcessor processor; // Injected via Spring DI

    public void processPayment(PaymentRequest request) {
        PaymentResult result = processor.process(request); // Polymorphic call
    }
}

```

4. Abstraction — hide complexity behind contracts

1.4 Interfaces vs Abstract Classes

This is a frequently tested Java concept:

	Interface	Abstract Class
State	No instance state	Can have fields
Constructor	None	Yes
Multiple inherit	YES (a class can	NO (single extends)

	implement many)	
Default methods	Yes (Java 8+)	Yes
When to use	Define a contract (can-do capability)	Share code + state (is-a relationship)

```
// Interface – defines a CAPABILITY contract
public interface Auditable {
    void logAccess(String userId, String action); // abstract by default

    default String getAuditPrefix() { // Default implementation
        return "[AUDIT]";
    }
}

public interface Cacheable {
    String getCacheKey();
    Duration getTTL();
}

// A class can implement BOTH capabilities
public class UserService implements Auditable, Cacheable {
    @Override
    public void logAccess(String userId, String action) { ... }

    @Override
    public String getCacheKey() { return "user:" + userId; }

    @Override
    public Duration getTTL() { return Duration.ofMinutes(5); }
}

// Abstract class – defines shared state + partial implementation
public abstract class BaseRepository<T, ID> {
    protected final EntityManager em; // Shared state

    public BaseRepository(EntityManager em) {
        this.em = em;
    }

    public Optional<T> findById(ID id) {
        // Common implementation all repos share
        return Optional.ofNullable(em.find(getEntityClass(), id));
    }

    // Subclasses MUST implement this
    protected abstract Class<T> getEntityClass();
}
```

Enterprise rule: In Spring applications, prefer interfaces for service contracts (allows proxying, mocking, multiple implementations). Use abstract classes for shared persistence or utility logic.

1.5 Collections Framework

The Hierarchy That Matters

```
Collection
├── List (ordered, allows duplicates)
│   ├── ArrayList – dynamic array, O(1) get, O(n) insert middle
│   └── LinkedList – doubly linked, O(1) insert, O(n) get
├── Set (no duplicates)
│   ├── HashSet – O(1) ops, no order (HashMap internally)
│   ├── LinkedHashSet – preserves insertion order
│   └── TreeSet – sorted, O(log n) ops (Red-Black tree)
└── Queue/Deque
    ├── ArrayDeque – fast double-ended queue
    └── PriorityQueue – heap-based, min/max priority

Map (key-value, not Collection)
```

- └─ HashMap → O(1) avg, unordered
- └─ LinkedHashMap → insertion-ordered HashMap
- └─ TreeMap → sorted by key, O(log n)
- └─ ConcurrentHashMap → thread-safe HashMap (critical in Java)

Choosing the Right Collection

```
// ArrayList – default choice for ordered lists
List<Transaction> transactions = new ArrayList<>();

// LinkedList – when you frequently insert/remove at head/tail
Deque<Event> eventQueue = new LinkedList<>();

// HashSet – when uniqueness matters, order doesn't
Set<String> processedEventIds = new HashSet<>();

// LinkedHashMap – LRU cache implementation
Map<String, CachedValue> lruCache = new LinkedHashMap<>(100, 0.75f, true) {
    @Override
    protected boolean removeEldestEntry(Map.Entry eldest) {
        return size() > 100;
    }
};

// TreeMap – sorted by key (useful for range queries, leaderboards)
TreeMap<LocalDate, List<Transaction>> txByDate = new TreeMap<>();

// ConcurrentHashMap – multi-threaded environments (NO HashMap in threads!)
ConcurrentHashMap<String, RateLimiter> rateLimiters = new ConcurrentHashMap<>();
```

Node.js → Java Collections Mapping

```
Node.js Array           → ArrayList<T>
Node.js Set             → HashSet<T>
Node.js Map             → HashMap<K,V>
Node.js Object {}       → HashMap<String,Object> or a proper DTO class
Node.js Array methods → Java Streams API
```

1.6 Streams and Functional Programming

Streams are Java's answer to JavaScript's `.map()`, `.filter()`, `.reduce()`. They are **lazy**, **declarative**, and **pipeline-based**.

Stream Operations

```
List<Transaction> transactions = getTransactions();

// Equivalent to:
// transactions
// .filter(t => t.getAmount() > 1000)
// .map(t => t.getCustomerId())
// .filter((v, i, a) => a.indexOf(v) === i) // distinct
// .sort()

List<String> highValueCustomers = transactions.stream()
    .filter(t -> t.getAmount().compareTo(new BigDecimal("1000")) > 0) // filter
    .map(Transaction::getCustomerId) // map
    .distinct() // dedupe
    .sorted() // sort
    .collect(Collectors.toList()); // terminal

// Group by (like lodash groupBy)
Map<String, List<Transaction>> txByCustomer = transactions.stream()
    .collect(Collectors.groupingBy(Transaction::getCustomerId));
```

```
// Aggregate
BigDecimal totalVolume = transactions.stream()
    .map(Transaction::getAmount)
    .reduce(BigDecimal.ZERO, BigDecimal::add);

// Statistics
DoubleSummaryStatistics stats = transactions.stream()
    .mapToDouble(t -> t.getAmount().doubleValue())
    .summaryStatistics();
// stats.getAverage(), stats.getMax(), stats.getCount()

// Parallel processing (use carefully – has overhead for small lists)
long count = transactions.parallelStream()
    .filter(t -> t.isHighRisk())
    .count();
```

Lazy Evaluation — Why It Matters

```
// Streams are LAZY – intermediate operations don't execute until terminal
Stream<Transaction> stream = transactions.stream()
    .filter(t -> {
        System.out.println("filtering: " + t.getId()); // Won't print yet
        return t.getAmount().compareTo(BigDecimal.ZERO) > 0;
    })
    .map(t -> {
        System.out.println("mapping: " + t.getId()); // Won't print yet
        return t;
    });

// Terminal operation triggers execution
stream.findFirst(); // Only processes until first match!
```

Production insight: Use `Stream.of(...)` or `.stream()` for in-memory processing. For large datasets, never load everything into memory — use database cursors or `JdbcTemplate.queryForStream()`.

1.7 Lambda Expressions

```
// Syntax: (parameters) -> expression OR (parameters) -> { statements }

// Node.js: (x) => x * 2
// Java:
Function<Integer, Integer> doubler = x -> x * 2;

// Node.js: (a, b) => a + b
BinaryOperator<Integer> add = (a, b) -> a + b;

// Multi-line
Predicate<String> isValidEmail = email -> {
    if (email == null || email.isEmpty()) return false;
    return email.contains("@") && email.contains(".");
};

// Method references (cleaner than lambdas when method already exists)
// Node.js: arr.map(x => x.toString()) -> Java:
List<String> strings = numbers.stream()
    .map(Object::toString) // instance method
    .collect(Collectors.toList());

List<String> upper = strings.stream()
    .map(String::toUpperCase) // unbound instance method
    .collect(Collectors.toList());

// Static method reference
.filter(Objects::nonNull) // Objects.nonNull(x)

// Constructor reference
.map(User::new) // new User(x)
```

Functional Interfaces

```
// The four core functional interfaces
Function<T, R>      - takes T, returns R (map)
Predicate<T>       - takes T, returns boolean (filter)
Consumer<T>        - takes T, returns void (forEach)
Supplier<T>        - takes nothing, returns T (lazy init)

// Common usage
Function<String, Integer> parser = Integer::parseInt;
Predicate<User> isAdmin = user -> user.hasRole("ADMIN");
Consumer<Event> logger = event -> log.info("Event: {}", event);
Supplier<UUID> idGenerator = UUID::randomUUID;
```

1.8 Exception Handling

Checked vs Unchecked — A Critical Java Distinction

```
Throwable
├── Error (JVM-level: OutOfMemoryError, StackOverflowError) – don't catch
└── Exception
    ├── RuntimeException (UNCHECKED – no throws declaration required)
    │   ├── NullPointerException
    │   ├── IllegalArgumentException
    │   ├── IllegalStateException
    │   └── (your custom runtime exceptions)
    └── IOException, SQLException, etc. (CHECKED – must declare or catch)
```

```
// CHECKED exception – must handle or declare
public void readFile(String path) throws IOException {
    // If you don't catch it, you must declare throws
    Files.readAllBytes(Paths.get(path));
}

// UNCHECKED exception – runtime, no declaration needed
public User findUser(String id) {
    User user = repo.findById(id);
    if (user == null) {
        throw new UserNotFoundException("User not found: " + id); // RuntimeException
    }
    return user;
}

// Try-with-resources (ALWAYS use for I/O, DB connections)
try (Connection conn = dataSource.getConnection();
     PreparedStatement stmt = conn.prepareStatement(sql)) {
    // Resources auto-closed even if exception thrown
    return stmt.executeQuery();
} catch (SQLException e) {
    log.error("DB error on query: {}", sql, e);
    throw new DataAccessException("Failed to execute query", e); // Wrap and rethrow
}
```

Custom Exception Hierarchy — Enterprise Pattern

```
// Base application exception
public class AppException extends RuntimeException {
    private final ErrorCode errorCode;
    private final String correlationId;

    public AppException(ErrorCode errorCode, String message) {
        super(message);
        this.errorCode = errorCode;
        this.correlationId = MDC.get("correlationId"); // From logging context
    }
}
```

```

}

// Domain-specific exceptions
public class InsufficientFundsException extends AppException {
    private final BigDecimal requested;
    private final BigDecimal available;

    public InsufficientFundsException(BigDecimal requested, BigDecimal available) {
        super(ErrorCode.INSUFFICIENT_FUNDS,
            String.format("Requested %.2f but only %.2f available", requested, available));
        this.requested = requested;
        this.available = available;
    }
}

// Spring exception handler — maps exceptions to HTTP responses
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(InsufficientFundsException.class)
    public ResponseEntity<ErrorResponse> handleInsufficientFunds(InsufficientFundsException ex) {
        return ResponseEntity.status(HttpStatus.UNPROCESSABLE_ENTITY)
            .body(new ErrorResponse(ex.getErrorCode(), ex.getMessage()));
    }
}

```

1.9 Generics

Generics are like TypeScript generics — compile-time type safety:

```

// Generic class
public class ApiResponse<T> {
    private final T data;
    private final String requestId;
    private final Instant timestamp;

    public static <T> ApiResponse<T> success(T data) {
        return new ApiResponse<>(data, generateRequestId(), Instant.now());
    }
}

// Usage
ApiResponse<User> userResponse = ApiResponse.success(user);
ApiResponse<List<Transaction>> listResponse = ApiResponse.success(transactions);

// Bounded type parameters
public <T extends Comparable<T>> T findMax(List<T> list) {
    return list.stream().max(Comparator.naturalOrder()).orElseThrow();
}

// Wildcards
public void printAll(List<? extends Shape> shapes) {
    shapes.forEach(s -> System.out.println(s.area())); // PECS: producer extends
}

public void addToList(List<? super Integer> list) {
    list.add(42); // PECS: consumer super
}

```

Type erasure — the key JVM concept: Generic types are erased at runtime. `List<String>` and `List<Integer>` are both `List` at the bytecode level. This affects reflection and some serialization patterns.

1.10 Enums

Java enums are far more powerful than TypeScript enums:

```
// Simple enum
public enum TransactionStatus {
    PENDING, PROCESSING, COMPLETED, FAILED, REVERSED
}

// Enum with fields and methods (very common in enterprise code)
public enum PaymentMethod {
    CREDIT_CARD("CC", true, 0.029),
    DEBIT_CARD("DC", true, 0.01),
    BANK_TRANSFER("BT", false, 0.0),
    CRYPTO("CRYPTO", false, 0.001);

    private final String code;
    private final boolean requiresCVV;
    private final double feeRate;

    PaymentMethod(String code, boolean requiresCVV, double feeRate) {
        this.code = code;
        this.requiresCVV = requiresCVV;
        this.feeRate = feeRate;
    }

    public BigDecimal calculateFee(BigDecimal amount) {
        return amount.multiply(BigDecimal.valueOf(feeRate));
    }

    // Factory method pattern with enum
    public static PaymentMethod fromCode(String code) {
        return Arrays.stream(values())
            .filter(pm -> pm.code.equals(code))
            .findFirst()
            .orElseThrow(() -> new IllegalArgumentException("Unknown code: " + code));
    }
}

// Usage
PaymentMethod method = PaymentMethod.CREDIT_CARD;
BigDecimal fee = method.calculateFee(new BigDecimal("100.00")); // 2.90
```

1.11 Annotations

Annotations are metadata decorators (like TypeScript decorators, but baked into the language):

```
// Built-in annotations
@Override          // Compiler check: must override parent method
@Deprecated        // Marks API as deprecated
@SuppressWarnings  // Suppress compiler warnings
@FunctionalInterface // Marks interface as having single abstract method

// Spring annotations (most common in enterprise code)
@Component          // Register as Spring bean
@Service            // Semantic: business logic layer
@Repository         // Semantic: data access layer + exception translation
@Controller/@RestController // HTTP endpoint handler
@Autowired           // Inject dependency
@Value("${prop}")    // Inject property value
@Transactional       // Wrap in database transaction
@Cacheable          // Cache method result
@Async              // Execute in thread pool
@Scheduled           // Cron/fixed-rate scheduling

// Custom annotation example
@Target(ElementType.METHOD) // Can be applied to methods
@Retention(RetentionPolicy.RUNTIME) // Available at runtime via reflection
@Documentd
public @interface AuditLog {
    String action();
    String resource() default "";
}
```

```
// Usage
@AuditLog(action = "TRANSFER", resource = "account")
public void transfer(String fromId, String toId, BigDecimal amount) {
    // Spring AOP intercepts this call and logs the audit
}
```

1.12 Records — Modern Java (Java 16+)

Records are immutable data carriers — like TypeScript interfaces with auto-generated constructors:

```
// Java 16+: Records (immutable DTOs)
public record TransferRequest(
    String fromAccountId,
    String toAccountId,
    BigDecimal amount,
    String currency,
    String description
) {
    // Compact constructor for validation
    public TransferRequest {
        Objects.requireNonNull(fromAccountId, "fromAccountId required");
        Objects.requireNonNull(toAccountId, "toAccountId required");
        if (amount.compareTo(BigDecimal.ZERO) <= 0) {
            throw new IllegalArgumentException("Amount must be positive");
        }
    }
}

// Auto-generated: constructor, getters, equals, hashCode, toString
TransferRequest req = new TransferRequest("ACC-1", "ACC-2",
    new BigDecimal("500.00"), "USD", "Rent payment");
req.amount(); // BigDecimal(500.00) — getter named after field, not getAmount()
```

1.13 Sealed Classes — Modern Java (Java 17+)

```
// Like TypeScript discriminated unions
public sealed interface Shape
    permits Circle, Rectangle, Triangle {}

public record Circle(double radius) implements Shape {}
public record Rectangle(double width, double height) implements Shape {}
public record Triangle(double base, double height) implements Shape {}

// Pattern matching switch (Java 21)
double area = switch (shape) {
    case Circle c -> Math.PI * c.radius() * c.radius();
    case Rectangle r -> r.width() * r.height();
    case Triangle t -> 0.5 * t.base() * t.height();
};
```

1.14 Key Syntax Reference

```
// Variable declaration
int count = 0;
final String NAME = "constant"; // final = const in JS
var list = new ArrayList<String>();

// Control flow
if (condition) { } else if (other) { } else { }

// Enhanced for loop (like JS for...of)
for (String item : collection) { }
```

```

// Traditional for
for (int i = 0; i < n; i++) { }

// While
while (condition) { }

// Switch expression (Java 14+)
String result = switch (status) {
    case PENDING -> "Waiting";
    case COMPLETED -> "Done";
    default -> "Unknown";
};

// Ternary
String label = isActive ? "Active" : "Inactive";

// String formatting
String msg = "User %s has balance %,.2f".formatted(name, balance); // Java 15+
String msg2 = String.format("User %s has balance %,.2f", name, balance);

// Text blocks (Java 15+) – like template literals
String json = """
    {
        "name": "%s",
        "amount": %s
    }
    """.formatted(name, amount);

// Null safety
String result = Optional.ofNullable(getValue())
    .map(String::trim)
    .filter(s -> !s.isEmpty())
    .orElse("default");

// instanceof pattern matching (Java 16+)
if (obj instanceof String s) {
    System.out.println(s.toUpperCase()); // s already cast
}

```

Section Summary: What Matters for Interviews

FAANG interviewers expect: - Fluent Java syntax (no Googling basics) - Deep understanding of generics and type erasure - Collections API mastery (choosing the right data structure) - Streams for data processing problems - Exception hierarchy and checked vs unchecked

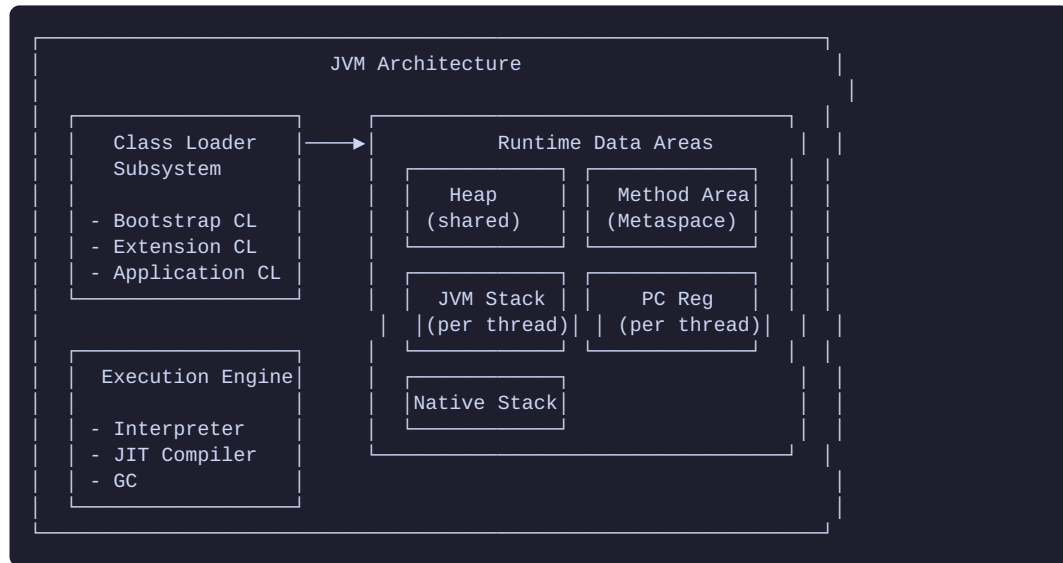
Bank interviewers additionally ask: - BigDecimal for monetary calculations - Null safety strategies (Optional, annotations) - Custom exception hierarchies - Annotation-driven enterprise patterns

What to practice: 1. Rewrite 5 of your Node.js utility functions in Java 2. Use Streams to solve 10 LeetCode array problems 3. Design a custom exception hierarchy for a payment system 4. Build a generic `Repository<T, ID>` interface

Section 2: JVM Deep Understanding

Why This Matters: JVM internals are tested at every senior Java interview at FAANG and banks. More importantly, understanding the JVM is what separates engineers who can diagnose production incidents from those who can only write code that works in development.

2.1 JVM Architecture Overview



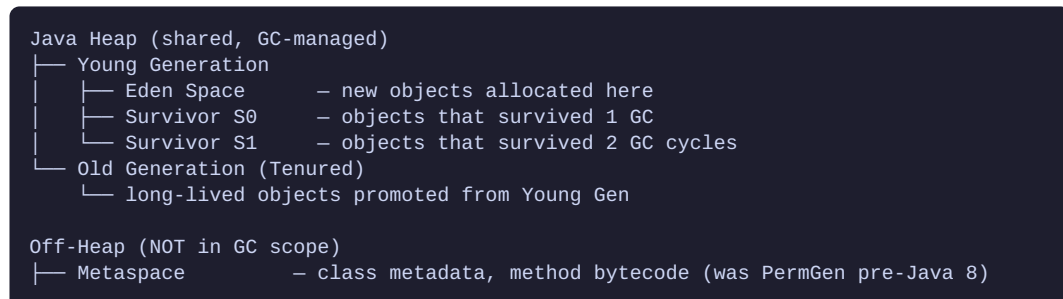
Node.js vs JVM Runtime

Node.js (V8 Engine)	Java (JVM)
Single process, event loop	Multi-threaded by default
V8 heap (all objects)	Heap (GC-managed objects)
V8 stack (call frames)	Stack per thread
Node native modules	JNI (Java Native Interface)
V8 JIT compilation	JVM JIT (C1+C2 compilers)
Automatic GC (V8 GC)	Pluggable GC (G1, ZGC, etc.)
No warm-up concept	JIT warm-up critical

2.2 Memory Architecture — Deep Dive

Heap Memory

The heap is where all objects live. It is shared across all threads (which is why thread safety matters in Java).



```
└─ Direct Buffer    - ByteBuffer.allocateDirect() - used in Netty/NIO
└─ Native Memory   - JNI, malloc()
```

Stack Memory (per thread)

```
Each Thread has its own Stack:
└─ Frame 1: main()
    └─ local variables (primitives stored by value, refs stored as address)
        └─ operand stack
└─ Frame 2: processPayment()
    └─ local variables
        └─ partial results
└─ Frame 3: validateAmount()
    └─ local variables
```

Critical insight: - **Primitives in methods** → Stack (cheap, no GC) - **Objects** → Heap (GC-managed, more expensive) - **Stack frames** are automatically reclaimed when method returns - `StackOverflowError` = too deep recursion (stack runs out of space)

Practical Memory Model

```
public void processOrder(String orderId) {           // New stack frame created
    int retryCount = 0;                               // int on stack (value)
    Order order = orderService.findById(orderId);     // reference on stack,
                                                    // Order object on heap
    List<OrderItem> items = order.getItems();         // reference on stack

    for (OrderItem item : items) {                   // item ref on stack
        processItem(item);                           // new stack frame
    }
    // All local refs go out of scope - objects become GC eligible
}                                                    // Stack frame popped
```

2.3 Garbage Collection — Production Knowledge

GC Fundamentals

```
Object lifecycle:
Eden Space → (Minor GC) → Survivor → (if age > threshold) → Old Gen
                                                    → (Major/Full GC)
```

Minor GC (Young Gen collection): - Frequent, fast (milliseconds) - "Stop the world" but brief - Most objects die young (infant mortality hypothesis)

Major/Full GC (includes Old Gen): - Infrequent, slow (can be seconds) - "Stop the world" pauses affect latency - What causes your P99 latency spikes in production

GC Algorithms — Choosing for Production

GC	Default	Latency	Throughput	Use Case
Serial GC	No	High pauses	Low	Single-core/small apps
Parallel GC	Java 8 default	Medium	High	Batch processing
G1 GC	Java 9+ default	Low pauses	Good	General purpose
ZGC	Java 15+ production	Sub-millisecond	Good	Low-latency apps
Shenandoah	Available	Sub-millisecond	Good	Red Hat OpenJDK

For banking/FAANG APIs — use G1GC or ZGC:

```
-XX:+UseG1GC
-XX:MaxGCPauseMillis=200      # Target max pause
-XX:G1HeapRegionSize=16m      # Region size

# For ultra-low latency (trading systems, real-time APIs):
-XX:+UseZGC
-XX:SoftMaxHeapSize=4g
```

GC Tuning — What You'll See in Production

```
# Common JVM flags in enterprise docker-compose / k8s manifests
java \
  -Xms2g \                # Initial heap (set equal to Xmx to avoid GC on startup)
  -Xmx4g \                # Max heap
  -XX:+UseG1GC \          # GC algorithm
  -XX:MaxGCPauseMillis=200 \ # GC pause target
  -XX:+HeapDumpOnOutOfMemoryError \ # OOM heap dump for post-mortem
  -XX:HeapDumpPath=/var/log/dumps \ # Where to write dumps
  -XX:+PrintGCDetails \   # GC logs (critical for tuning)
  -Xlog:gc*:file=/var/log/gc.log \ # GC log file
  -jar app.jar
```

Memory Leaks — What Causes Them in Java

Unlike Node.js where memory leaks are common (event listeners, closures), Java's GC is more forgiving. But leaks still happen:

```
// 1. Static collections that grow unboundedly
public class BadCache {
  private static final Map<String, Object> CACHE = new HashMap<>();
  // Objects never removed → OOM eventually
}

// 2. Listener not deregistered
eventBus.register(listener); // If never unregistered, listener object is retained

// 3. ThreadLocal not cleaned up
private static final ThreadLocal<UserContext> context = new ThreadLocal<>();
// In thread pools (like in Spring), threads are reused!
// Must call context.remove() after request handling

// 4. Inner class holding outer class reference
class MyRunnable implements Runnable {
  void doSomething() {
    OuterClass.this.someField; // Implicit reference to OuterClass
  }
}
// If MyRunnable lives longer than OuterClass intends, OuterClass is not GC'd
```

2.4 JIT Compilation — Why Java Is Fast

The Tiered Compilation Model

```
Code execution path:
1. Interpreter      - immediate start, slow (profiling)
2. C1 Compiler      - quick compile, moderate speed (client compiler)
3. C2 Compiler      - deep optimize after profiling, fast (server compiler)

Tiered: C1 → profiling data → C2 → optimized native code
```

JVM warm-up phenomenon:

```
Node.js: V8 optimizes quickly, warm-up ~seconds
Java:    Full C2 optimization takes 30-60 seconds or more of real load
```

This is why:

- Don't benchmark immediately after startup
- Kubernetes readiness probes must account for warm-up
- Spring Boot + GraalVM native images eliminate warm-up (AOT compilation)

JIT Optimizations You Should Know

Inlining: The JIT inlines small/frequently-called methods into the call site, eliminating method call overhead. This is why Java tiny getter/setter methods are "free."

Escape Analysis: Objects that don't "escape" a method (not returned or shared) can be allocated on the stack, eliminating GC pressure:

```
public void process() {
    Point p = new Point(1, 2); // May be stack-allocated if JIT determines it doesn't escape
    int sum = p.x + p.y;
    // p is never returned or stored – JIT may optimize away heap allocation
}
```

Dead Code Elimination: Unreachable branches are removed at JIT level, which is why some micro-benchmarks give misleading results.

2.5 Thread Model — Fundamental Difference from Node.js

```
Node.js: Single main thread + libuv thread pool (I/O)
         → Never blocks main thread (async everything)
         → No shared mutable state issues (single thread)

Java:    True OS threads, many per JVM
         → Can block on I/O (but has async APIs too)
         → Shared mutable state = concurrency bugs
         → Need synchronization, thread-safe data structures
```

Thread States

```
NEW → RUNNABLE → BLOCKED/WAITING/TIMED_WAITING → TERMINATED

BLOCKED: waiting for a monitor lock (synchronized block)
WAITING: waiting indefinitely (Object.wait(), LockSupport.park())
TIMED_WAITING: waiting with timeout (Thread.sleep(), wait(timeout))
```

Thread Pools — How Java Applications Actually Run

```
// Spring HTTP request handling (default: embedded Tomcat)
// 200 threads handle 200 concurrent requests
// Each request gets its own thread, can block on DB/I/O freely

// vs Node.js: single thread handles all requests via event loop
// blocking I/O would freeze the entire server

// Spring configures Tomcat thread pool:
server.tomcat.threads.max=200
server.tomcat.threads.min-spare=10

// Virtual Threads (Java 21+) – Project Loom
// Lightweight threads – millions of them, JVM-managed (like goroutines)
Thread.ofVirtual().start(() -> {
    // Can block freely – JVM parks virtual thread, OS thread continues
})
```

```
var result = database.query(...); // Blocking call is fine now
});
```

2.6 Class Loading

```
Class Loading Process:
Source: MyClass.java
Compiled: MyClass.class (bytecode)
Runtime: ClassLoader finds .class → loads into Method Area

Class Loader Hierarchy:
Bootstrap ClassLoader (JDK core: java.lang, java.util)
├── Platform/Extension ClassLoader (JDK extensions)
│   └── Application ClassLoader (your app's classpath)

Delegation model: child asks parent first (parent-first)
```

Why it matters: - Hot reload in development (Spring DevTools) works by using a separate ClassLoader - OSGi, plugin systems use ClassLoader isolation - Certain reflection patterns require explicit ClassLoader handling

2.7 JVM Profiling — Production Tooling

Profiling Tools

```
CPU Profiling:
- async-profiler — low-overhead, production-safe flame graphs
- JFR (Java Flight Recorder) — built-in, minimal overhead
- VisualVM — GUI, good for development

Memory Profiling:
- MAT (Eclipse Memory Analyzer) — heap dump analysis
- JFR heap events
- jmap -heap <pid> — quick heap summary

Thread Analysis:
- jstack <pid> — thread dump (debug deadlocks)
- thread dump in VisualVM
```

Reading a Thread Dump (critical for incident response)

```
"http-nio-8080-exec-1" #25 daemon prio=5
  java.lang.Thread.State: WAITING (parking)
    at sun.misc.Unsafe.park(Unsafe.java)
    at java.util.concurrent.locks.LockSupport.park(LockSupport.java:175)
    at HikariCP.waitForConnection(...) ← WAITING FOR DB CONNECTION POOL
    at MyService.processPayment(...) ← Your code stalled here

Diagnosis: HikariCP pool exhausted — DB connections maxed out
Fix: Increase pool size or find connection leak
```

Java Flight Recorder — Production Profiling

```
// Start JFR programmatically
Recording recording = new Recording();
recording.enable("jdk.CPUload").withPeriod(Duration.ofSeconds(1));
recording.enable("jdk.GarbageCollection");
recording.start();
// ... run workload ...
recording.dump(Paths.get("/tmp/app.jfr"));
```

```
recording.close();  
// Open .jfr in JDK Mission Control (JMC) for visual analysis
```

2.8 JVM Performance Tuning Checklist

Startup Performance

```
# Slow startup? Consider:  
# 1. Spring Boot lazy initialization  
spring.main.lazy-initialization=true  
  
# 2. Class Data Sharing (CDS)  
java -Xshare:dump          # Create archive  
java -Xshare:on -jar app.jar # Use archive on startup  
  
# 3. GraalVM Native Image (AOT – best startup)  
# Startup in milliseconds, but no JIT optimization at runtime
```

Runtime Performance

```
# Heap sizing – CRITICAL  
# Set Xms = Xmx to avoid heap resizing GC overhead  
-Xms4g -Xmx4g  
  
# For containerized environments (Docker/k8s):  
-XX:+UseContainerSupport      # Read cgroup limits (Java 8u191+)  
-XX:MaxRAMPercentage=75.0     # Use 75% of container memory for heap  
# Leave 25% for Metaspace, thread stacks, direct buffers, OS  
  
# G1GC tuning for API servers  
-XX:+UseG1GC  
-XX:MaxGCPauseMillis=200  
-XX:G1NewSizePercent=30       # 30% of heap for young gen  
-XX:G1MaxNewSizePercent=40
```

Container/Kubernetes Specifics

```
# k8s pod spec – proper JVM resource configuration  
resources:  
  requests:  
    memory: "1Gi"  
    cpu: "500m"  
  limits:  
    memory: "2Gi"    # MUST set – OOMKilled without it  
    cpu: "2000m"  
  
# JVM startup args (via JAVA_OPTS env):  
env:  
  - name: JAVA_OPTS  
    value: "-XX:+UseContainerSupport -XX:MaxRAMPercentage=75.0 -XX:+ExitOnOutOfMemoryError"  
    # ExitOnOutOfMemoryError: better to crash + restart than zombie process
```

2.9 Common JVM Production Issues

OOM (OutOfMemoryError) — Types and Diagnosis

```
java.lang.OutOfMemoryError: Java heap space  
→ Heap exhausted: memory leak or undersized heap  
→ Action: Heap dump + MAT analysis, check for unbounded caches  
  
java.lang.OutOfMemoryError: Metaspace  
→ Class metadata exhausted: class loader leak (in app servers, OSGi)
```

```
→ Action: -XX:MaxMetaspaceSize=256m, investigate class loader leaks

java.lang.OutOfMemoryError: GC overhead limit exceeded
→ JVM spending >98% time in GC with <2% reclamation
→ Action: Increase heap or find memory leak

java.lang.OutOfMemoryError: Direct buffer memory
→ Off-heap ByteBuffers exhausted (NIO, Netty)
→ Action: -XX:MaxDirectMemorySize=512m
```

Performance Degradation Patterns

```
Symptom: Latency spikes every ~60 seconds
Cause: Full GC pause
Action: -XX:+UseG1GC, tune region sizes, reduce old gen promotion

Symptom: CPU spikes to 100%
Cause: Tight loop, infinite retry loop, CPU-bound thread
Action: Thread dump, look for tight loops in RUNNABLE threads

Symptom: Thread pool exhausted (RejectedExecutionException)
Cause: Task arrival rate > processing rate
Action: Queue metrics, increase pool or use backpressure

Symptom: Memory grows steadily, never freed
Cause: Memory leak (unbounded cache, listener, ThreadLocal)
Action: Heap dump comparison (before vs after load), MAT
```

Section Summary: JVM Interview Essentials

Questions you must answer confidently:

1. "What is the difference between heap and stack in Java?"
2. "Explain Young Generation, Old Generation, and how promotion works"
3. "What causes a Full GC and how do you reduce them?"
4. "How does the JIT compiler work and what is JVM warm-up?"
5. "Explain memory leaks in Java — how do they happen and how do you detect them?"
6. "What JVM flags would you set for a production microservice in Kubernetes?"
7. "How would you diagnose a thread deadlock in production?"
8. "What is the difference between thread states BLOCKED and WAITING?"
9. "Explain Java class loading — what is the delegation model?"
10. "What is escape analysis and how does it optimize performance?"

Section 3: Enterprise Java Ecosystem — Spring Boot & Beyond

The Reality: Spring Boot is NOT just a framework. It is the operating system of enterprise Java. Understanding how Spring's IoC container, AOP proxy model, and autoconfiguration work is what makes you fluent in enterprise Java codebases at JP Morgan, HSBC, and every FAANG Java backend.

3.1 Spring Boot Architecture — What It Actually Does

The Spring Application Context

```
Spring Application Context = Dependency Injection Container
                           = Object graph manager
                           = Configuration resolver
                           = Bean lifecycle orchestrator
```

When you call `SpringApplication.run()`, Spring: 1. Scans classpath for `@Component`, `@Service`, `@Repository`, `@Controller` 2. Reads `@Configuration` classes and `@Bean` methods 3. Resolves `application.properties` / `application.yml` 4. Runs autoconfiguration (100+ auto-configs based on classpath) 5. Creates and wires all beans (resolves dependencies) 6. Starts embedded Tomcat/Netty 7. Fires `ApplicationReadyEvent`

Node.js → Spring Boot Mental Model

<code>Express app bootstrap</code>	→	Spring Boot <code>@SpringBootApplication</code>
<code>express.use(middleware)</code>	→	<code>@Bean</code> or <code>@Component</code> registration
<code>app.use('/api', router)</code>	→	<code>@RequestMapping</code> on <code>@Controller</code>
<code>require('./config')</code>	→	<code>@ConfigurationProperties</code>
<code>process.env.PORT</code>	→	<code>\${server.port}</code> or <code>@Value("\${server.port}")</code>
<code>module.exports = service</code>	→	<code>@Service</code> class (Spring manages lifecycle)
<code>new Service(dep1, dep2)</code>	→	Constructor injection (Spring injects)

3.2 Dependency Injection — The Core Principle

Why DI Exists in Enterprise Software

In a 50-engineer team working on a monorepo, you need: 1. **Testability:** Replace real DB with mock without changing service code 2. **Modularity:** Swap implementations (`StripePayment` → `BraintreePayment`) without changing callers 3. **Lifecycle management:** Centralized creation, destruction, pooling of objects 4. **Cross-cutting concerns:** Add logging/metrics/security to any bean without touching its code

Three Ways to Inject Dependencies

```
// 1. Constructor Injection — PREFERRED (immutable, testable, explicit)
@Service
public class PaymentService {
    private final PaymentRepository repository;
    private final NotificationService notificationService;
    private final KafkaProducer<String, PaymentEvent> producer;

    // Spring injects when creating PaymentService
    // If any dependency missing → fail at startup (not at runtime)
    public PaymentService(
        PaymentRepository repository,
```



```

        NotificationService notificationService,
        KafkaProducer<String, PaymentEvent> producer) {
    this.repository = repository;
    this.notificationService = notificationService;
    this.producer = producer;
    }
}

// 2. Field Injection – AVOID (not testable without Spring context)
@Service
public class BadService {
    @Autowired
    private PaymentRepository repository; // Cannot inject in unit tests without Spring
}

// 3. Setter Injection – for optional dependencies only
@Service
public class ConfigurableService {
    private MetricsClient metricsClient;

    @Autowired(required = false)
    public void setMetricsClient(MetricsClient metricsClient) {
        this.metricsClient = metricsClient;
    }
}

```

Bean Scopes

```

@Component
@Scope("singleton") // DEFAULT – one instance per Spring context
public class UserService { }

@Component
@Scope("prototype") // New instance every time it's injected/requested
public class ReportBuilder { }

@Component
@RequestScope // New instance per HTTP request (web apps)
public class RequestContext { }

@Component
@SessionScope // One instance per HTTP session
public class UserPreferences { }

```

Interview question: "When would you use prototype scope?"

Answer: Stateful beans that cannot be shared (e.g., a builder that accumulates state, a non-thread-safe processor).

3.3 Spring Bean Lifecycle

```

1. BeanDefinition read (from @Component, @Bean, XML)
2. BeanDefinition registered in BeanFactory
3. BeanFactoryPostProcessor runs (modifies bean definitions)
4. Bean instantiation (constructor called)
5. Dependency injection (fields/setters if applicable)
6. BeanPostProcessor.postProcessBeforeInitialization()
7. @PostConstruct method
8. InitializingBean.afterPropertiesSet()
9. Custom init-method
10. BeanPostProcessor.postProcessAfterInitialization()
    – AOP PROXIES ARE CREATED HERE
11. Bean is READY (in context)

...application runs...

12. @PreDestroy

```

```
13. DisposableBean.destroy()
14. Custom destroy-method
```

```
@Service
@Slf4j
public class CacheWarmingService {
    private final CacheService cache;
    private final ConfigRepository configRepo;

    public CacheWarmingService(CacheService cache, ConfigRepository configRepo) {
        this.cache = cache;
        this.configRepo = configRepo;
        // Don't load data here – dependencies may not be ready yet
    }

    @PostConstruct
    public void warmUp() {
        // ALL dependencies are injected by now – safe to use them
        log.info("Warming up configuration cache");
        configRepo.findAll().forEach(config ->
            cache.put(config.getKey(), config.getValue()));
    }

    @PreDestroy
    public void shutdown() {
        log.info("Flushing cache before shutdown");
        cache.flush();
    }
}
```

3.4 Spring MVC — Request Lifecycle

```
HTTP Request
↓
DispatcherServlet (Front Controller)
↓
HandlerMapping → find @Controller and @RequestMapping
↓
HandlerInterceptor.preHandle() (auth checks, logging)
↓
ArgumentResolver (deserialize request body, path variables, headers)
↓
@Controller method executes
↓
HandlerInterceptor.postHandle()
↓
MessageConverter (serialize response to JSON)
↓
HandlerInterceptor.afterCompletion()
↓
HTTP Response
```

Controller Best Practices

```
@RestController
@RequestMapping("/api/v1/payments")
@Validated
@Slf4j
public class PaymentController {

    private final PaymentService paymentService;

    public PaymentController(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    @PostMapping
```

```

@ResponseStatus(HttpStatus.CREATED)
public ApiResponse<PaymentResponse> createPayment(
    @Valid @RequestBody CreatePaymentRequest request,
    @RequestHeader("X-Idempotency-Key") String idempotencyKey,
    @RequestHeader("X-Request-ID") String requestId) {

    log.info("Creating payment requestId={}, idempotencyKey={}", requestId, idempotencyKey);
    PaymentResponse response = paymentService.createPayment(request, idempotencyKey);
    return ApiResponse.success(response);
}

@GetMapping("/{paymentId}")
public ApiResponse<PaymentResponse> getPayment(
    @PathVariable String paymentId,
    @RequestParam(required = false, defaultValue = "false") boolean includeHistory) {

    PaymentResponse response = paymentService.findById(paymentId, includeHistory);
    return ApiResponse.success(response);
}

@GetMapping
public Page<PaymentResponse> listPayments(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "20") int size,
    @RequestParam(required = false) String status,
    @SortDefault(sort = "createdAt", direction = Sort.Direction.DESC) Pageable pageable) {

    return paymentService.findAll(status, pageable);
}
}

```

Request/Response DTOs — Best Practices

```

// Request: validate inputs
public record CreatePaymentRequest(
    @NotBlank String fromAccountId,
    @NotBlank String toAccountId,
    @NotNull @DecimalMin("0.01") @DecimalMax("1000000.00") BigDecimal amount,
    @NotBlank @Size(min = 3, max = 3) @Pattern(regexp = "[A-Z]{3}") String currency,
    @Size(max = 500) String description,
    @NotNull @FutureOrPresent LocalDate valueDate
) {}

// Response: never expose internal fields directly
public record PaymentResponse(
    String paymentId,
    String status,
    BigDecimal amount,
    String currency,
    Instant createdAt,
    Instant completedAt
) {
    public static PaymentResponse from(Payment payment) {
        return new PaymentResponse(
            payment.getId(),
            payment.getStatus().name(),
            payment.getAmount(),
            payment.getCurrency(),
            payment.getCreatedAt(),
            payment.getCompletedAt()
        );
    }
}
}

```

3.5 Spring Security — Architecture Deep Dive

Request
↓

```

SecurityFilterChain (ordered chain of filters)
├── SecurityContextPersistenceFilter (load security context from session/JWT)
├── UsernamePasswordAuthenticationFilter (form login)
├── BearerTokenAuthenticationFilter (JWT)
├── BasicAuthenticationFilter
├── ExceptionTranslationFilter
└──
↓
FilterSecurityInterceptor → check @PreAuthorize, URL patterns
↓
Controller (if authorized)

```

JWT Authentication Configuration

```

@Configuration
@EnableWebSecurity
@EnableMethodSecurity // Enables @PreAuthorize, @PostAuthorize
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable()) // Stateless API – no CSRF needed
            .sessionManagement(session ->
                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/v1/auth/**").permitAll()
                .requestMatchers("/actuator/health").permitAll()
                .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtAuthFilter(), UsernamePasswordAuthenticationFilter.class)
            .exceptionHandling(ex -> ex
                .authenticationEntryPoint(customAuthEntryPoint())
                .accessDeniedHandler(customAccessDeniedHandler())
            )
            .build();
    }

    @Bean
    public JwtAuthenticationFilter jwtAuthFilter() {
        return new JwtAuthenticationFilter(jwtService);
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(12); // Cost factor 12 – standard for banking
    }
}

// JWT Filter
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws IOException, ServletException {
        String header = request.getHeader("Authorization");
        if (header == null || !header.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        String token = header.substring(7);
        try {
            Claims claims = jwtService.validateAndExtract(token);
            UsernamePasswordAuthenticationToken auth = new UsernamePasswordAuthenticationToken(
                claims.getSubject(),
                null,
                extractAuthorities(claims)
            );
            SecurityContextHolder.getContext().setAuthentication(auth);
        } catch {
            // Invalid token
        }
        filterChain.doFilter(request, response);
    }
}

```

```

    } catch (JwtException e) {
        response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
        return;
    }

    filterChain.doFilter(request, response);
}
}

```

Method-Level Security

```

@Service
public class AccountService {

    @PreAuthorize("hasRole('ADMIN') or @accountSecurity.isOwner(#accountId, authentication)")
    public Account getAccount(String accountId) { ... }

    @PreAuthorize("hasRole('ADMIN')")
    @PostAuthorize("returnObject.balance < 10000 or hasRole('COMPLIANCE')")
    public Account getHighValueAccount(String accountId) { ... }

    @PreFilter("filterObject.ownerId == authentication.name")
    public List<Account> processAccounts(List<Account> accounts) { ... }
}

```

3.6 Spring Data JPA — Enterprise Persistence

```

// Repository interface – Spring generates implementation at startup
public interface PaymentRepository extends JpaRepository<Payment, UUID> {

    // Method naming → SQL generation (JPQL)
    List<Payment> findByStatusAndCreatedAtAfter(PaymentStatus status, Instant cutoff);

    Optional<Payment> findByIdempotencyKey(String idempotencyKey);

    // Custom JPQL
    @Query("SELECT p FROM Payment p WHERE p.accountId = :accountId AND p.amount > :threshold")
    List<Payment> findLargePayments(@Param("accountId") String accountId,
                                    @Param("threshold") BigDecimal threshold);

    // Native SQL (when you need DB-specific features)
    @Query(value = """
        SELECT * FROM payments
        WHERE account_id = :accountId
        AND created_at >= NOW() - INTERVAL '30 days'
        ORDER BY amount DESC
        LIMIT :limit
        """, nativeQuery = true)
    List<Payment> findRecentLargeNative(@Param("accountId") String accountId,
                                       @Param("limit") int limit);

    // Projection – fetch only needed columns (avoid N+1 and over-fetching)
    List<PaymentSummary> findByAccountId(String accountId);

    // Streaming for large datasets (avoid loading all into memory)
    @QueryHints(@QueryHint(name = HINT_FETCH_SIZE, value = "500"))
    Stream<Payment> findAllByStatus(PaymentStatus status);

    // Paging
    Page<Payment> findByAccountId(String accountId, Pageable pageable);

    // Exists check (more efficient than findBy + null check)
    boolean existsByIdempotencyKey(String idempotencyKey);

    // Count
    long countByStatusAndCreatedAtAfter(PaymentStatus status, Instant cutoff);
}

```

JPA Entity Best Practices

```
@Entity
@Table(name = "payments",
      indexes = {
        @Index(name = "idx_payments_account_id", columnList = "account_id"),
        @Index(name = "idx_payments_idempotency", columnList = "idempotency_key", unique = true),
        @Index(name = "idx_payments_status_created", columnList = "status, created_at")
      })
@EntityListeners(AuditingEntityListener.class) // Automatic audit fields
public class Payment {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @Column(name = "account_id", nullable = false, length = 50)
    private String accountId;

    @Column(nullable = false, precision = 19, scale = 4)
    private BigDecimal amount;

    @Column(length = 3, nullable = false)
    @Enumerated(EnumType.STRING) // Store as "PENDING" not 0
    private PaymentStatus status;

    @Column(name = "idempotency_key", unique = true, length = 100)
    private String idempotencyKey;

    @CreatedDate // Spring Data Auditing
    @Column(name = "created_at", nullable = false, updatable = false)
    private Instant createdAt;

    @LastModifiedDate
    @Column(name = "updated_at")
    private Instant updatedAt;

    @Version // Optimistic locking – prevents lost updates in concurrent scenarios
    private Long version;

    // Relationships
    @ManyToOne(fetch = FetchType.LAZY) // ALWAYS use LAZY for ManyToOne
    @JoinColumn(name = "account_id", insertable = false, updatable = false)
    private Account account;

    @OneToMany(mappedBy = "payment",
              cascade = CascadeType.ALL,
              orphanRemoval = true,
              fetch = FetchType.LAZY)
    private List<PaymentItem> items = new ArrayList<>();
}
```

3.7 Aspect-Oriented Programming (AOP)

AOP is how Spring adds cross-cutting concerns (logging, security, transactions, caching) without polluting business code.

```
Without AOP:
PaymentService.transfer() {
    log.info("starting transfer"); // logging
    checkAuth();                  // security
    startTransaction();           // transaction
    doTransfer();                  // business logic
    commitTransaction();          // transaction
    updateCache();                 // caching
    logAudit();                   // audit
    emitMetrics();                 // metrics
}
```

```

With AOP:
PaymentService.transfer() {
    doTransfer(); // ONLY business logic
}
// All cross-cutting concerns added by Spring AOP interceptors

```

AOP Concepts

```

@Aspect
@Component
@Slf4j
public class PerformanceMonitorAspect {

    // Pointcut: match all methods in service layer
    @Pointcut("within(@org.springframework.stereotype.Service *)")
    public void serviceLayer() {}

    // Around advice: wraps method execution
    @Around("serviceLayer()")
    public Object measureTime(ProceedingJoinPoint joinPoint) throws Throwable {
        String methodName = joinPoint.getSignature().toShortString();
        long start = System.currentTimeMillis();
        try {
            Object result = joinPoint.proceed(); // Execute actual method
            long duration = System.currentTimeMillis() - start;
            log.debug("Method {} took {}ms", methodName, duration);
            meterRegistry.timer("service.method.duration",
                "method", methodName).record(duration, TimeUnit.MILLISECONDS);
            return result;
        } catch (Exception e) {
            meterRegistry.counter("service.method.errors",
                "method", methodName, "exception", e.getClass().getSimpleName()).increment();
            throw e;
        }
    }
}

// AOP for audit logging
@Aspect
@Component
public class AuditAspect {

    @AfterReturning(
        pointcut = "@annotation(auditLog)",
        returning = "result"
    )
    public void logAudit(JoinPoint joinPoint, AuditLog auditLog, Object result) {
        String userId = SecurityContextHolder.getContext()
            .getAuthentication().getName();
        auditLogRepository.save(new AuditEntry(
            userId, auditLog.action(), auditLog.resource(), Instant.now()
        ));
    }
}

```

@Transactional — AOP in Action

`@Transactional` is implemented via AOP. Spring creates a proxy around your bean and intercepts calls to transactional methods.

```

@Service
public class TransferService {

    @Transactional(
        isolation = Isolation.SERIALIZABLE, // Highest isolation for transfers
        propagation = Propagation.REQUIRED, // Join existing or create new tx
        rollbackFor = Exception.class,      // Rollback on ANY exception
        timeout = 30                         // Fail if transaction > 30 seconds
    )
    public void transfer(String fromId, String toId, BigDecimal amount) {

```

```

        Account from = accountRepo.findByIdWithLock(fromId); // SELECT FOR UPDATE
        Account to = accountRepo.findById(toId).orElseThrow();

        if (from.getBalance().compareTo(amount) < 0) {
            throw new InsufficientFundsException(amount, from.getBalance());
        }

        from.debit(amount);
        to.credit(amount);

        accountRepo.save(from);
        accountRepo.save(to);
        // On method exit: Spring commits transaction
        // On exception: Spring rolls back
    }
}

// COMMON @Transactional PITFALLS:
// 1. Self-invocation – calling @Transactional method from same class bypasses proxy
// 2. Rollback on checked exceptions – by default only RuntimeException causes rollback
// 3. @Transactional on private methods – ignored (proxy can't intercept)

```

3.8 Spring WebFlux — Reactive Programming

WebFlux is Spring's reactive framework — like Node.js event loop but explicit:

```

// Node.js: async/await is implicit non-blocking
async function getUser(id) {
    const user = await db.findUser(id);
    return user;
}

// Spring WebFlux: explicit reactive types
@GetMapping("/users/{id}")
public Mono<User> getUser(@PathVariable String id) {
    return userRepository.findById(id) // Reactive repo
        .switchIfEmpty(Mono.error(new UserNotFoundException(id)));
}

// Multiple async operations
@GetMapping("/dashboard/{userId}")
public Mono<Dashboard> getDashboard(@PathVariable String userId) {
    Mono<User> userMono = userRepo.findById(userId);
    Mono<List<Account>> accountsMono = accountRepo.findById(userId);
    Mono<List<Transaction>> txMono = transactionRepo.findRecentByUser(userId, 10);

    return Mono.zip(userMono, accountsMono, txMono)
        .map(tuple -> new Dashboard(tuple.getT1(), tuple.getT2(), tuple.getT3()));
    // All three DB calls execute in parallel!
}

// Streaming large datasets
@GetMapping(value = "/reports/transactions", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<Transaction> streamTransactions() {
    return transactionRepo.findAllAsFlux() // Don't load all into memory
        .delayElements(Duration.ofMillis(10)); // Backpressure-aware
}

```

When to Use WebFlux vs MVC

```

Use Spring MVC (blocking) when:
- CRUD apps with traditional JDBC
- Team is familiar with blocking model
- Using blocking libraries (most Spring Data JDBC/JPA)
- Simpler code, easier debugging

Use Spring WebFlux (reactive) when:
- High concurrency with many I/O-bound operations

```


- Streaming large datasets to clients
- Integrating with reactive systems (Cassandra, MongoDB reactive, R2DBC)
- Building API gateways

3.9 Spring Configuration — Enterprise Patterns

```
# application.yml – base config
spring:
  application:
    name: payment-service
  profiles:
    active: ${APP_ENV:local}

# application-prod.yml – production overrides
spring:
  datasource:
    url: ${DB_URL}           # Always externalize sensitive config
    username: ${DB_USER}
    password: ${DB_PASSWORD}
  jpa:
    hibernate:
      ddl-auto: validate     # NEVER use create/update in production!
      show-sql: false

# application-local.yml – local dev
spring:
  datasource:
    url: jdbc:h2:mem:testdb  # In-memory DB for local
  h2:
    console.enabled: true
```

Type-Safe Configuration

```
@ConfigurationProperties(prefix = "payment")
@Validated
@Data
public class PaymentProperties {

    @NotNull
    private Limits limits;

    @NotBlank
    private String processorUrl;

    @Min(1000) @Max(30000)
    private int timeoutMs = 5000;

    @Data
    public static class Limits {
        @DecimalMin("0.01")
        private BigDecimal maxSinglePayment;

        @DecimalMin("0.01")
        private BigDecimal dailyLimit;
    }
}

// payment.limits.max-single-payment=10000.00
// payment.limits.daily-limit=100000.00
// payment.timeout-ms=8000
// payment.processor-url=https://payments.internal.bank.com
```

Section Summary: Spring Interview Essentials

Must-know Spring concepts:

1. **DI container internals** — how beans are created, wired, scoped
2. **AOP proxy model** — `@Transactional` pitfalls (self-invocation, visibility)
3. **Request lifecycle** — `DispatcherServlet` → `Interceptors` → `Controller`
4. **Security filter chain** — JWT validation, `SecurityContextHolder`
5. **JPA patterns** — N+1 problem, lazy loading, optimistic locking
6. **@Transactional isolation levels** — when each level is appropriate
7. **Spring Boot autoconfiguration** — how it works, how to override it
8. **Profiles** — managing config across environments

Most common Spring bug in interviews: "What happens when `@Transactional` calls another `@Transactional` method in the same class?"

Answer: The inner method's transaction settings are IGNORED because self-invocation bypasses the AOP proxy. You must inject the bean into itself or refactor.

Section 4: Production Backend Engineering

The Real World: Knowing Spring Boot syntax gets you an interview. Knowing how to build systems that handle 100k RPS with 99.9% availability, survive partial failures, and can be debugged at 3 AM gets you the senior/staff role at Stripe, Goldman Sachs, or Google.

4.1 Microservices Architecture — Enterprise Reality

From Monolith to Microservices



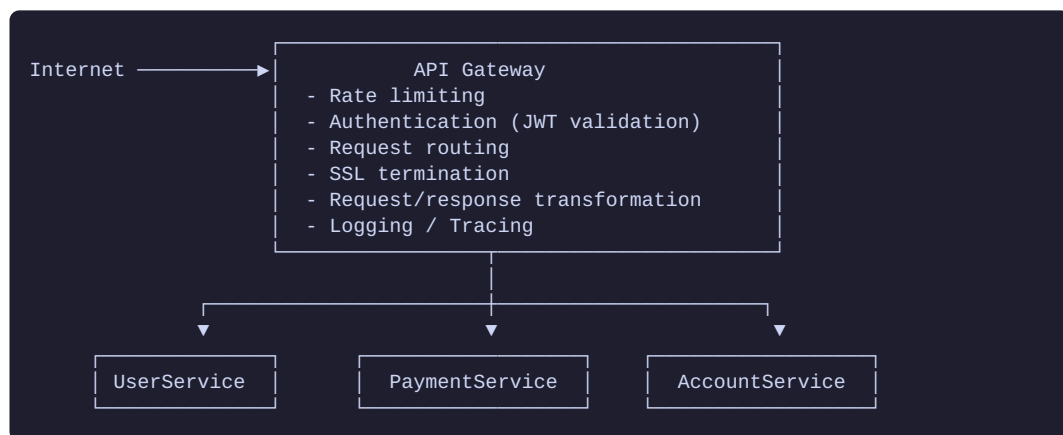
Service Decomposition — When to Split

Good split criteria: - Independent scalability requirements (payment service needs 10x more CPU than user service) - Independent deployment frequency (notifications deploy daily, core banking weekly) - Different team ownership - Different compliance/security boundaries (PCI scope isolation)

Premature decomposition problems: - Distributed transaction complexity - Network latency on what was method calls - Operational overhead of 50 services - Service-to-service versioning hell

Bank reality: Most enterprise Java shops run large "macro-services" (5-15 bounded contexts), not 100s of nano-services.

4.2 API Gateway Pattern



Spring Cloud Gateway Configuration

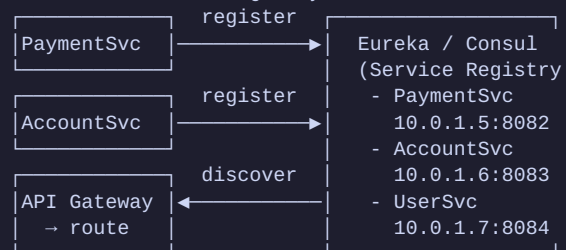
```
@Configuration
public class GatewayConfig {

    @Bean
    public RouteLocator routeLocator(RouteLocatorBuilder builder) {
        return builder.routes()
            .route("payment-service", r -> r
                .path("/api/v1/payments/**")
                .filters(f -> f
                    .stripPrefix(2)
                    .addRequestHeader("X-Internal-Source", "gateway")
                    .retry(retryConfig -> retryConfig
                        .setRetries(3)
                        .setStatuses(HttpStatus.SERVICE_UNAVAILABLE)
                        .setMethods(HttpMethod.GET))
                    .circuitBreaker(cb -> cb
                        .setName("paymentCB")
                        .setFallbackUri("forward:/fallback/payment")))
                .uri("lb://payment-service")) // Load-balanced via service discovery
            .build();
    }
}
```

4.3 Service Discovery

Problem: In k8s/cloud, service IPs change. How does Service A find Service B?

Solution: Service Registry



In Kubernetes: kube-dns handles this natively
payment-service.default.svc.cluster.local → Service IP

Spring Cloud Netflix Eureka

```
// Server
@SpringBootApplication
@EnableEurekaServer
public class DiscoveryServer { ... }

// Client (every microservice)
@SpringBootApplication
@EnableEurekaClient
public class PaymentService { ... }

# application.yml
eureka:
  client:
    service-url:
      defaultZone: http://eureka:8761/eureka
    register-with-eureka: true
    fetch-registry: true
  instance:
    prefer-ip-address: true
    health-check-url-path: /actuator/health
```

4.4 Circuit Breaker Pattern

The Problem: Service B is slow/failing. Service A keeps calling it. Service A's threads fill up waiting. Service A dies too. **Cascading failure.**

The Solution: Circuit breaker opens after N failures. Fast-fail until service recovers.

```
Circuit Breaker States:
CLOSED (normal) → calls pass through, track failures
    ↓ failure rate > threshold
OPEN (tripped) → fast fail, no calls to downstream
    ↓ after wait window
HALF-OPEN → allow test call
    ↓ test call succeeds → CLOSED
    ↓ test call fails → OPEN
```

```
// Resilience4j – industry standard for Java
@Configuration
public class Resilience4jConfig {

    @Bean
    public CircuitBreakerConfig paymentCircuitBreakerConfig() {
        return CircuitBreakerConfig.custom()
            .failureRateThreshold(50)           // Open if 50% fail
            .slowCallRateThreshold(80)         // Open if 80% are slow
            .slowCallDurationThreshold(Duration.ofSeconds(2))
            .waitDurationInOpenState(Duration.ofSeconds(30))
            .permittedNumberOfCallsInHalfOpenState(3)
            .slidingWindowSize(10)             // Based on last 10 calls
            .recordExceptions(IOException.class, TimeoutException.class)
            .build();
    }
}

@Service
public class PaymentGatewayClient {

    @CircuitBreaker(name = "payment-gateway", fallbackMethod = "fallbackProcess")
    @Retry(name = "payment-gateway")
    @TimeLimiter(name = "payment-gateway")
    public CompletableFuture<PaymentResult> processPayment(PaymentRequest request) {
        return CompletableFuture.supplyAsync(() ->
            httpClient.post(gatewayUrl, request, PaymentResult.class));
    }

    // Fallback – executed when circuit is open or retries exhausted
    public CompletableFuture<PaymentResult> fallbackProcess(
        PaymentRequest request, Exception e) {
        log.warn("Payment gateway unavailable, queuing for retry: {}", e.getMessage());
        // Queue to Kafka for async retry
        kafkaTemplate.send("payment.retry.queue", request);
        return CompletableFuture.completedFuture(
            PaymentResult.pending("Payment queued for processing"));
    }
}
```

4.5 Retry Mechanisms — Enterprise Patterns

```
// Retry with exponential backoff + jitter (prevents thundering herd)
@Bean
public RetryConfig retryConfig() {
    return RetryConfig.custom()
        .maxAttempts(3)
        .waitDuration(Duration.ofMillis(500))
        .intervalFunction(IntervalFunction.ofExponentialRandomBackoff(
            500, // initial interval ms

```

```

        2.0,        // multiplier
        0.5,        // randomization factor (jitter)
        30000       // max interval ms
    ))
    .retryExceptions(IOException.class, TimeoutException.class)
    .ignoreExceptions(BusinessValidationException.class) // Don't retry validation errors
    .build();
}

// Spring Retry annotation approach
@Service
public class ExternalApiClient {

    @Retryable(
        value = {RestClientException.class},
        maxAttempts = 3,
        backoff = @Backoff(delay = 1000, multiplier = 2, random = true)
    )
    public ExternalResponse call(Request request) {
        return restTemplate.postForObject(url, request, ExternalResponse.class);
    }

    @Recover
    public ExternalResponse fallback(RestClientException ex, Request request) {
        // Called after all retries exhausted
        alertingService.raiseAlert("External API unavailable: " + ex.getMessage());
        throw new ExternalServiceUnavailableException(request.getId());
    }
}

```

4.6 Distributed Tracing — Observability

Request flow across services:

```

[Browser] → [API Gateway] → [User Service] → [Payment Service] → [Database]
                        ↓
                    All share: traceId = "abc-123"
                    Each span has own: spanId

```

Trace: abc-123

```

├─ Span 1: API Gateway (5ms)
├─ Span 2: User Service validation (15ms)
├─ Span 3: Payment Service (120ms)
│   └─ Span 4: DB query (45ms)
│       └─ Span 5: Kafka publish (8ms)

```

```

// Spring Boot 3 + Micrometer Tracing (replaces Sleuth)
// Auto-configured with spring-boot-starter-actuator + micrometer-tracing-bridge-otel

@Service
@Slf4j
public class PaymentService {

    private final Tracer tracer;

    public PaymentResult processPayment(PaymentRequest request) {
        // Current span auto-populated from MDC
        log.info("Processing payment amount={}" currency={}, // Logs include traceId automatically
            request.getAmount(), request.getCurrency());

        Span dbSpan = tracer.nextSpan().name("db.findAccount").start();
        try (Tracer.SpanInScope ws = tracer.withSpan(dbSpan)) {
            Account account = accountRepo.findById(request.getAccountId());
            return processWithAccount(account, request);
        } finally {
            dbSpan.end();
        }
    }
}

```

```
// application.yml
management:
  tracing:
    sampling:
      probability: 0.1 # Sample 10% in production (100% in dev)
  zipkin:
    tracing:
      endpoint: http://zipkin:9411/api/v2/spans
```

4.7 Logging — Production Standards

```
// Structured logging with SLF4J + Logback
// ALWAYS use parameterized logging – NEVER string concatenation
log.info("Payment processed: amount={} currency={} accountId={}",
    amount, currency, accountId);
// NOT: log.info("Payment processed: amount=" + amount + "...");

// MDC (Mapped Diagnostic Context) – correlate logs across calls
// Set at request entry point (filter/interceptor)
MDC.put("traceId", request.getHeader("X-Trace-ID"));
MDC.put("userId", authentication.getName());
MDC.put("requestId", UUID.randomUUID().toString());
// MDC values appear in every log line for this thread
// MUST clear after request: MDC.clear() in finally block

// Logging levels – production strategy
// ERROR: System errors that need immediate attention (PagerDuty alert)
// WARN: Unexpected but handled (retry, fallback activated)
// INFO: Business events (payment created, order completed)
// DEBUG: Technical details (turned off in production)
// TRACE: Very detailed (never in production)
```

Logback Production Configuration

```
<!-- logback-spring.xml -->
<configuration>
  <springProfile name="prod">
    <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
      <encoder class="net.logstash.logback.encoder.LogstashEncoder">
        <!-- JSON output for log aggregation (ELK/Splunk/Datadog) -->
        <includeMdcKeyName>traceId</includeMdcKeyName>
        <includeMdcKeyName>userId</includeMdcKeyName>
        <includeMdcKeyName>requestId</includeMdcKeyName>
      </encoder>
    </appender>

    <root level="INFO">
      <appender-ref ref="STDOUT"/>
    </root>

    <!-- Reduce Spring framework noise -->
    <logger name="org.springframework" level="WARN"/>
    <logger name="org.hibernate" level="WARN"/>
    <!-- Your app at INFO/DEBUG as needed -->
    <logger name="com.bank.payment" level="INFO"/>
  </springProfile>
</configuration>
```

4.8 Monitoring and Observability — Metrics

Spring Actuator + Micrometer + Prometheus

```
// Auto-configured metrics with Spring Boot Actuator

// Custom business metrics
@Service
public class PaymentMetricsService {

    private final MeterRegistry meterRegistry;
    private final Counter paymentSuccessCounter;
    private final Counter paymentFailureCounter;
    private final Timer paymentProcessingTimer;
    private final Gauge activePaymentsGauge;

    public PaymentMetricsService(MeterRegistry meterRegistry,
                                PaymentRepository repo) {
        this.meterRegistry = meterRegistry;

        this.paymentSuccessCounter = Counter.builder("payment.processed")
            .description("Number of successfully processed payments")
            .tag("status", "success")
            .register(meterRegistry);

        this.paymentFailureCounter = Counter.builder("payment.processed")
            .tag("status", "failure")
            .register(meterRegistry);

        this.paymentProcessingTimer = Timer.builder("payment.processing.duration")
            .description("Time to process a payment")
            .publishPercentiles(0.5, 0.95, 0.99) // p50, p95, p99 latencies
            .register(meterRegistry);

        // Gauge reads live value each time Prometheus scrapes
        Gauge.builder("payment.pending.count", repo, PaymentRepository::countPending)
            .description("Number of pending payments")
            .register(meterRegistry);
    }

    public void recordPayment(BigDecimal amount, Duration duration, boolean success) {
        paymentProcessingTimer.record(duration);
        if (success) paymentSuccessCounter.increment();
        else paymentFailureCounter.increment();

        meterRegistry.summary("payment.amount.distribution")
            .record(amount.doubleValue());
    }
}
```

Health Checks — Production-Grade

```
@Component
public class PaymentGatewayHealthIndicator implements HealthIndicator {

    @Override
    public Health health() {
        try {
            ResponseEntity<String> response = restTemplate
                .getForEntity(gatewayUrl + "/health", String.class);

            if (response.getStatusCode().is2xxSuccessful()) {
                return Health.up()
                    .withDetail("gateway", "available")
                    .withDetail("responseTime", measureLatency())
                    .build();
            } else {
                return Health.down()
                    .withDetail("status", response.getStatusCode())
                    .build();
            }
        }
    }
}
```



```

    }
    } catch (Exception e) {
        return Health.down()
            .withDetail("error", e.getMessage())
            .build();
    }
}

# application.yml
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus,loggers
  endpoint:
    health:
      show-details: when_authorized
  probes:
    enabled: true # /health/liveness, /health/readiness for k8s

```

4.9 Rate Limiting

```

// Bucket4j – token bucket algorithm
@Configuration
public class RateLimitConfig {

    @Bean
    public Map<String, Bucket> rateLimiters() {
        Map<String, Bucket> buckets = new ConcurrentHashMap<>();
        return buckets;
    }
}

@Component
public class RateLimitFilter extends OncePerRequestFilter {

    private final Map<String, Bucket> rateLimiters;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain chain) throws IOException, ServletException {
        String clientId = extractClientId(request);

        Bucket bucket = rateLimiters.computeIfAbsent(clientId, this::createBucket);

        ConsumptionProbe probe = bucket.tryConsumeAndReturnRemaining(1);
        if (probe.isConsumed()) {
            response.setHeader("X-Rate-Limit-Remaining",
                              String.valueOf(probe.getRemainingTokens()));
            chain.doFilter(request, response);
        } else {
            long retryAfterSeconds = TimeUnit.NANOSECONDS.toSeconds(
                probe.getNanosToWaitForRefill());
            response.setHeader("Retry-After", String.valueOf(retryAfterSeconds));
            response.setHeader("X-Rate-Limit-Remaining", "0");
            response.sendError(HttpStatus.TOO_MANY_REQUESTS.value(),
                              "Rate limit exceeded");
        }
    }

    private Bucket createBucket(String clientId) {
        return Bucket.builder()
            .addLimit(Bandwidth.classic(100, Refill.intervally(100, Duration.ofMinutes(1))))
            .build();
    }
}

```

4.10 Idempotency — Critical for Financial Systems

```
// Idempotency ensures duplicate requests have no side effects
// Essential for: payment APIs, order creation, booking systems

@Service
@Transactional
public class IdempotentPaymentService {

    private final PaymentRepository paymentRepo;
    private final IdempotencyKeyRepository idempotencyRepo;

    public PaymentResponse createPayment(CreatePaymentRequest request,
                                         String idempotencyKey) {
        // Check if we've processed this request before
        Optional<IdempotencyRecord> existing =
            idempotencyRepo.findByKey(idempotencyKey);

        if (existing.isPresent()) {
            IdempotencyRecord record = existing.get();
            if (record.isCompleted()) {
                // Return cached response – safe to repeat
                return objectMapper.readValue(record.getResponse(), PaymentResponse.class);
            } else if (record.isProcessing()) {
                throw new ConflictException("Payment is already being processed");
            }
        }

        // Lock the idempotency key (prevent concurrent duplicate requests)
        idempotencyRepo.save(new IdempotencyRecord(idempotencyKey, "PROCESSING"));

        try {
            PaymentResponse response = processPaymentInternal(request);
            // Save response for future duplicate requests
            idempotencyRepo.updateCompleted(idempotencyKey,
                objectMapper.writeValueAsString(response));
            return response;
        } catch (Exception e) {
            idempotencyRepo.updateFailed(idempotencyKey);
            throw e;
        }
    }
}
```

4.11 Event-Driven Architecture with Spring

```
// Spring Events – within single application
@Service
public class OrderService {

    private final ApplicationEventPublisher publisher;

    public Order createOrder(CreateOrderRequest request) {
        Order order = orderRepo.save(new Order(request));
        // Publish event – other services within same app react
        publisher.publishEvent(new OrderCreatedEvent(order));
        return order;
    }
}

@Component
public class NotificationHandler {

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    // AFTER_COMMIT – only notify if order transaction actually committed
    public void onOrderCreated(OrderCreatedEvent event) {
        emailService.sendOrderConfirmation(event.getOrder());
    }
}
```

```
// Kafka-based cross-service events (see Section 6 for deep dive)
@Service
public class PaymentEventPublisher {

    private final KafkaTemplate<String, PaymentEvent> kafkaTemplate;

    public void publishPaymentCompleted(Payment payment) {
        PaymentEvent event = PaymentEvent.completed(payment);
        kafkaTemplate.send("payments.completed", payment.getId(), event)
            .addCallback(
                result -> log.info("Event published: {}", payment.getId()),
                failure -> {
                    log.error("Failed to publish event", failure);
                    // Outbox pattern: save to outbox table, retry via scheduler
                    outboxRepo.save(new OutboxMessage(event));
                }
            );
    }
}
```

Section Summary: Production Readiness Checklist

Every production Spring service should have:

- ☐ Health endpoints (liveness + readiness probes)
- ☐ Structured JSON logging with MDC (traceId, requestId, userId)
- ☐ Distributed tracing (Micrometer/OpenTelemetry)
- ☐ Business metrics (custom Micrometer counters/timers/gauges)
- ☐ Circuit breakers on all external service calls
- ☐ Retry with exponential backoff + jitter
- ☐ Rate limiting on public endpoints
- ☐ Idempotency for mutating operations
- ☐ Graceful shutdown (spring.lifecycle.timeout-per-shutdown-phase)
- ☐ Thread pool configuration (don't use defaults in production)
- ☐ Connection pool sizing (HikariCP, not defaults)
- ☐ Timeout configuration on all HTTP clients
- ☐ Sensitive config externalized (never in code or git)
- ☐ Security headers (Content-Security-Policy, HSTS, etc.)
- ☐ Input validation on all API endpoints
- ☐ Audit logging for security-sensitive operations

Section 5: Databases & Persistence

Enterprise Reality: Database performance and correctness separate junior from senior engineers. At banks and FAANG, every senior interview includes transaction isolation, deadlocks, N+1 queries, and connection pool sizing. If you can't explain `SERIALIZABLE` vs `READ COMMITTED` and when to use each, you're not ready for the room.

5.1 Transaction Isolation Levels — Deep Dive

The Four Problems Transactions Solve

1. Dirty Read: Read uncommitted data from another transaction
2. Non-Repeatable Read: Row read twice within same transaction returns different data
3. Phantom Read: Query returns different set of rows when repeated (rows added/deleted)
4. Lost Update: Two transactions both read-modify-write, one overwrites the other

Isolation Levels vs Problems

	Dirty Read	Non-Repeatable Read	Phantom Read	Lost Update
READ UNCOMMITTED	✓ Possible	✓ Possible	✓ Possible	✓ Possible
READ COMMITTED	x Prevented	✓ Possible	✓ Possible	✓ Possible
REPEATABLE READ	x x Prevented		✓ Possible	x Prevented
SERIALIZABLE	x x		x Prevented	x Prevented

✓ = problem can occur x = prevented by this level

Real Enterprise Usage

```
// READ_COMMITTED – DEFAULT for most operations
// Performance: Good | Use: CRUD, reads, most writes
@Transactional(isolation = Isolation.READ_COMMITTED)
public UserProfile updateProfile(String userId, UpdateRequest request) { ... }

// REPEATABLE_READ – Balances, running totals
// Performance: Moderate | Use: Balance checks, inventory
@Transactional(isolation = Isolation.REPEATABLE_READ)
public void checkAndReserveInventory(String productId, int quantity) {
    Product product = repo.findById(productId); // Row locked for this transaction
    // product.quantity won't change even if another transaction updates it
    if (product.getQuantity() >= quantity) {
        product.setQuantity(product.getQuantity() - quantity);
        repo.save(product);
    }
}

// SERIALIZABLE – Financial transfers, double-spend prevention
// Performance: Slowest | Use: Money movement, compliance-critical
@Transactional(isolation = Isolation.SERIALIZABLE)
public void transferFunds(String fromId, String toId, BigDecimal amount) {
    // No concurrent transaction can see partial state
    Account from = accountRepo.findById(fromId);
    Account to = accountRepo.findById(toId);
    // Debit and credit atomically, serialized with all other transfers
}
```

5.2 Database Locking

Optimistic vs Pessimistic Locking

Optimistic Locking:

- No locks held during read
- At write time: check version hasn't changed
- If version conflict → retry
- Use when: Low contention, long-lived operations, distributed reads
- In JPA: @Version annotation

Pessimistic Locking:

- Acquire lock immediately at read time (SELECT FOR UPDATE)
- Hold lock until transaction commits
- Blocking – other transactions wait
- Use when: High contention, MUST succeed, short operations

```
// Optimistic locking with @Version
@Entity
public class Account {
    @Id private UUID id;
    private BigDecimal balance;

    @Version // JPA adds WHERE version = :expectedVersion on UPDATE
    private Long version;
}

// If two threads both read account at version=5 and both try to update:
// Thread 1: UPDATE accounts SET balance=900, version=6 WHERE id=? AND version=5 → succeeds
// Thread 2: UPDATE accounts SET balance=850, version=6 WHERE id=? AND version=5 → 0 rows → OptimisticLockException

// Handle optimistic lock conflicts:
@Retryable(value = OptimisticLockingFailureException.class, maxAttempts = 3)
@Transactional
public void transfer(String fromId, String toId, BigDecimal amount) {
    // Spring-Retry will retry this method on version conflict
}

// Pessimistic locking with JPQL
@Repository
public interface AccountRepository extends JpaRepository<Account, UUID> {

    @Lock(LockModeType.PESSIMISTIC_WRITE) // SELECT ... FOR UPDATE
    @Query("SELECT a FROM Account a WHERE a.id = :id")
    Optional<Account> findByIdForUpdate(@Param("id") UUID id);

    @Lock(LockModeType.PESSIMISTIC_READ) // SELECT ... FOR SHARE
    @Query("SELECT a FROM Account a WHERE a.accountNumber = :num")
    Optional<Account> findByAccountNumberForRead(@Param("num") String num);
}
```

Deadlock Prevention

Deadlock scenario:

Thread 1: Lock Account A → waiting for Account B
Thread 2: Lock Account B → waiting for Account A
→ Deadlock!

Prevention strategy: ALWAYS acquire locks in the same order

```
@Transactional(isolation = Isolation.SERIALIZABLE)
public void transfer(String fromId, String toId, BigDecimal amount) {
    // Sort IDs to ensure consistent lock acquisition order
    // Prevents deadlock between concurrent transfers
    List<String> sortedIds = List.of(fromId, toId).stream()
        .sorted()
        .toList();
```

```

// Acquire locks in deterministic order
Account first = accountRepo.findByIdForUpdate(UUID.fromString(sortedIds.get(0)));
Account second = accountRepo.findByIdForUpdate(UUID.fromString(sortedIds.get(1)));

Account from = fromId.equals(sortedIds.get(0)) ? first : second;
Account to = toId.equals(sortedIds.get(0)) ? first : second;

from.debit(amount);
to.credit(amount);
}

```

5.3 Connection Pooling — HikariCP

HikariCP is the Spring Boot default and fastest Java connection pool. Sizing it correctly is critical:

```

@Configuration
public class DataSourceConfig {

    @Bean
    @ConfigurationProperties("spring.datasource.hikari")
    public DataSource dataSource() {
        HikariDataSource ds = new HikariDataSource();
        ds.setJdbcUrl(dbUrl);
        ds.setUsername(dbUser);
        ds.setPassword(dbPassword);

        // Pool sizing formula: connections = (core_count * 2) + effective_spindle_count
        // For 4-core server with SSD: ~9 connections per pod
        ds.setMaximumPoolSize(10);
        ds.setMinimumIdle(5);

        // Timeouts (critical for production)
        ds.setConnectionTimeout(30000); // 30s wait for available connection
        ds.setIdleTimeout(600000); // 10min: remove idle connections
        ds.setMaxLifetime(1800000); // 30min: force connection refresh
        ds.setKeepaliveTime(60000); // Ping every 60s to prevent firewall timeout

        // Validation
        ds.setConnectionTestQuery("SELECT 1");

        // Metrics
        ds.setMetricRegistry(meterRegistry);

        return ds;
    }
}

# application.yml equivalent
spring:
  datasource:
    hikari:
      maximum-pool-size: 10
      minimum-idle: 5
      connection-timeout: 30000
      idle-timeout: 600000
      max-lifetime: 1800000
      keepalive-time: 60000
      pool-name: PaymentServicePool

```

Connection Pool Exhaustion — Diagnosing in Production

```

Symptoms: Requests hang or return 500 with "Unable to acquire JDBC Connection"
Causes:
1. Pool too small for load
2. Long-running transactions holding connections
3. Missing connection.close() (resource leak)
4. Database is slow (connections busy waiting for DB response)

```

```

Diagnosis:
- Check HikariCP metrics: hikaricp.connections.active vs hikaricp.connections.max
- Thread dump: many threads in "waiting for connection" state
- Trace slow queries: look for queries > 100ms that hold connections

Fix:
1. Identify slow queries → optimize or add index
2. Ensure @Transactional transactions are short-lived
3. Increase pool size (but beware DB connection limit)
4. Use connection leak detection: spring.datasource.hikari.leak-detection-threshold=2000

```

5.4 ORM Internals — Hibernate Deep Dive

The N+1 Problem — Most Common JPA Bug

```

// N+1 problem:
List<Order> orders = orderRepo.findAll(); // 1 query: SELECT * FROM orders
for (Order order : orders) {
    order.getCustomer().getName(); // N queries: SELECT * FROM customers WHERE id=?
}
// For 1000 orders = 1001 database queries!

// Solution 1: JOIN FETCH in JPQL
@Query("SELECT o FROM Order o JOIN FETCH o.customer WHERE o.status = :status")
List<Order> findByStatusWithCustomer(@Param("status") OrderStatus status);

// Solution 2: @EntityGraph
@EntityGraph(attributePaths = {"customer", "items", "items.product"})
List<Order> findByStatus(OrderStatus status);

// Solution 3: @BatchSize for collections
@OneToMany(mappedBy = "order")
@BatchSize(size = 100) // Load 100 at a time instead of 1
private List<OrderItem> items;

// Solution 4: DTO projection (best for read-only views)
@Query("""
    SELECT new com.example.dto.OrderSummaryDto(
        o.id, o.createdAt, c.name, SUM(i.price * i.quantity))
    FROM Order o
    JOIN o.customer c
    JOIN o.items i
    WHERE o.status = :status
    GROUP BY o.id, o.createdAt, c.name
    """)
List<OrderSummaryDto> findOrderSummaries(@Param("status") OrderStatus status);

```

Hibernate Session and Persistence Context

```

Persistence Context (first-level cache) = Map<EntityKey, Entity> per Session

When you call repo.findById(id):
1. Check persistence context (first-level cache)
2. If found → return cached version (no DB query!)
3. If not found → query DB, store in cache, return

This is why:
Entity a = repo.findById(1); // DB query
Entity b = repo.findById(1); // Returns SAME object, no DB query
a == b // true – same object reference!

Implications:
- Within same @Transactional method, reading same entity twice = one DB hit
- dirty checking: Hibernate tracks entity state changes automatically
  → if you change a field of a managed entity, Hibernate will UPDATE on commit
  → no need to explicitly call save() for changes within transaction!

```

Hibernate Caching — Second Level Cache

```
// L2 Cache: Shared across sessions, reduces DB hits
@Entity
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE) // Cacheable entity
public class Currency {
    @Id private String code;
    private String name;
    private int decimalPlaces;
    // Reference data – rarely changes, read millions of times
}

@Repository
public interface CurrencyRepository extends JpaRepository<Currency, String> {
    @Cacheable("currencies") // Spring Cache abstraction over L2
    Optional<Currency> findByCode(String code);
}

# application.yml
spring:
  jpa:
    properties:
      hibernate:
        cache:
          use_second_level_cache: true
          use_query_cache: true
          region.factory_class: org.hibernate.cache.jcache.JCacheRegionFactory
          javax.cache.missing_cache_strategy: create
```

5.5 SQL Optimization

Index Strategy

```
-- B-tree index (default) – range queries, =, >, <, BETWEEN, LIKE 'prefix%'
CREATE INDEX idx_payments_account_created ON payments(account_id, created_at DESC);

-- Covering index – all query columns in index (no table lookup)
CREATE INDEX idx_payments_status_covering
ON payments(status, created_at)
INCLUDE (amount, currency); -- PostgreSQL syntax

-- Partial index – index only subset of rows
CREATE INDEX idx_active_subscriptions
ON subscriptions(user_id, renewed_at)
WHERE status = 'ACTIVE'; -- Only active records, much smaller index

-- When to index:
-- ✓ Frequently filtered columns (WHERE clause)
-- ✓ JOIN columns (foreign keys)
-- ✓ ORDER BY columns (eliminates sort)
-- ✗ Low-cardinality columns (boolean, status with 2-3 values)
-- ✗ Columns rarely queried
-- ✗ Very small tables (full scan is faster)
-- ✗ Columns updated very frequently (index maintenance cost)
```

EXPLAIN ANALYZE — Reading Query Plans

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT p.id, p.amount, a.name
FROM payments p
JOIN accounts a ON p.account_id = a.id
WHERE p.status = 'PENDING'
AND p.created_at > NOW() - INTERVAL '7 days'
ORDER BY p.created_at DESC
LIMIT 100;
```



```
-- Key things to look for:
-- Seq Scan → Full table scan (add index?)
-- Index Scan → Using index (good)
-- Index Only Scan → Covering index hit (best)
-- Hash Join → For large result sets (normal)
-- Nested Loop → For small result sets (normal)
-- Rows: actual vs estimated → large difference = stale statistics (ANALYZE table)
-- Buffers hit/read → cache hits vs disk reads
```

Common Query Anti-Patterns

```
-- 1. Function on indexed column (prevents index use)
-- BAD:
WHERE YEAR(created_at) = 2024
-- GOOD:
WHERE created_at >= '2024-01-01' AND created_at < '2025-01-01'

-- 2. Leading wildcard (prevents B-tree use)
-- BAD:
WHERE name LIKE '%Smith%'
-- GOOD: Use full-text search (PostgreSQL tsvector, Elasticsearch)

-- 3. OR conditions on different columns (can't use composite index)
-- BAD:
WHERE account_id = '123' OR transaction_id = '456'
-- GOOD: UNION (can use separate indexes)
SELECT * FROM payments WHERE account_id = '123'
UNION
SELECT * FROM payments WHERE transaction_id = '456'

-- 4. SELECT * (over-fetches data, prevents covering index)
-- BAD:
SELECT * FROM payments WHERE account_id = ?
-- GOOD:
SELECT id, amount, status, created_at FROM payments WHERE account_id = ?

-- 5. COUNT(*) on large tables without index
-- Use approximate counts: pg_stat_user_tables.n_live_tup
```

5.6 Database Migrations — Flyway & Liquibase

Flyway — Convention-Based Migration

```
db/migration/
├─ V1__create_payments_table.sql
├─ V2__add_idempotency_key.sql
├─ V3__create_accounts_table.sql
└─ V4__add_transfer_table.sql

Naming: V{version}__{description}.sql
V = versioned, R = repeatable, U = undo
```

```
-- V1__create_payments_table.sql
CREATE TABLE payments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  account_id VARCHAR(50) NOT NULL,
  amount NUMERIC(19, 4) NOT NULL,
  currency CHAR(3) NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
  idempotency_key VARCHAR(100) UNIQUE,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
  version BIGINT NOT NULL DEFAULT 0
);

CREATE INDEX idx_payments_account_id ON payments(account_id);
CREATE INDEX idx_payments_status_created ON payments(status, created_at DESC);
```

```
-- V2__add_payment_metadata.sql
ALTER TABLE payments
  ADD COLUMN description VARCHAR(500),
  ADD COLUMN metadata JSONB;

CREATE INDEX idx_payments_metadata ON payments USING gin(metadata);
```

```
// Spring Boot auto-applies migrations on startup
spring:
  flyway:
    enabled: true
    locations: classpath:db/migration
    baseline-on-migrate: false    # True for existing DBs
    validate-on-migrate: true    # Fail if migration checksums change
    out-of-order: false         # Fail if migration applied out of order
```

Liquibase — XML/YAML-Based (More Enterprise Features)

```
# db/changelog/db.changelog-master.yaml
databaseChangeLog:
  - include:
      file: db/changelog/changes/001-create-payments.yaml
  - include:
      file: db/changelog/changes/002-add-indexes.yaml

# db/changelog/changes/001-create-payments.yaml
databaseChangeLog:
  - changeSet:
      id: 001
      author: engineering-team
      changes:
        - createTable:
            tableName: payments
            columns:
              - column:
                  name: id
                  type: UUID
                  constraints:
                    primaryKey: true
              - column:
                  name: amount
                  type: DECIMAL(19,4)
                  constraints:
                    nullable: false
      rollback:
        - dropTable:
            tableName: payments
```

Migration Strategy in Production

```
NEVER:
- hibernate.ddl-auto = create/update (wipes/corrupts production data)
- Apply migrations directly on production DB without testing on staging
- Modify existing migrations (Flyway validates checksums)
- Deploy code and migration simultaneously (downtime risk)

ALWAYS:
- Test migrations on staging with production data copy
- Use backwards-compatible migrations
- Add-only approach: add column with default, populate, then add NOT NULL constraint
- Feature flags for new columns while migrating
- Blue/green deployment: migrate DB first, then deploy new code

Safe migration sequence for adding NOT NULL column:
Step 1: Add column as nullable (deploy, old code ignores it)
Step 2: Backfill existing rows (batch update)
Step 3: Add NOT NULL constraint (deploy new code using it)
```

5.7 Read Replicas & CQRS at DB Level

```
// Routing reads to replica, writes to primary
@Configuration
public class DataSourceRoutingConfig {

    @Bean
    public DataSource routingDataSource(
        @Qualifier("primaryDataSource") DataSource primary,
        @Qualifier("replicaDataSource") DataSource replica) {

        AbstractRoutingDataSource routing = new AbstractRoutingDataSource() {
            @Override
            protected Object determineCurrentLookupKey() {
                // Use replica for read-only transactions
                return TransactionSynchronizationManager.isCurrentTransactionReadOnly()
                    ? "REPLICA" : "PRIMARY";
            }
        };

        routing.setTargetDataSources(Map.of("PRIMARY", primary, "REPLICA", replica));
        routing.setDefaultTargetDataSource(primary);
        return routing;
    }
}

// Mark read-only services
@Service
public class ReportingService {

    @Transactional(readOnly = true) // → routes to replica
    public List<PaymentSummary> getMonthlyReport(YearMonth month) { ... }
}
```

Section Summary: Database Interview Must-Know

Questions asked at every bank interview:

1. "Explain transaction isolation levels. When would you use SERIALIZABLE?"
2. "What is the N+1 problem and how do you solve it?"
3. "How would you handle concurrent balance updates in a banking system?"
4. "Explain optimistic vs pessimistic locking and when to use each"
5. "How does HikariCP work? How do you size the pool?"
6. "How do you run database migrations safely in production?"
7. "How would you optimize a query returning results too slowly?"
8. "What is dirty checking in Hibernate?"
9. "How do you prevent deadlocks in database transactions?"
10. "Explain the difference between first-level and second-level cache in Hibernate"

Section 6: Distributed Systems

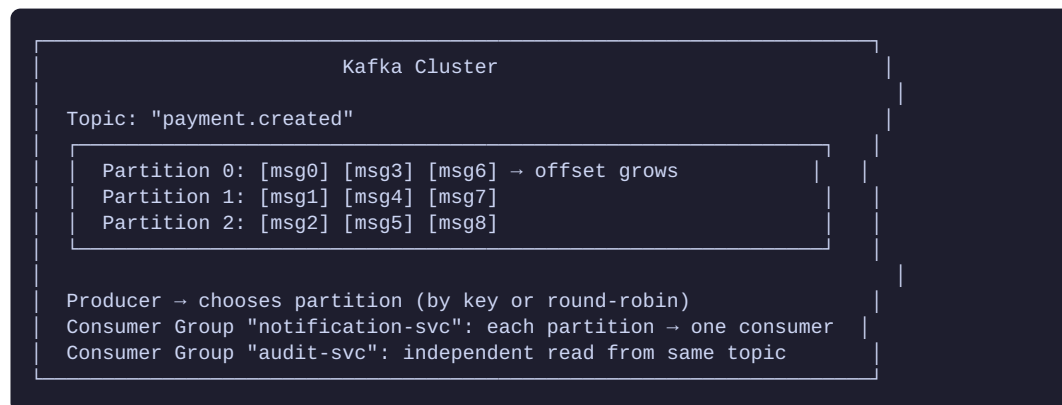
The Differentiator: Distributed systems knowledge separates engineers who can "code features" from engineers who design systems that "work at scale." At Goldman Sachs and Stripe, you're expected to discuss CAP theorem, Kafka ordering guarantees, and saga patterns in a 45-minute conversation.

6.1 Kafka Deep Dive

Node.js → Kafka Mental Model

Node.js EventEmitter:	Kafka:
In-process pub/sub	Distributed pub/sub
Lost if process dies	Durable (configurable retention)
No replay	Fully replayable
No ordering guarantee	Ordered within partition
No backpressure	Configurable consumer lag

Kafka Architecture



Key Kafka Guarantees

```
At-most-once: Message may be lost, never duplicated
               → Producer: acks=0, Consumer: commit before processing

At-least-once: Message never lost, may be duplicated
               → Producer: acks=1 or all, Consumer: process then commit
               → DEFAULT in most enterprise setups (requires idempotent consumer)

Exactly-once:  Never lost, never duplicated
               → Kafka Transactions + Idempotent Producer
               → Most complex, significant performance cost
               → Use for: financial ledger, inventory deduction

Ordering:
- Within a partition: STRICTLY ORDERED
- Across partitions: NO ORDER GUARANTEE
  → All events for the same entity (same accountId) must go to same partition
  → Use entity ID as partition key
```

Spring Kafka — Production Configuration

```
// Producer Configuration
@Configuration
public class KafkaProducerConfig {

    @Bean
    public ProducerFactory<String, PaymentEvent> producerFactory() {
        Map<String, Object> config = new HashMap<>();
        config.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapServers);
        config.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
        config.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, JsonSerializer.class);

        // Reliability settings
        config.put(ProducerConfig.ACKS_CONFIG, "all"); // Leader + all replicas ack
        config.put(ProducerConfig.RETRIES_CONFIG, 10);
        config.put(ProducerConfig.RETRY_BACKOFF_MS_CONFIG, 1000);
        config.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true); // Exactly-once producer

        // Performance settings
        config.put(ProducerConfig.COMPRESSION_TYPE_CONFIG, "lz4");
        config.put(ProducerConfig.LINGER_MS_CONFIG, 5); // Batch for 5ms
        config.put(ProducerConfig.BATCH_SIZE_CONFIG, 32 * 1024); // 32KB batch

        return new DefaultKafkaProducerFactory<>(config);
    }
}

// Consumer Configuration
@Configuration
public class KafkaConsumerConfig {

    @Bean
    public ConsumerFactory<String, PaymentEvent> consumerFactory() {
        Map<String, Object> config = new HashMap<>();
        config.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapServers);
        config.put(ConsumerConfig.GROUP_ID_CONFIG, "notification-service");
        config.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class);
        config.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, JsonDeserializer.class);

        // At-least-once: manual commit after processing
        config.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, false);
        config.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");

        // Performance
        config.put(ConsumerConfig.MAX_POLL_RECORDS_CONFIG, 500); // Process 500 at once
        config.put(ConsumerConfig.MAX_POLL_INTERVAL_MS_CONFIG, 300000); // 5min to process batch

        return new DefaultKafkaConsumerFactory<>(config);
    }

    @Bean
    public ConcurrentKafkaListenerContainerFactory<String, PaymentEvent>
        containerFactory(ConsumerFactory<String, PaymentEvent> factory) {

        ConcurrentKafkaListenerContainerFactory<String, PaymentEvent> container =
            new ConcurrentKafkaListenerContainerFactory<>();
        container.setConsumerFactory(factory);
        container.setConcurrency(3); // 3 threads = 3 partitions consumed in parallel

        // Manual offset management
        container.getContainerProperties().setAckMode(ContainerProperties.AckMode.MANUAL);

        // Error handling with DLQ
        container.setCommonErrorHandler(new DefaultErrorHandler(
            new DeadLetterPublishingRecoverer(kafkaTemplate),
            new FixedBackOff(1000L, 3) // 3 retries, 1s apart
        ));

        return container;
    }
}
```

```

// Consumer Implementation
@Component
@Slf4j
public class PaymentEventConsumer {

    @KafkaListener(
        topics = "payment.created",
        groupId = "notification-service",
        containerFactory = "containerFactory"
    )
    public void handlePaymentCreated(
        @Payload PaymentEvent event,
        @Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
        @Header(KafkaHeaders.OFFSET) long offset,
        Acknowledgment acknowledgment) {

        log.info("Processing payment event: id={} partition={} offset={}",
            event.getPaymentId(), partition, offset);

        try {
            // IDEMPOTENT PROCESSING – check if already processed
            if (processedEventRepo.existsById(event.getEventId())) {
                log.info("Duplicate event, skipping: {}", event.getEventId());
                acknowledgment.acknowledge(); // Still ack – already processed
                return;
            }

            notificationService.sendPaymentNotification(event);
            processedEventRepo.save(event.getEventId());
            acknowledgment.acknowledge(); // Commit offset after successful processing

        } catch (RetryableException e) {
            log.warn("Retryable error for event {}: {}", event.getEventId(), e.getMessage());
            throw e; // DLQ handler will retry 3 times then send to DLQ
        } catch (NonRetryableException e) {
            log.error("Non-retryable error for event {}, sending to DLQ", event.getEventId());
            acknowledgment.acknowledge(); // Don't block queue – handle separately
            deadLetterService.record(event, e);
        }
    }
}

```

Dead Letter Queue (DLQ) Pattern

```

// DLQ processor – handles failed messages
@Component
public class DlpProcessor {

    @KafkaListener(topics = "payment.created.DLT") // DLT = Dead Letter Topic
    public void processDlp(PaymentEvent event,
        @Header("kafka_dlt-original-topic") String originalTopic,
        @Header("kafka_dlt-exception-message") String error) {
        log.error("DLQ message from topic={} error={} eventId={}",
            originalTopic, error, event.getEventId());

        // Options:
        // 1. Alert on-call engineer
        alerting.critical("Kafka DLQ message requires attention: " + event.getEventId());

        // 2. Save for manual review
        dlqRepository.save(new DlpRecord(event, error));

        // 3. Attempt compensating action
        compensationService.handleFailedPaymentEvent(event);
    }
}

```

6.2 Event Sourcing

What It Is

Traditional: Store current state only
accounts table: { id, balance=900, status=ACTIVE, updated_at }

Event Sourcing: Store all events, derive state
events table: { id=1, type=ACCOUNT_OPENED, amount=1000, ts }
 { id=2, type=CREDIT, amount=500, ts }
 { id=3, type=DEBIT, amount=600, ts }
Current state = replay all events $\rightarrow$ balance = $1000 + 500 - 600 = 900$

Benefits:

- Complete audit trail (required by banking regulators)
- Time travel (reconstruct state at any point)
- Event replay for new features
- Natural Kafka integration

Challenges:

- Query complexity (need projections/snapshots)
- Storage growth
- Schema evolution of events
- Eventual consistency in read models

```
// Event store
public interface EventStore {
    void append(String aggregateId, List<DomainEvent> events, long expectedVersion);
    List<DomainEvent> loadEvents(String aggregateId);
    List<DomainEvent> loadEvents(String aggregateId, long fromVersion);
}

// Account aggregate – reconstituted from events
public class AccountAggregate {
    private UUID id;
    private BigDecimal balance;
    private AccountStatus status;
    private long version = 0;

    private final List<DomainEvent> uncommittedEvents = new ArrayList<>();

    // Reconstitute from event history
    public static AccountAggregate from(List<DomainEvent> events) {
        AccountAggregate account = new AccountAggregate();
        events.forEach(account::apply);
        return account;
    }

    // Business method – validates and creates event
    public void debit(BigDecimal amount) {
        if (status != AccountStatus.ACTIVE) {
            throw new IllegalStateException("Account is not active");
        }
        if (balance.compareTo(amount) < 0) {
            throw new InsufficientFundsException(amount, balance);
        }
        apply(new MoneyDebited(id, amount, Instant.now())); // Apply + record
    }

    // Event application – pure state mutation, no side effects
    private void apply(DomainEvent event) {
        if (event instanceof AccountOpened e) {
            this.id = e.accountId();
            this.balance = e.initialBalance();
            this.status = AccountStatus.ACTIVE;
        } else if (event instanceof MoneyDebited e) {
            this.balance = this.balance.subtract(e.amount());
        } else if (event instanceof MoneyCredited e) {
            this.balance = this.balance.add(e.amount());
        }
        this.version++;
    }
}
```

```

        uncommittedEvents.add(event);
    }
}

```

6.3 CQRS — Command Query Responsibility Segregation

Traditional MVC: Same model for reads and writes
 Service → reads from DB → writes to same DB

CQRS:

Commands (writes) → Command Model → Write DB (normalized, consistent)
 Queries (reads) → Query Model → Read DB (denormalized, fast, eventual)

Event flow:

POST /payments → PaymentService → PaymentCreated event → Kafka

GET /payments → ReadProjection ← updates from event consumer ↓

```

// Command side
@Service
@Transactional
public class PaymentCommandService {

    public PaymentId createPayment(CreatePaymentCommand command) {
        Payment payment = new Payment(command);
        paymentWriteRepo.save(payment);
        eventBus.publish(new PaymentCreatedEvent(payment));
        return payment.getId();
    }
}

// Query side – separate optimized read model
@Service
@Transactional(readOnly = true)
public class PaymentQueryService {

    public PaymentDetailsView getPaymentDetails(UUID paymentId) {
        // Denormalized view – already joined with account, customer info
        return paymentReadRepo.findDetailView(paymentId)
            .orElseThrow(() -> new PaymentNotFoundException(paymentId));
    }

    public Page<PaymentListItem> listPayments(PaymentFilter filter, Pageable pageable) {
        // Optimized for listing – no joins needed
        return paymentReadRepo.findByFilter(filter, pageable);
    }
}

// Event handler that updates read model
@Component
public class PaymentReadModelUpdater {

    @EventListener // Or @KafkaListener for cross-service
    public void on(PaymentCreatedEvent event) {
        // Build denormalized read model
        PaymentReadModel readModel = buildReadModel(event);
        paymentReadRepo.save(readModel);
    }
}

```


6.4 Distributed Transactions — Saga Pattern

The Problem with Distributed Transactions

```
Multi-service operation:
1. Debit account (AccountService DB)
2. Record payment (PaymentService DB)
3. Send notification (NotificationService)

Traditional 2PC (Two-Phase Commit):
- Coordinator locks all resources, then commits
- Blocking, slow, doesn't work across microservices
- Not suitable for modern distributed systems

Solution: Saga Pattern
- Sequence of local transactions
- Each step publishes event triggering next step
- Compensating transactions for rollback
```

Choreography-Based Saga

```
AccountService          PaymentService          NotificationService
|                        |                        |
| Debit Account          |                        |
| → Publish: AccountDebited |                        |
|                        |                        |
|                        | Listens to AccountDebited |
|                        | → Create Payment          |
|                        | → Publish: PaymentCreated |
|                        |                        |
|                        | Listens to PaymentCreated |
|                        | → Send Notification        |
|                        | → Publish: NotificationSent |

Rollback (if PaymentService fails):
→ Publish: PaymentFailed
↓
AccountService listens: PaymentFailed
→ Credit Account back (compensating transaction)
```

Orchestration-Based Saga

```
@Component
public class PaymentSagaOrchestrator {

    public void executePaymentSaga(PaymentSagaContext context) {
        SagaExecutionCoordinator
            .step("debit-account")
                .execute(() -> accountService.debit(context.getFromAccount(), context.getAmount()))
                .compensate(() -> accountService.credit(context.getFromAccount(), context.getAmount()))
            .step("create-payment-record")
                .execute(() -> paymentService.create(context))
                .compensate(() -> paymentService.cancel(context.getPaymentId()))
            .step("notify-customer")
                .execute(() -> notificationService.notifyPayment(context))
                .compensate(() -> {}) // Notifications: idempotent, ignore failures
        .run()
        .onSuccess(result -> log.info("Payment saga completed: {}", context.getPaymentId()))
        .onFailure((step, error) -> {
            log.error("Payment saga failed at step: {}", step);
            // Compensation is automatic
        });
    }
}
```

6.5 CAP Theorem — Practical Application

CAP Theorem: A distributed system can guarantee at most 2 of 3:
C = Consistency (all nodes see same data at same time)
A = Availability (every request gets a response)
P = Partition tolerance (system works despite network splits)

Since P (network partitions) always happen in real distributed systems, the real choice is: C vs A during a network partition.

CP Systems (consistency over availability):
→ HBase, ZooKeeper, MongoDB (in certain modes)
→ Use: Banking ledgers, inventory counts, leader election
→ During partition: refuse requests rather than risk inconsistency

AP Systems (availability over consistency):
→ Cassandra, DynamoDB, CouchDB
→ Use: Shopping carts, social feeds, cache
→ During partition: serve possibly stale data, reconcile later

Kafka sits in between:
→ Leader election (CP-like) for partition leadership
→ Producer acks control the C/A tradeoff

PACELC — More Practical Model

When no partition (E – Else):
Choose between Latency (L) vs Consistency (C)

PA/EL: DynamoDB, Cassandra – Available during partition, Low latency normally
PC/EC: HBase, ZooKeeper – Consistent during partition, Consistent normally
PC/EL: MongoDB – Consistent during partition, Low latency normally (dirty reads)

6.6 Backpressure

Problem: Producer sends events faster than consumer can process
Kafka producer → 10,000 msg/sec
DB consumer → 1,000 msg/sec
→ Queue fills up → OOM → crash

Solution strategies:

1. Buffer with bounded queue

```
BlockingQueue<Task> queue = new ArrayBlockingQueue<>(1000);  
// Producer blocks when queue full → natural backpressure
```
2. Rate limiting consumer

```
Semaphore semaphore = new Semaphore(maxConcurrent);  
semaphore.acquire();  
processAsync(event).whenComplete((r, e) -> semaphore.release());
```
3. Reactive backpressure (Project Reactor)

```
Flux.fromIterable(items)  
    .onBackpressureBuffer(1000)           // Buffer up to 1000  
    .onBackpressureDrop(dropped ->      // Drop oldest when full  
        log.warn("Dropping event: {}", dropped.getId()))  
    .flatMap(item -> process(item), 50) // 50 concurrent max  
    .subscribe();
```
4. Kafka: increase partitions, scale consumers

```
KafkaConsumer: max.poll.records = 100 // Process in batches
```

6.7 Message Ordering Guarantees

Challenge: Events for Account A must be processed in order
(open → deposit → withdrawal must not be reordered)

Kafka Ordering Rules:

- Within partition: Strict ordering guaranteed
- Across partitions: No ordering

Strategy: Use entity ID as Kafka partition key

```
kafkaTemplate.send("account.events", account.getId(), event);  
// All events for account "ACC-123" → same partition → ordered
```

Multi-partition concerns:

If a consumer group has 3 instances and Account A's events land on partition 2, only Consumer Instance 2 processes them. No reordering possible.

Ordering violation trap:

Don't use `UUID.randomUUID()` as partition key – random distribution means related events land on different partitions!

Section Summary: Distributed Systems Interview Topics

Must-master topics for FAANG and banks:

1. **Kafka architecture** — producers, consumers, partitions, offsets, consumer groups
2. **Delivery guarantees** — at-most-once, at-least-once, exactly-once
3. **DLQ pattern** — what it is, when to use, how to process
4. **Saga pattern** — choreography vs orchestration, compensation transactions
5. **CAP theorem** — with practical database examples
6. **Event sourcing** — pros, cons, when appropriate
7. **CQRS** — command vs query separation, eventual consistency
8. **Partition key strategy** — ordering guarantees, hot partition problems
9. **Consumer lag** — what it means, how to monitor, how to recover
10. **Idempotent consumers** — deduplication strategies

Commonly asked design question:

"Design a payment system that guarantees exactly-once payment processing across microservices"

Expected answer: Kafka transactions + idempotent consumers + saga pattern + outbox pattern

Section 7: Concurrency & Multithreading

This Is Where Java Shines and Burns: Concurrency is Java's most powerful feature and most dangerous pitfall. At every senior Java interview at FAANG and banks, you WILL get concurrency questions. This is the section that separates Java engineers from Node.js engineers who "just switched."

7.1 The Fundamental Shift from Node.js

Node.js Concurrency Model:

Single thread → event loop → async I/O
No shared mutable state (same thread sees everything)
No race conditions on simple data reads/writes
Concurrency via: callbacks, promises, async/await

Java Concurrency Model:

Multiple threads → true parallelism → shared heap
Same object can be modified by multiple threads simultaneously
Race conditions, deadlocks, memory visibility issues are REAL
Concurrency via: synchronized, locks, concurrent collections, thread pools

The key mindset shift: **in Java, any shared mutable state is a potential data race unless explicitly synchronized.**

7.2 Thread Basics

Creating and Managing Threads

```
// Don't create raw threads in production – use thread pools
// But understand this for interviews:

// Method 1: Extend Thread
Thread t = new Thread() {
    @Override
    public void run() {
        System.out.println("Running in: " + Thread.currentThread().getName());
    }
};
t.start();

// Method 2: Runnable (preferred – separates task from execution)
Runnable task = () -> processPayment(paymentId);
new Thread(task).start();

// Method 3: Virtual Thread (Java 21+) – million lightweight threads
Thread.ofVirtual().name("payment-handler").start(() -> {
    // Can block freely – JVM handles scheduling
    processWithBlockingDB(payment);
});
```

Thread States Revisited

```
Thread t = new Thread(task);
// State: NEW

t.start();
```

```
// State: RUNNABLE (executing or ready to execute)

synchronized (lock) {
    // Another thread holds lock → State: BLOCKED (waiting for monitor)
}

Object.wait();           // State: WAITING (indefinite)
Thread.sleep(1000);      // State: TIMED_WAITING
join();                  // Wait for another thread to finish

// After run() completes → State: TERMINATED
```

7.3 Synchronization — The Foundation

The `synchronized` Keyword

```
public class BankAccount {
    private BigDecimal balance;

    // Method-level: locks on 'this' instance
    public synchronized void deposit(BigDecimal amount) {
        balance = balance.add(amount);
        // Only one thread can execute this at a time per account instance
    }

    // Block-level: more granular
    public void transfer(BankAccount target, BigDecimal amount) {
        // Lock ordering to prevent deadlock
        BankAccount first = this.id < target.id ? this : target;
        BankAccount second = first == this ? target : this;

        synchronized (first) {
            synchronized (second) {
                this.balance = this.balance.subtract(amount);
                target.balance = target.balance.add(amount);
            }
        }
    }

    // Static synchronized: locks on CLASS object, not instance
    public static synchronized void globalOperation() {
        // Only one thread across ALL instances
    }
}
```

The `volatile` Keyword

```
// Problem: CPU caches – each core may cache variables locally
// Thread 1 updates running = false but Thread 2's cache still shows true

// Solution: volatile ensures visibility across threads
public class ServiceShutdownCoordinator {
    private volatile boolean running = true; // Visible to all threads immediately

    public void shutdown() {
        this.running = false; // Immediately visible to all threads
    }

    public void processLoop() {
        while (running) { // Reads fresh value, not cached
            processNextBatch();
        }
    }
}

// volatile guarantees: visibility (read/write atomic for longs/doubles too)
// volatile does NOT guarantee: compound operations (check-then-act, read-modify-write)
```

```
// WRONG: volatile doesn't make this atomic
volatile int counter = 0;
counter++; // Read, increment, write – not atomic! Use AtomicInteger
```

7.4 Thread-Safe Data Structures

```
// WRONG in multi-threaded environment:
Map<String, Integer> cache = new HashMap<>(); // Not thread-safe!

// CORRECT:
Map<String, Integer> cache = new ConcurrentHashMap<>(); // Thread-safe, high performance

// ConcurrentHashMap internals:
// - Segment locking (bucket-level locks, not whole map)
// - Reads are lock-free in most cases
// - Writes lock only affected bucket

// Thread-safe collections:
CopyOnWriteArrayList<String> list = new CopyOnWriteArrayList<>(); // Reads fast, writes copy
CopyOnWriteArraySet<String> set = new CopyOnWriteArraySet<>();
ConcurrentLinkedQueue<Task> queue = new ConcurrentLinkedQueue<>();
ArrayBlockingQueue<Task> bounded = new ArrayBlockingQueue<>(1000); // Blocks on full/empty
LinkedBlockingQueue<Task> unbounded = new LinkedBlockingQueue<>();

// Atomic variables – lock-free operations using CAS (Compare-And-Swap)
AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet(); // Atomic read-modify-write
counter.compareAndSet(5, 10); // CAS: if value==5, set to 10

AtomicReference<Config> config = new AtomicReference<>(initialConfig);
config.updateAndGet(old -> new Config(old.withUpdatedSetting())); // Atomic update

// LongAdder – better than AtomicLong for high-contention counters
LongAdder hitCounter = new LongAdder();
hitCounter.increment(); // Low contention via striping
long total = hitCounter.sum(); // Merge all stripes
```

7.5 Lock API — Beyond synchronized

```
// java.util.concurrent.locks – more flexible than synchronized

// ReentrantLock: same as synchronized but with try-lock, timeout
ReentrantLock lock = new ReentrantLock(true); // fair = FIFO ordering

lock.lock();
try {
    // critical section
} finally {
    lock.unlock(); // ALWAYS in finally!
}

// Try-lock – avoid blocking indefinitely
if (lock.tryLock(100, TimeUnit.MILLISECONDS)) {
    try {
        // Got the lock
    } finally {
        lock.unlock();
    }
} else {
    // Could not acquire lock – fail fast or try later
}

// ReadWriteLock – multiple readers OR one writer
ReadWriteLock rwLock = new ReentrantReadWriteLock();
Lock readLock = rwLock.readLock();
Lock writeLock = rwLock.writeLock();
```

```

// Many threads can hold read lock simultaneously
readLock.lock();
try { return cache.get(key); } finally { readLock.unlock(); }

// Only one thread can hold write lock (excludes all readers)
writeLock.lock();
try { cache.put(key, value); } finally { writeLock.unlock(); }

// StampedLock – optimistic reads (Java 8+, better performance)
StampedLock stampedLock = new StampedLock();

// Optimistic read – no actual lock acquisition
long stamp = stampedLock.tryOptimisticRead();
double x = this.x, y = this.y;
if (!stampedLock.validate(stamp)) {
    // Data was modified – fall back to regular read lock
    stamp = stampedLock.readLock();
    try { x = this.x; y = this.y; }
    finally { stampedLock.unlockRead(stamp); }
}

```

7.6 ExecutorService — Thread Pool Management

```

// NEVER create raw Thread in production – use thread pools

// Fixed thread pool: N threads always alive
ExecutorService fixedPool = Executors.newFixedThreadPool(10);

// Cached thread pool: grows/shrinks – DANGER in production (unbounded!)
ExecutorService cached = Executors.newCachedThreadPool(); // Can create 1000s of threads!

// Scheduled pool: for periodic/delayed tasks
ScheduledExecutorService scheduled = Executors.newScheduledThreadPool(5);
scheduled.scheduleAtFixedRate(task, 0, 60, TimeUnit.SECONDS);

// Production-grade thread pool configuration:
ThreadPoolExecutor executor = new ThreadPoolExecutor(
    5, // corePoolSize: always-alive threads
    20, // maximumPoolSize: max threads under load
    60, TimeUnit.SECONDS, // keepAliveTime: idle thread lifetime
    new ArrayBlockingQueue<>(1000), // workQueue: bounded! rejects when full
    new ThreadFactoryBuilder() // named threads for better debugging
        .setNameFormat("payment-worker-%d")
        .setDaemon(false)
        .build(),
    new ThreadPoolExecutor.CallerRunsPolicy() // When queue full: caller executes task
);
// Alternatives:
// AbortPolicy: throw RejectedExecutionException (default)
// DiscardPolicy: silently drop
// DiscardOldestPolicy: drop oldest queued task

// In Spring – use @Async with configured executor
@Configuration
@EnableAsync
public class AsyncConfig implements AsyncConfigurer {

    @Override
    @Bean(name = "taskExecutor")
    public Executor getAsyncExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(5);
        executor.setMaxPoolSize(20);
        executor.setQueueCapacity(200);
        executor.setThreadNamePrefix("async-payment-");
        executor.setRejectedExecutionHandler(new ThreadPoolExecutor.CallerRunsPolicy());
        executor.initialize();
        return executor;
    }
}

```

```

@Service
public class ReportService {

    @Async("taskExecutor")
    public CompletableFuture<Report> generateReport(ReportRequest request) {
        // Runs in taskExecutor thread pool, not HTTP request thread
        Report report = heavyComputation(request);
        return CompletableFuture.completedFuture(report);
    }
}

```

7.7 CompletableFuture — Async Programming

```

// Node.js Promise → Java CompletableFuture

// Single async operation
CompletableFuture<User> userFuture = CompletableFuture.supplyAsync(
    () -> userService.findById(userId), // Runs in ForkJoinPool.commonPool()
    customExecutor                      // Or specific thread pool
);

// Chain operations (like .then())
CompletableFuture<UserProfile> profileFuture = userFuture
    .thenApply(user -> buildProfile(user)) // Synchronous transform
    .thenApplyAsync(profile -> enrichProfile(profile)) // Async transform
    .exceptionally(ex -> {
        log.error("Failed to fetch profile", ex);
        return UserProfile.defaultProfile(); // Fallback
    });

// Parallel execution (like Promise.all())
CompletableFuture<User> userFuture = fetchUserAsync(userId);
CompletableFuture<List<Account>> accountsFuture = fetchAccountsAsync(userId);
CompletableFuture<CreditScore> creditFuture = fetchCreditScoreAsync(userId);

CompletableFuture.allOf(userFuture, accountsFuture, creditFuture)
    .thenApply(v -> new Dashboard(
        userFuture.join(), // join() gets result (already complete)
        accountsFuture.join(),
        creditFuture.join()
    ))
    .get(5, TimeUnit.SECONDS); // Overall timeout

// First to complete (like Promise.race())
CompletableFuture.anyOf(primaryFuture, fallbackFuture)
    .thenAccept(result -> process(result));

// Exception handling
userFuture
    .thenApply(this::process)
    .handle((result, ex) -> { // Always called, regardless of success/failure
        if (ex != null) return handleError(ex);
        return result;
    });

// Banking example: parallel enrichment
public CompletableFuture<EnrichedPayment> enrichPayment(Payment payment) {
    CompletableFuture<Account> fromAccount =
        CompletableFuture.supplyAsync(() -> accountService.get(payment.getFromId()));
    CompletableFuture<Account> toAccount =
        CompletableFuture.supplyAsync(() -> accountService.get(payment.getToId()));
    CompletableFuture<FxRate> fxRate =
        CompletableFuture.supplyAsync(() -> fxService.getRate(payment.getCurrency()));

    return CompletableFuture.allOf(fromAccount, toAccount, fxRate)
        .thenApply(v -> new EnrichedPayment(
            payment,
            fromAccount.join(),
            toAccount.join(),
            fxRate.join()
        ));
}

```



```
    });  
}
```

7.8 Race Conditions and Deadlocks

Race Condition Example and Fix

```
// RACE CONDITION: check-then-act  
public class InventoryService {  
    private int stock = 10;  
  
    // WRONG: Thread 1 reads stock=1, Thread 2 reads stock=1, both proceed!  
    public boolean reserve(int quantity) {  
        if (stock >= quantity) {           // Thread 1 passes  
            // Context switch happens here!  
            stock -= quantity;             // Both threads subtract!  
            return true;                   // Both return true – oversold!  
        }  
        return false;  
    }  
}  
  
// CORRECT: atomic check-and-act  
public synchronized boolean reserve(int quantity) {  
    if (stock >= quantity) {  
        stock -= quantity;  
        return true;  
    }  
    return false;  
}  
  
// BETTER: AtomicInteger for simple increment/decrement  
private final AtomicInteger atomicStock = new AtomicInteger(10);  
  
public boolean reserveAtomic(int quantity) {  
    while (true) {  
        int current = atomicStock.get();  
        if (current < quantity) return false;  
        if (atomicStock.compareAndSet(current, current - quantity)) return true;  
        // CAS failed – another thread changed value – retry  
    }  
}
```

Deadlock — Detailed Example

```
// DEADLOCK: Account A → Account B and Account B → Account A  
Object lockA = new Object();  
Object lockB = new Object();  
  
Thread t1 = new Thread(() -> {  
    synchronized (lockA) {  
        Thread.sleep(100);  
        synchronized (lockB) { /* ... */ } // Waiting for lockB  
    }  
});  
  
Thread t2 = new Thread(() -> {  
    synchronized (lockB) {  
        Thread.sleep(100);  
        synchronized (lockA) { /* ... */ } // Waiting for lockA  
    }  
});  
// t1 holds A, wants B. t2 holds B, wants A → deadlock!  
  
// Prevention: consistent lock ordering  
private void transferSafe(Account from, Account to) {  
    Account first = from.getId().compareTo(to.getId()) < 0 ? from : to;  
    Account second = first == from ? to : from;  
    synchronized (first) {  
        synchronized (second) {  
            // Transfer logic  
        }  
    }  
}
```

```

    synchronized (first) {
        synchronized (second) {
            // Lock order is always smaller-ID first → no deadlock
        }
    }
}

// Detection: jstack <pid> will show:
// "Found one Java-level deadlock:"
// Thread X waiting for lock held by Thread Y
// Thread Y waiting for lock held by Thread X

```

7.9 Java 21 Virtual Threads — Project Loom

```

// Traditional: 1 OS thread per request → expensive, limited to ~10,000
// Virtual Thread: millions of lightweight JVM-managed threads

// Virtual threads with executor
try (ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor()) {
    IntStream.range(0, 100_000).forEach(i -> {
        executor.submit(() -> {
            // Each of 100,000 requests gets its own virtual thread
            // Blocking DB call is fine — JVM unmounts and remounts
            String result = database.query("SELECT * FROM accounts WHERE id = " + i);
            return result;
        });
    });
} // Awaits all tasks, then shuts down

// Spring Boot 3.2+: enable virtual threads globally
spring:
  threads:
    virtual:
      enabled: true # All Tomcat threads become virtual threads

// Impact: I/O-bound services can handle dramatically more concurrency
// without reactive programming complexity

// Limitations of virtual threads:
// - Pinning: synchronized blocks pin to OS thread (performance concern)
//   → Prefer ReentrantLock over synchronized in virtual thread code
// - CPU-bound work: no benefit (virtual threads don't parallelize CPU)
// - Thread-local variables work but may hold more data than expected

```

7.10 Concurrent Collections — Production Patterns

```

// ConcurrentHashMap advanced operations (all atomic)
ConcurrentHashMap<String, Integer> counts = new ConcurrentHashMap<>();

// Atomic computeIfAbsent — create entry only if missing
RateLimiter limiter = limiters.computeIfAbsent(clientId,
    id -> Bucket4j.builder()
        .addLimit(Bandwidth.classic(100, Refill.greedy(100, Duration.ofMinutes(1))))
        .build());

// Atomic merge — update or create
counts.merge(category, 1, Integer::sum); // Atomically increment counter

// Atomic compute — read-modify-write atomically
counts.compute(key, (k, v) -> v == null ? 1 : v + 1);

// forEach with concurrency level (parallelism threshold)
counts.forEach(4, (key, val) -> process(key, val)); // 4 parallel threads

// Search (returns first non-null result)
String hotKey = counts.search(4, (k, v) -> v > 1000 ? k : null);

```

```
// BlockingQueue – producer-consumer pattern
BlockingQueue<Task> taskQueue = new ArrayBlockingQueue<>(1000);

// Producer
executor.submit(() -> {
    while (running) {
        Task task = generateTask();
        taskQueue.put(task); // Blocks if queue full – backpressure!
    }
});

// Consumer
executor.submit(() -> {
    while (running) {
        Task task = taskQueue.poll(1, TimeUnit.SECONDS); // Wait 1s for task
        if (task != null) process(task);
    }
});
```

Section Summary: Concurrency Interview Must-Know

Java concurrency is different from Node.js — demonstrate you understand this:

1. **Explain Java memory model** — happens-before relationship, visibility
2. **synchronized vs volatile vs AtomicInteger** — when to use which
3. **Thread pool sizing** — core pool, max pool, queue capacity, rejection policy
4. **CompletableFuture chains** — map, flatMap, allOf, anyOf, exception handling
5. **Deadlock** — demonstrate prevention strategy with lock ordering
6. **ConcurrentHashMap vs Collections.synchronizedMap** — performance difference
7. **ReadWriteLock** — when multiple readers benefit
8. **Virtual threads** — what they are, benefits, limitations (pinning)
9. **ThreadLocal** — usage, and why you MUST clear it in thread pools
10. **Race condition** — identify and fix check-then-act patterns

Most common interview bug: Write a thread-safe counter without using synchronized on the method.
Expected: AtomicInteger or CAS loop.

Section 8: Cloud & DevOps Integration

You Already Know This: Your AWS, Docker, Kubernetes, Terraform, and Kafka background is a serious advantage. This section focuses specifically on how Java/Spring services integrate with these tools — the patterns and idioms that differ from Node.js deployments.

8.1 Spring Boot on AWS

Spring Cloud AWS

```
// Spring Cloud AWS – native AWS service integration
// Replaces manual SDK usage with Spring-idiomatic code

// application.yml
spring:
  cloud:
    aws:
      region:
        static: eu-west-1
      credentials:
        instance-profile: true # Use EC2/ECS instance role (IAM role)

// AWS Secrets Manager – replace hardcoded secrets
spring:
  config:
    import: aws-secretsmanager:/myapp/prod/secrets
# Secrets from AWS Secrets Manager available as Spring properties
# database.password=${DB_PASSWORD} → fetched from secrets manager
```

SQS Consumer (AWS Queue)

```
@Component
public class SqsPaymentConsumer {

    @SqsListener("payment-processing-queue")
    public void receivePayment(@Payload PaymentMessage message,
                              @Header("ApproximateReceiveCount") int receiveCount) {
        if (receiveCount > 3) {
            log.error("Message exceeded retry limit, sending to DLQ: {}", message.getId());
            return; // SQS will send to DLQ after maxReceiveCount
        }

        try {
            paymentService.process(message);
        } catch (RetryableException e) {
            throw e; // Trigger SQS retry
        }
    }
}

// SQS Producer
@Service
public class SqsProducer {

    private final SqsTemplate sqsTemplate;

    public void sendPayment(PaymentMessage message) {
        sqsTemplate.send(to -> to
            .queue("payment-processing-queue")
            .payload(message)
            .header("paymentType", message.getType())
            .delaySeconds(5));
    }
}
```

```
}  
}
```

S3 Integration

```
@Service  
public class DocumentStorageService {  
  
    private final S3Client s3Client;  
  
    public String uploadDocument(String key, InputStream content, String contentType) {  
        PutObjectRequest request = PutObjectRequest.builder()  
            .bucket(bucketName)  
            .key(key)  
            .contentType(contentType)  
            .serverSideEncryption(ServerSideEncryption.AWS_KMS) // Encrypt at rest  
            .build();  
  
        s3Client.putObject(request, RequestBody.fromInputStream(content, contentLength));  
        return "s3://" + bucketName + "/" + key;  
    }  
  
    // Pre-signed URL – temporary access (no proxy required)  
    public String generateDownloadUrl(String key, Duration expiry) {  
        S3Presigner presigner = S3Presigner.create();  
        GetObjectPresignRequest presignRequest = GetObjectPresignRequest.builder()  
            .signatureDuration(expiry)  
            .getObjectRequest(r -> r.bucket(bucketName).key(key))  
            .build();  
        return presigner.presignGetObject(presignRequest).url().toString();  
    }  
}
```

8.2 Docker — Java-Specific Patterns

Production Dockerfile (Multi-stage)

```
# Stage 1: Build  
FROM eclipse-temurin:21-jdk-alpine AS builder  
  
WORKDIR /app  
COPY pom.xml .  
COPY .mvn/ .mvn/  
COPY mvnw .  
# Download deps separately – cache this layer  
RUN ./mvnw dependency:go-offline -B  
  
COPY src/ src/  
RUN ./mvnw package -DskipTests -B  
  
# Unpack JAR for better layer caching (Spring Boot layered JARs)  
RUN java -Djarmode=layertools -jar target/*.jar extract  
  
# Stage 2: Runtime – minimal image  
FROM eclipse-temurin:21-jre-alpine  
  
RUN addgroup -S appgroup && adduser -S appuser -G appgroup  
WORKDIR /app  
  
# Copy Spring Boot layers (changes least frequently → better caching)  
COPY --from=builder /app/dependencies/ ./  
COPY --from=builder /app/spring-boot-loader/ ./  
COPY --from=builder /app/snapshot-dependencies/ ./  
COPY --from=builder /app/application/ ./  
  
# Security: run as non-root  
USER appuser
```

```
EXPOSE 8080
```

```
# JVM memory configured via env – not hardcoded
ENV JAVA_OPTS="-XX:+UseContainerSupport -XX:MaxRAMPercentage=75.0 \
              -XX:+ExitOnOutOfMemoryError \
              -Djava.security.egd=file:/dev/./urandom"
```

```
ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS org.springframework.boot.loader.JarLauncher"]
```

Spring Boot Layered JARs

Traditional JAR: all-in-one, any code change = large Docker layer invalidated
Layered JAR: separated layers, incremental pushes

```
spring-boot:repackage with layers:
├─ dependencies          (rarely changes – library updates)
├─ spring-boot-loader    (almost never changes)
├─ snapshot-dependencies (dev dependencies)
└─ application           (your code – changes every commit)
```

Only the application layer (~100KB) needs to be pushed on most builds
vs full JAR (~50MB) without layers

8.3 Kubernetes — Java Service Deployment

Kubernetes Manifest for Spring Boot

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: payment-service
  labels:
    app: payment-service
spec:
  replicas: 3
  selector:
    matchLabels:
      app: payment-service
  template:
    metadata:
      labels:
        app: payment-service
    annotations:
      prometheus.io/scrape: "true"
      prometheus.io/path: "/actuator/prometheus"
      prometheus.io/port: "8080"
    spec:
      containers:
        - name: payment-service
          image: myregistry/payment-service:1.2.3
          ports:
            - containerPort: 8080
          env:
            - name: SPRING_PROFILES_ACTIVE
              value: "prod"
            - name: DB_URL
              valueFrom:
                secretKeyRef:
                  name: payment-secrets
                  key: database-url
            - name: JAVA_OPTS
              value: "-XX:+UseContainerSupport -XX:MaxRAMPercentage=75.0"
          resources:
            requests:
              memory: "512Mi"
              cpu: "250m"
            limits:
```

```

        memory: "1Gi"
        cpu: "1000m"
    # Liveness: restart pod if JVM is stuck
    livenessProbe:
        httpGet:
            path: /actuator/health/liveness
            port: 8080
        initialDelaySeconds: 60    # Account for JVM startup
        periodSeconds: 10
        failureThreshold: 3
    # Readiness: route traffic only when app is warm
    readinessProbe:
        httpGet:
            path: /actuator/health/readiness
            port: 8080
        initialDelaySeconds: 30
        periodSeconds: 5
        failureThreshold: 3
    # Graceful shutdown
    lifecycle:
        preStop:
            exec:
                command: ["sh", "-c", "sleep 10"] # Let load balancer deregister first
    terminationGracePeriodSeconds: 60 # Allow in-flight requests to complete

```

Graceful Shutdown in Spring Boot

```

// application.yml
server:
    shutdown: graceful # Wait for in-flight requests to complete

spring:
    lifecycle:
        timeout-per-shutdown-phase: 30s # Max wait time

// On SIGTERM:
// 1. Spring marks app as "not ready" (readiness probe fails → k8s stops routing)
// 2. Wait 10s (preStop sleep) for load balancer to deregister
// 3. Process in-flight requests (up to 30s)
// 4. Close DB connections, Kafka producers, etc.
// 5. JVM exits

```

Horizontal Pod Autoscaler

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
    name: payment-service-hpa
spec:
    scaleTargetRef:
        apiVersion: apps/v1
        kind: Deployment
        name: payment-service
    minReplicas: 3
    maxReplicas: 20
    metrics:
        - type: Resource
          resource:
            name: cpu
            target:
                type: Utilization
                averageUtilization: 70
        - type: External
          external:
            metric:
                name: kafka_consumer_lag # Scale on Kafka consumer lag
            target:
                type: AverageValue
                averageValue: "1000" # Scale up when lag > 1000 per pod

```

8.4 CI/CD Pipeline for Java Services

GitHub Actions — Java/Maven Pipeline

```
# .github/workflows/ci.yml
name: CI/CD Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

env:
  JAVA_VERSION: '21'
  REGISTRY: ghcr.io
  IMAGE_NAME: ${ github.repository }

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_PASSWORD: test
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

    steps:
      - uses: actions/checkout@v4

      - name: Set up JDK ${ env.JAVA_VERSION }
        uses: actions/setup-java@v4
        with:
          java-version: ${ env.JAVA_VERSION }
          distribution: 'temurin'
          cache: 'maven'

      - name: Run tests with coverage
        run: ./mvnw verify -B --no-transfer-progress
        env:
          SPRING_DATASOURCE_URL: jdbc:postgresql://localhost:5432/testdb

      - name: Upload coverage to Codecov
        uses: codecov/codecov-action@v4

      - name: Run OWASP dependency check
        run: ./mvnw org.owasp:dependency-check-maven:check -B

  build-and-push:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Set up JDK ${ env.JAVA_VERSION }
        uses: actions/setup-java@v4
        with:
          java-version: ${ env.JAVA_VERSION }
          distribution: 'temurin'
          cache: 'maven'

      - name: Build with Maven
        run: ./mvnw package -DskipTests -B
```



```

- name: Build and push Docker image
  uses: docker/build-push-action@v5
  with:
    context: .
    push: true
    tags: |
      ${ env.REGISTRY }}/${ env.IMAGE_NAME }}:latest
      ${ env.REGISTRY }}/${ env.IMAGE_NAME }}:${ env.github.sha }}
    cache-from: type=gha
    cache-to: type=gha,mode=max

deploy-staging:
  needs: build-and-push
  runs-on: ubuntu-latest
  environment: staging

  steps:
    - name: Deploy to Kubernetes
      run: |
        kubectl set image deployment/payment-service \
          payment-service=${ env.REGISTRY }}/${ env.IMAGE_NAME }}:${ env.github.sha }} \
          -n staging
        kubectl rollout status deployment/payment-service -n staging --timeout=5m

```

8.5 Terraform for Java Infrastructure

```

# terraform/modules/java-service/main.tf
# ECS Fargate task for Java microservice

resource "aws_ecs_task_definition" "payment_service" {
  family              = "payment-service"
  requires_compatibilities = ["FARGATE"]
  network_mode        = "awsvpc"
  cpu                  = 1024 # 1 vCPU
  memory               = 2048 # 2GB RAM

  container_definitions = jsonencode([
    {
      name      = "payment-service"
      image     = "${var.ecr_repo}:${var.image_tag}"

      environment = [
        { name = "SPRING_PROFILES_ACTIVE", value = var.environment },
        { name = "JAVA_OPTS", value = "-XX:+UseContainerSupport -XX:MaxRAMPercentage=75.0 -XX:+ExitOnOutOfMemoryError" }
      ]

      secrets = [
        { name = "DB_PASSWORD", valueFrom = aws_secretsmanager_secret.db_password.arn },
        { name = "JWT_SECRET", valueFrom = aws_secretsmanager_secret.jwt_secret.arn }
      ]

      portMappings = [{ containerPort = 8080 }]

      logConfiguration = {
        logDriver = "awslogs"
        options = {
          awslogs-group = "/ecs/payment-service"
          awslogs-region = var.aws_region
        }
      }

      healthCheck = {
        command = ["CMD-SHELL", "curl -f http://localhost:8080/actuator/health || exit 1"]
        interval = 30
        timeout = 5
        retries = 3
        startPeriod = 60 # Account for JVM startup
      }
    }
  ])
}

```

8.6 Helm Chart for Java Microservice

```
# helm/payment-service/values.yaml
replicaCount: 3

image:
  repository: myregistry/payment-service
  tag: "1.2.3"
  pullPolicy: IfNotPresent

service:
  type: ClusterIP
  port: 8080

resources:
  limits:
    cpu: 1000m
    memory: 1Gi
  requests:
    cpu: 250m
    memory: 512Mi

jvmOptions: "-XX:+UseContainerSupport -XX:MaxRAMPercentage=75.0 -XX:+ExitOnOutOfMemoryError"

springProfile: prod

autoscaling:
  enabled: true
  minReplicas: 3
  maxReplicas: 20
  targetCPUUtilizationPercentage: 70

livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
  initialDelaySeconds: 60
  periodSeconds: 10

readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 5

# Production: use AWS Secrets Manager / Vault
secrets:
  dbUrl: "" # Override in values-prod.yaml
  dbPassword: ""
```

8.7 GraalVM Native Image

```
<!-- pom.xml - Spring Boot Native -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aot</artifactId>
</dependency>

<plugin>
  <groupId>org.graalvm.buildtools</groupId>
  <artifactId>native-maven-plugin</artifactId>
  <version>0.10.3</version>
</plugin>
```

```
# Build native image (takes 5-15 minutes)
./mvnw native:compile -Pnative
```

```
# Result: native executable, ~50MB, no JVM needed
./target/payment-service

# Startup time: 50-200ms vs 3-8s for JVM
# Memory: 50-80MB vs 200-500MB for JVM

# Tradeoffs:
# + Instant startup (perfect for Lambda/serverless)
# + Much lower memory
# - No JIT optimization (peak throughput slightly lower)
# - Reflection requires explicit configuration (hint files)
# - Longer build time
# - Some libraries not yet native-compatible
```

Section Summary: Cloud/DevOps Key Points

What companies actually care about:

1. **Container best practices** — multi-stage builds, non-root user, proper JVM flags for containers
2. **k8s probes** — liveness vs readiness, correct `initialDelaySeconds` for JVM
3. **Graceful shutdown** — why it matters, how to configure
4. **Resource limits** — JVM heap vs container memory, why `MaxRAMPercentage` > hardcoded `Xmx`
5. **Secret management** — never hardcode, use AWS Secrets Manager, Vault, k8s secrets
6. **HPA scaling** — CPU-based and custom metrics (Kafka lag)
7. **Layered JARs** — why and how for better Docker layer caching

Your Node.js advantage: You already understand containers, k8s, and Terraform deeply. The Java-specific knowledge to add: JVM flag tuning for containers, Spring Boot actuator endpoints, and layered JAR builds.

Section 9: Security

Banking-Grade Security: At HSBC, Goldman Sachs, and JP Morgan, security is not a checkbox — it's an engineering discipline. You're expected to know OAuth2 flows, JWT vulnerabilities, OWASP Top 10 mitigations, and how to implement them correctly in Spring Security.

9.1 OAuth2 & OpenID Connect — Deep Dive

OAuth2 Flows — When to Use Each

```
Authorization Code Flow (+ PKCE):
  → Use for: Web apps, mobile apps (user-facing)
  → Browser redirects to auth server → code → exchange for token
  → Never exposes tokens in URL

Client Credentials Flow:
  → Use for: Service-to-service (M2M) communication
  → No user involved → service gets token directly
  → How payment-service calls account-service in enterprise

Implicit Flow:
  → DEPRECATED — never use (tokens in URL, no PKCS)

Resource Owner Password:
  → DEPRECATED — never use (service gets user credentials directly)

Device Code Flow:
  → Use for: CLI tools, IoT devices
```

Spring Security OAuth2 Resource Server

```
// Resource Server — validates JWT tokens from auth server
@Configuration
@EnableWebSecurity
public class ResourceServerConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .oauth2ResourceServer(oauth2 -> oauth2
                .jwt(jwt -> jwt
                    .decoder(jwtDecoder())
                    .jwtAuthenticationConverter(jwtAuthConverter()))
                .authenticationEntryPoint(customEntryPoint()))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/actuator/health/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/v1/payments/**")
                    .hasAuthority("SCOPE_payments:read")
                .requestMatchers(HttpMethod.POST, "/api/v1/payments")
                    .hasAuthority("SCOPE_payments:write")
                .requestMatchers("/api/v1/admin/**")
                    .hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .build();
    }

    @Bean
    public JwtDecoder jwtDecoder() {
        // Verify JWT signature using auth server's public key (JWKS endpoint)
        return NimbusJwtDecoder.withJwkSetUri("https://auth.bank.com/.well-known/jwks.json")
            .jwsAlgorithm(SignatureAlgorithm.RS256)
            .build();
    }
}
```

```

    }

    @Bean
    public JwtAuthenticationConverter jwtAuthConverter() {
        JwtGrantedAuthoritiesConverter scopeConverter = new JwtGrantedAuthoritiesConverter();
        scopeConverter.setAuthorityPrefix("SCOPE_");
        scopeConverter.setAuthoritiesClaimName("scp"); // Custom claim name

        JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
        converter.setJwtGrantedAuthoritiesConverter(jwt -> {
            Collection<GrantedAuthority> scopes = scopeConverter.convert(jwt);
            // Add roles from custom claim
            List<GrantedAuthority> roles = jwt.getClaimAsStringList("roles").stream()
                .map(role -> new SimpleGrantedAuthority("ROLE_" + role))
                .collect(Collectors.toList());
            return Stream.concat(scopes.stream(), roles.stream()).toList();
        });
        converter.setPrincipalClaimName("sub");
        return converter;
    }
}

```

Client Credentials (Service-to-Service)

```

// OAuth2 client config for calling protected microservices
@Configuration
public class OAuth2ClientConfig {

    @Bean
    public WebClient accountServiceClient(
        ReactiveClientRegistrationRepository clientRegistrations,
        ServerOAuth2AuthorizedClientRepository authorizedClients) {

        ServerOAuth2AuthorizedClientExchangeFilterFunction oauth =
            new ServerOAuth2AuthorizedClientExchangeFilterFunction(
                clientRegistrations, authorizedClients);
        oauth.setDefaultClientRegistrationId("account-service");

        return WebClient.builder()
            .baseUrl("https://account-service.internal")
            .filter(oauth) // Automatically adds Bearer token
            .build();
    }
}

# application.yml
spring:
  security:
    oauth2:
      client:
        registration:
          account-service:
            provider: internal-auth
            client-id: payment-service
            client-secret: ${OAUTH2_CLIENT_SECRET}
            authorization-grant-type: client_credentials
            scope: accounts:read
        provider:
          internal-auth:
            token-uri: https://auth.bank.com/oauth2/token

```

9.2 JWT — Security Deep Dive

JWT Structure

```

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.  -- Header: algorithm, type
eyJzdWIiOiJ1c2VyLTZyYyMyIsInR5cCI6IkpXVCJ9.  -- Payload: user info
RSASHA256signature -- Signature: verifies integrity

```

```
Payload decoded:
{
  "sub": "user-123",           // Subject (user ID)
  "roles": ["USER"],          // Custom roles
  "scp": ["payments:read"],    // Scopes
  "exp": 1709139200,           // Expiration
  "iat": 1709135600,           // Issued at
  "jti": "abc-123",            // JWT ID (for revocation)
  "iss": "https://auth.bank.com", // Issuer
  "aud": "payment-service"     // Audience
}
```

JWT Security Best Practices

```
// JWT validation – ALL these checks MUST happen
@Component
public class JwtValidator {

    public Claims validateAndExtract(String token) {
        try {
            return Jwts.parserBuilder()
                .setSigningKey(publicKey)           // Verify signature
                .requireIssuer("https://auth.bank.com") // Verify issuer
                .requireAudience("payment-service")   // Verify intended audience
                .setAllowedClockSkewSeconds(30)        // Allow 30s clock drift
                .build()
                .parseClaimsJws(token)
                .getBody();
        } catch (ExpiredJwtException e) {
            throw new UnauthorizedException("Token expired");
        } catch (JwtException e) {
            // Don't reveal WHY validation failed – security
            log.warn("JWT validation failed: {}", e.getClass().getSimpleName());
            throw new UnauthorizedException("Invalid token");
        }
    }
}

// Common JWT vulnerabilities to prevent:

// 1. alg:none attack – NEVER accept "none" algorithm
// Spring's NimbusJwtDecoder with RS256 prevents this automatically

// 2. Algorithm confusion (RS256 → HS256) – use separate public/private key configs
// Always explicitly specify allowed algorithm

// 3. Token not expiring – always set exp, typical: 15min access, 7d refresh

// 4. Sensitive data in payload – JWT is base64 encoded, NOT encrypted
// NEVER put: passwords, SSN, PII in JWT payload

// 5. Missing audience check – prevents tokens for service A being used at service B

// Token revocation (stateless JWT cannot be revoked – workarounds):
// Option 1: Short expiry (15 min) + refresh token rotation
// Option 2: Token blacklist in Redis (compromise: stateful)
// Option 3: JTI (JWT ID) tracking – check Redis on each request
```

9.3 OWASP Top 10 — Java Mitigations

A1: Injection (SQL, LDAP, OS Command)

```
// SQL Injection – NEVER use string concatenation
// WRONG:
String sql = "SELECT * FROM users WHERE name = " + userInput + " ";
// If userInput = "; DROP TABLE users; --" → catastrophic
```

```

// CORRECT: Parameterized queries
@Query("SELECT u FROM User u WHERE u.name = :name")
Optional<User> findByName(@Param("name") String name);

// JdbcTemplate parameterized
jdbcTemplate.query(
    "SELECT * FROM users WHERE email = ?",
    new Object[]{email}, // Properly escaped by driver
    rowMapper
);

// NEVER construct JPQL/HQL dynamically with user input
// Use Specification API (JPA Criteria) for dynamic queries:
public Specification<Payment> withFilters(PaymentFilter filter) {
    return (root, query, cb) -> {
        List<Predicate> predicates = new ArrayList<>();
        if (filter.getStatus() != null) {
            predicates.add(cb.equal(root.get("status"), filter.getStatus()));
        }
        if (filter.getFromDate() != null) {
            predicates.add(cb.greaterThanOrEqualTo(root.get("createdAt"), filter.getFromDate()));
        }
        return cb.and(predicates.toArray(new Predicate[0]));
    };
}

```

A2: Broken Authentication

```

// Secure password storage
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder(12); // Cost 12 → ~300ms per hash (brute-force resistant)
}

// Account lockout after failed attempts
@Service
public class AuthenticationService {
    private final LoadingCache<String, AtomicInteger> loginAttempts =
        CacheBuilder.newBuilder()
            .expireAfterWrite(15, TimeUnit.MINUTES)
            .build(CacheLoader.from(k -> new AtomicInteger(0)));

    public LoginResult login(String email, String password) {
        int attempts = loginAttempts.getUnchecked(email).get();
        if (attempts >= 5) {
            throw new AccountLockedException("Account locked. Try again in 15 minutes.");
        }

        User user = userRepo.findByEmail(email)
            .orElseThrow(() -> {
                loginAttempts.getUnchecked(email).incrementAndGet();
                return new BadCredentialsException("Invalid credentials");
            });

        if (!passwordEncoder.matches(password, user.getPasswordHash())) {
            loginAttempts.getUnchecked(email).incrementAndGet();
            throw new BadCredentialsException("Invalid credentials");
        }

        loginAttempts.invalidate(email); // Reset on success
        return generateTokens(user);
    }
}

```

A3: Sensitive Data Exposure

```

// Mask sensitive data in logs
@Slf4j
public class PaymentService {
    public void processCard(CardPaymentRequest request) {

```

```

        log.info("Processing card payment: last4={} amount={}",
            request.getCardNumber().substring(request.getCardNumber().length() - 4),
            request.getAmount());
        // NEVER: log.info("Card: {}", request.getCardNumber());
    }
}

// Sensitive field masking in Jackson (JSON responses)
public class CardDetails {
    @JsonProperty
    private String cardholderName;

    @JsonSerialize(using = MaskedCardSerializer.class)
    private String cardNumber; // Serializes as "****-****-****-1234"

    @JsonIgnore
    private String cvv; // Never include in any response
}

// HTTPS enforcement
http.requiresChannel()
    .requestMatchers(r -> r.getHeader("X-Forwarded-Proto") != null)
    .requiresSecure();

// HSTS header
http.headers(headers -> headers
    .httpStrictTransportSecurity(hsts -> hsts
        .includeSubDomains(true)
        .maxAgeInSeconds(31536000)));

```

A4: XML External Entity (XXE)

```

// Safe XML parsing – disable external entities
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
factory.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);
factory.setFeature("http://xml.org/sax/features/external-general-entities", false);
factory.setFeature("http://xml.org/sax/features/external-parameter-entities", false);
factory.setExpandEntityReferences(false);
DocumentBuilder builder = factory.newDocumentBuilder();
// Now safe to parse untrusted XML

```

A5: Broken Access Control

```

// ALWAYS verify ownership – never trust client-provided IDs
@Service
public class AccountService {

    @PreAuthorize("@accountSecurity.isOwnerOrAdmin(#accountId, authentication)")
    public Account getAccount(String accountId) { ... }

    // Custom security expression
    @Component("accountSecurity")
    public class AccountSecurityExpressions {
        public boolean isOwnerOrAdmin(String accountId, Authentication auth) {
            if (auth.getAuthorities().stream()
                .anyMatch(a -> a.getAuthority().equals("ROLE_ADMIN"))) {
                return true;
            }
            Account account = accountRepo.findById(accountId).orElse(null);
            return account != null && account.getOwnerId().equals(auth.getName());
        }
    }
}

```


9.4 Secrets Management

```
// Spring Vault integration
@Configuration
public class VaultConfig extends AbstractVaultConfiguration {

    @Override
    public VaultEndpoint vaultEndpoint() {
        return VaultEndpoint.from(URI.create("https://vault.internal.bank.com"));
    }

    @Override
    public ClientAuthentication clientAuthentication() {
        // Kubernetes auth – uses service account token
        return new KubernetesAuthentication(
            KubernetesAuthenticationOptions.builder()
                .role("payment-service")
                .build(),
            restOperations());
    }
}

# application.yml – Vault secrets injected as Spring properties
spring:
  cloud:
    vault:
      enabled: true
      uri: https://vault.internal.bank.com
      authentication: kubernetes
      kubernetes:
        role: payment-service
      kv:
        enabled: true
        backend: secret
        default-context: payment-service
        application-name: payment-service
# Secrets at vault path: secret/payment-service/
# Accessible as: ${database.password}, ${jwt.secret}
```

9.5 Secure Coding Checklist

```
// Input validation – always at API boundary
@Valid @RequestBody CreatePaymentRequest request

// Output encoding – prevent XSS
// Jackson's default JSON encoding prevents XSS in JSON APIs
// For HTML output: use Thymeleaf (auto-escapes) or explicitly encode

// Error messages – don't reveal internals
@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorResponse> handleGeneral(Exception e) {
    log.error("Unhandled exception", e); // Log full stack trace internally
    return ResponseEntity.status(500)
        .body(new ErrorResponse("INTERNAL_ERROR", "An error occurred"));
    // NEVER return e.getMessage() for unexpected exceptions
}

// Rate limiting on sensitive endpoints
@RateLimiter(name = "login-endpoint")
@PostMapping("/auth/login")
public ResponseEntity<TokenResponse> login(@RequestBody LoginRequest request) { ... }

// CSRF protection – enable for non-stateless APIs
// (stateless JWT APIs can disable CSRF since there's no session cookie)

// Dependency scanning
// Run: mvn dependency-check:check
// In CI: fail on CVSS score > 7
```

```
// Security headers
http.headers(headers -> headers
    .contentSecurityPolicy(csp -> csp
        .policyDirectives("default-src 'self'; script-src 'self'"))
    .frameOptions(fo -> fo.deny())
    .xssProtection(xss -> xss.block(true)));
```

Section Summary: Security Interview Questions

Banking/fintech companies always ask:

1. "How does OAuth2 client credentials flow work? Walk me through service-to-service auth."
2. "What are the JWT vulnerabilities you need to protect against?"
3. "How do you prevent SQL injection in a Spring Data JPA application?"
4. "How do you store sensitive configuration (DB passwords, API keys) in production?"
5. "What is the difference between authentication and authorization?"
6. "How would you implement row-level security? (user can only see their own data)"
7. "What OWASP vulnerabilities have you encountered and mitigated?"
8. "How do you handle token revocation with stateless JWTs?"
9. "What is PKCE and why is it needed for Authorization Code flow?"
10. "How do you audit sensitive operations in a banking system?"

Section 10: System Design

The Highest-Signal Interview Round: System design interviews reveal your architectural thinking, trade-off judgment, and engineering maturity. At FAANG and banks, you're expected to design systems that handle millions of transactions reliably. This section gives you the frameworks, patterns, and real-world examples.

10.1 System Design Framework — How to Approach Interviews

The 45-Minute Blueprint

Minutes 1-5: Requirements clarification
Minutes 5-10: Capacity estimation + constraints
Minutes 10-20: High-level design (HLD) — components and data flow
Minutes 20-35: Deep dive into critical components
Minutes 35-45: Trade-offs, bottlenecks, scaling strategies

Clarifying Questions Template

Functional Requirements:

- What are the core use cases?
- Who are the users? What's the access pattern?
- Read-heavy or write-heavy?
- Real-time or eventual consistency acceptable?

Non-Functional Requirements:

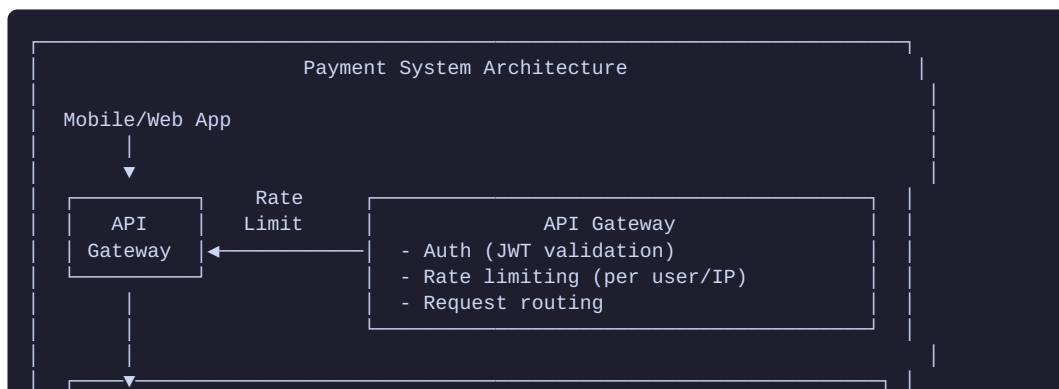
- Scale: How many users/transactions per day?
- Latency: P99 < 200ms? P50 < 50ms?
- Availability: 99.9%? 99.99%? (9s vs 52 min/year downtime)
- Consistency: Strong vs eventual?
- Durability: How much data loss is acceptable?
- Geography: Single region or global?

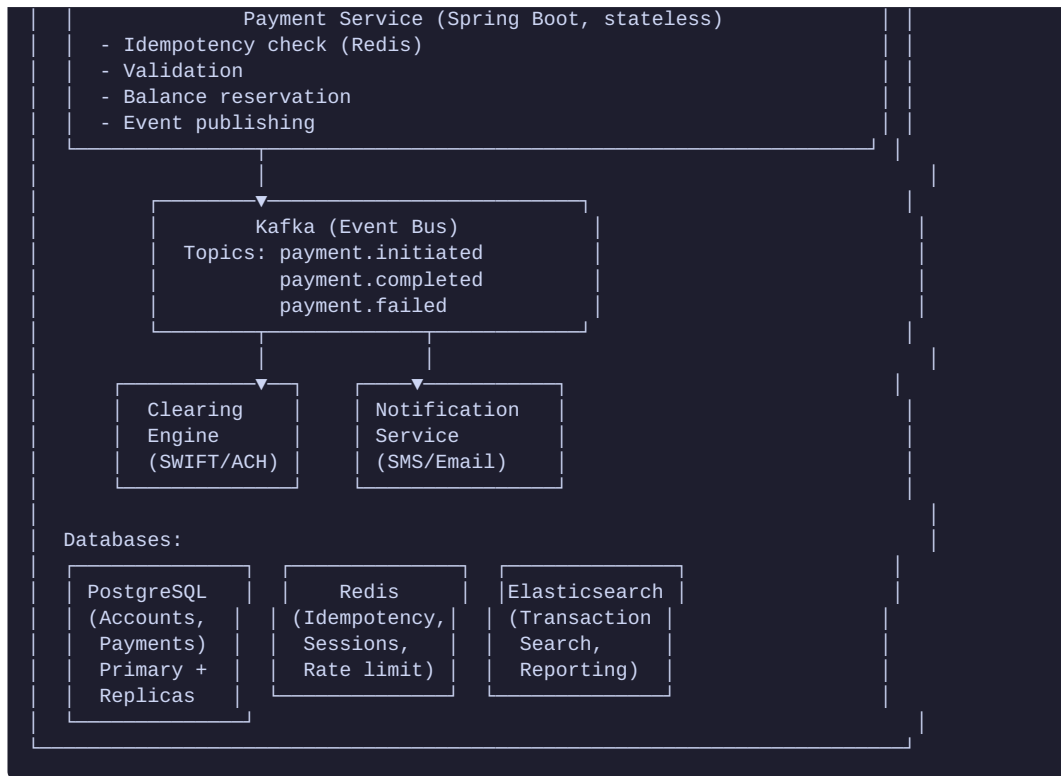
10.2 Banking Payment System Design

Problem: Design a Real-Time Payment Processing System

Requirements: - 50,000 transactions/second peak - P99 latency < 500ms - Exactly-once processing (no duplicate payments) - Immutable audit trail - Regulatory compliance (all transactions logged) - Multi-currency support

Architecture





Deep Dive: Exactly-Once Payment Processing

```
@Service
@Slf4j
public class PaymentProcessor {

    @Transactional
    public PaymentResponse initiatePayment(PaymentRequest request, String idempotencyKey) {

        // 1. Check idempotency (Redis – fast, distributed)
        String cached = redisTemplate.opsForValue().get("idem:" + idempotencyKey);
        if (cached != null) {
            return objectMapper.readValue(cached, PaymentResponse.class);
        }

        // 2. Reserve idempotency slot (with expiry to handle crashes)
        Boolean reserved = redisTemplate.opsForValue()
            .setIfAbsent("idem:" + idempotencyKey, "PROCESSING", Duration.ofMinutes(5));
        if (!Boolean.TRUE.equals(reserved)) {
            throw new ConflictException("Payment already in progress");
        }

        try {
            // 3. Validate and reserve funds
            Account fromAccount = accountRepo.findByIdForUpdate(request.getFromAccountId());
            validateAndReserveFunds(fromAccount, request.getAmount());

            // 4. Create payment record
            Payment payment = paymentRepo.save(new Payment(request, PaymentStatus.RESERVED));

            // 5. Publish to Kafka with Outbox pattern (transactional)
            outboxRepo.save(new OutboxMessage(
                "payment.initiated",
                payment.getId().toString(),
                objectMapper.writeValueAsString(PaymentEvent.from(payment))
            ));

            PaymentResponse response = PaymentResponse.from(payment);

            // 6. Cache response for idempotency
            redisTemplate.opsForValue().set("idem:" + idempotencyKey,
                objectMapper.writeValueAsString(response),
```

```

        Duration.ofHours(24));

        return response;

    } catch (Exception e) {
        redisTemplate.delete("idem:" + idempotencyKey);
        throw e;
    }
}
}

```

10.3 Distributed Caching with Redis — Patterns

Cache Patterns

```

// Cache-Aside (most common)
@Service
public class AccountService {

    public Account getAccount(String accountId) {
        // 1. Check cache
        Account cached = redisTemplate.opsForValue().get("account:" + accountId);
        if (cached != null) return cached;

        // 2. Cache miss – query DB
        Account account = accountRepo.findById(accountId)
            .orElseThrow(() -> new AccountNotFoundException(accountId));

        // 3. Populate cache
        redisTemplate.opsForValue().set("account:" + accountId, account, Duration.ofMinutes(5));
        return account;
    }

    // Invalidate on update
    @CacheEvict(value = "accounts", key = "#account.id")
    public Account updateAccount(Account account) {
        return accountRepo.save(account);
    }
}

// Write-Through – write to cache and DB simultaneously
@Transactional
public void updateBalance(String accountId, BigDecimal newBalance) {
    accountRepo.updateBalance(accountId, newBalance);
    redisTemplate.opsForValue().set("balance:" + accountId, newBalance, Duration.ofMinutes(10));
}

// Read-Through – Spring @Cacheable (auto cache + DB)
@Cacheable(value = "currencies", key = "#code", unless = "#result == null")
public Currency getCurrency(String code) {
    return currencyRepo.findByCode(code).orElse(null);
}

```

Redis Data Structures for Enterprise Use

```

// Rate limiting with sliding window (Redis sorted set)
public boolean isAllowed(String clientId) {
    long now = System.currentTimeMillis();
    long windowStart = now - WINDOW_SIZE_MS;
    String key = "ratelimit:" + clientId;

    // Remove expired entries
    redisTemplate.opsForZSet().removeRangeByScore(key, 0, windowStart);

    // Count requests in window
    Long count = redisTemplate.opsForZSet().count(key, windowStart, now);
    if (count < MAX_REQUESTS) {
        redisTemplate.opsForZSet().add(key, now + "-" + UUID.randomUUID(), now);
    }
}

```

```

        redisTemplate.expire(key, Duration.ofMillis(WINDOW_SIZE_MS));
        return true;
    }
    return false;
}

// Distributed lock (Redisson)
RLock lock = redisson.getLock("transfer:" + accountId);
try {
    if (lock.tryLock(3, 10, TimeUnit.SECONDS)) { // Wait 3s, hold 10s max
        try {
            performTransfer(accountId, amount);
        } finally {
            lock.unlock();
        }
    } else {
        throw new ConcurrentTransferException("Transfer already in progress");
    }
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    throw new RuntimeException("Lock interrupted", e);
}

// Leaderboard (sorted set)
redisTemplate.opsForZSet().incrementScore("payment:volumes:daily", merchantId, amount.doubleValue());
Set<ZSetOperations.TypedTuple<String>> topMerchants =
    redisTemplate.opsForZSet().reverseRangeWithScores("payment:volumes:daily", 0, 9);

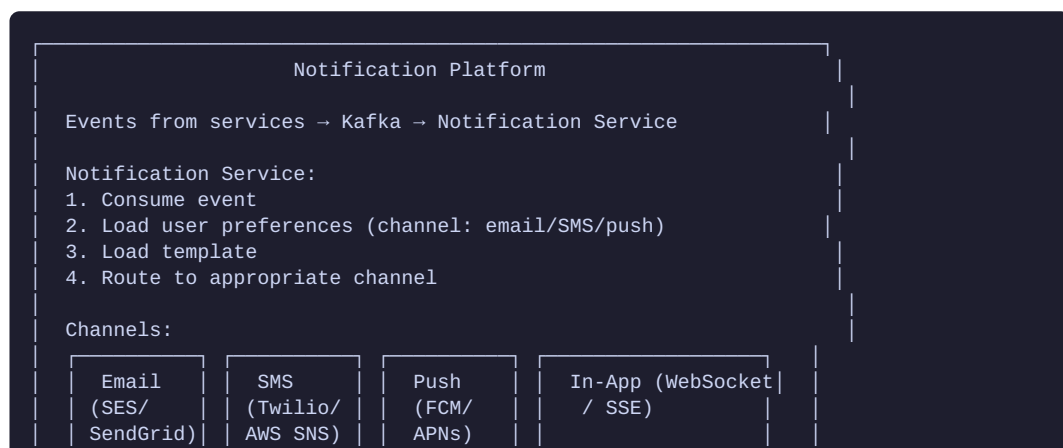
```

Cache Invalidation Strategies

1. TTL-based (simplest): data expires after N seconds
 - Risk: stale data up to TTL
 - Use for: reference data (currencies, countries), user preferences
2. Event-driven invalidation:
 - PaymentUpdated event → evict payment:* keys
 - Use for: frequently-changing data that needs near-real-time consistency
3. Cache stampede prevention:
 - When cache expires, many requests hit DB simultaneously
 - Solution: Probabilistic early expiration (refresh before expiry)
 - Solution: Locking (only one thread fetches, rest wait)
4. Write-through/write-behind:
 - Always write to cache + DB
 - Guarantees freshness
 - Use for: hot data (balances, session state)

10.4 Notification System Design

High-Level Design



```
Delivery tracking: status updates back to Kafka
```

```
// Template-driven notification system
@Service
public class NotificationService {

    @KafkaListener(topics = "payment.completed")
    public void handlePaymentCompleted(PaymentCompletedEvent event) {
        UserPreferences prefs = userPrefService.getPreferences(event.getUserId());

        NotificationContext context = NotificationContext.builder()
            .userId(event.getUserId())
            .templateId("PAYMENT_COMPLETED")
            .data(Map.of(
                "amount", event.getAmount(),
                "currency", event.getCurrency(),
                "recipientName", event.getRecipientName(),
                "timestamp", event.getCompletedAt()
            ))
            .build();

        // Send via preferred channels
        if (prefs.isEmailEnabled()) {
            emailChannel.send(context, prefs.getEmail());
        }
        if (prefs.isSmsEnabled()) {
            smsChannel.send(context, prefs.getPhoneNumber());
        }
        if (prefs.isPushEnabled()) {
            pushChannel.send(context, prefs.getDeviceTokens());
        }
    }
}
```

10.5 URL Shortener — LLD + HLD

This is a classic interview system design question:

```
HLD:
User → POST /shorten → ShortenerService → generate shortCode → store mapping
User → GET /{code} → RedirectService → lookup code → 301 redirect

Core Design Decisions:
1. Short code generation:
  - Option A: Base62 encode auto-increment DB ID (simple, predictable → security risk)
  - Option B: MD5/SHA hash of URL (collision risk)
  - Option C: Random base62 (best: unpredictable, no collision if long enough)

2. Storage: Redis for hot redirects + PostgreSQL for persistence
3. Scale: 10B URLs, 1K writes/sec, 10K reads/sec
  → Read:write ratio 10:1 → cache-heavy

4. Short code length:
  6 chars × 62 options = 56 billion combinations (sufficient for years)
```

```
@Service
public class UrlShortenerService {

    private final UrlRepository urlRepo;
    private final StringRedisTemplate redis;

    public String shorten(String longUrl, String userId) {
        // Idempotent: same URL from same user gets same code
        Optional<String> existing = urlRepo.findCodeByUrlAndUser(longUrl, userId);
        if (existing.isPresent()) return buildShortUrl(existing.get());
    }
}
```

```

        String code = generateCode();
        while (urlRepo.existsByCode(code)) {
            code = generateCode(); // Rare collision retry
        }

        urlRepo.save(new ShortUrl(code, longUrl, userId, Instant.now()));
        redis.opsForValue().set("url:" + code, longUrl, Duration.ofDays(30));
        return buildShortUrl(code);
    }

    public String redirect(String code) {
        // Cache hit: O(1) Redis lookup
        String longUrl = redis.opsForValue().get("url:" + code);
        if (longUrl != null) return longUrl;

        // Cache miss: DB lookup + repopulate cache
        return urlRepo.findByCode(code)
            .map(url -> {
                redis.opsForValue().set("url:" + code, url.getLongUrl(), Duration.ofDays(30));
                return url.getLongUrl();
            })
            .orElseThrow(() -> new UrlNotFoundException(code));
    }

    private String generateCode() {
        return RandomStringUtils.randomAlphanumeric(6);
    }
}

```

10.6 Scalability Patterns

Database Scaling

Vertical: Bigger server (limited, expensive)
 Horizontal Read: Add read replicas (scales reads, not writes)
 Horizontal Write: Sharding

Sharding strategies:

- Range-based: accounts A-M → Shard 1, N-Z → Shard 2
 → Simple but uneven distribution (hotspots)
- Hash-based: $\text{hash}(\text{accountId}) \% N \rightarrow \text{shard number}$
 → Even distribution, but resharding is painful
- Directory-based: lookup table maps entity → shard
 → Flexible, but lookup table becomes bottleneck

For banking: AVOID sharding if possible
 Use: Connection pooling, read replicas, caching, efficient queries
 Last resort: Logical sharding at application level

API Design for Scale

```

// Cursor-based pagination (better than offset for large datasets)
@GetMapping("/payments")
public CursorPage<PaymentResponse> listPayments(
    @RequestParam(required = false) String cursor,
    @RequestParam(defaultValue = "20") int limit) {

    UUID lastId = cursor != null ? decodeCursor(cursor) : null;
    List<Payment> payments = paymentRepo.findAfterCursor(lastId, limit + 1);

    boolean hasMore = payments.size() > limit;
    List<Payment> page = hasMore ? payments.subList(0, limit) : payments;

    String nextCursor = hasMore ? encodeCursor(page.get(page.size() - 1).getId()) : null;
    return new CursorPage<>(page.stream().map(PaymentResponse::from).toList(), nextCursor);
}

```



```
}

// Repository using cursor
@Query("""
    SELECT p FROM Payment p
    WHERE (:cursor IS NULL OR p.id > :cursor)
    ORDER BY p.id ASC
    LIMIT :limit
    """)
List<Payment> findAfterCursor(UUID cursor, int limit);
```

Section Summary: System Design Interview Topics

Essential designs to practice:

1. **Payment processing system** (banks will ask this)
2. **Notification platform** (commonly asked)
3. **URL shortener** (classic LLD + HLD)
4. **Rate limiter** (real implementation at every company)
5. **Distributed cache** (Redis patterns)
6. **API gateway** (routing, auth, rate limiting)
7. **Event-driven microservices** (Kafka-based system)
8. **Read-heavy system** (caching layers, read replicas)
9. **Write-heavy system** (buffering, batching, async writes)
10. **Search system** (Elasticsearch integration)

Framework to evaluate trade-offs: - Latency vs consistency - Throughput vs durability - Simplicity vs scalability - Cost vs performance

Section 11: Interview Preparation

Elite Performance: FAANG and top banks don't just test knowledge — they test how you think, communicate, and handle ambiguity. This section gives you the exact preparation strategy, question banks, and mental models to perform at the highest level.

11.1 DSA for Experienced Engineers

The Right Mindset

You are interviewing as a senior engineer. Companies expect: - LeetCode Medium fluency (< 20 minutes) - LeetCode Hard recognition (explain approach even if not fully coded) - Optimal time/space complexity - Clean, readable code (not just working code) - Edge case awareness - Code review quality

Data Structures You Must Know Cold

```
// 1. HashMap patterns – constant time lookups
// "Two Sum" pattern applied to finance: find pairs of transactions summing to X
public int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (seen.containsKey(complement)) {
            return new int[]{seen.get(complement), i};
        }
        seen.put(nums[i], i);
    }
    return new int[]{};
}

// 2. Sliding Window – rate limiting, moving averages
// "Max sum of K consecutive elements"
public int maxSumWindow(int[] arr, int k) {
    int sum = 0;
    for (int i = 0; i < k; i++) sum += arr[i];
    int maxSum = sum;
    for (int i = k; i < arr.length; i++) {
        sum += arr[i] - arr[i - k];
        maxSum = Math.max(maxSum, sum);
    }
    return maxSum;
}

// 3. Stack – expression evaluation, bracket matching
// Applied in: query parsers, undo/redo systems
public boolean isValidBrackets(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    Map<Character, Character> pairs = Map.of(')', '(', ']', '['), '}', '{');
    for (char c : s.toCharArray()) {
        if ("([{" .indexOf(c) >= 0) {
            stack.push(c);
        } else {
            if (stack.isEmpty() || stack.pop() != pairs.get(c)) return false;
        }
    }
    return stack.isEmpty();
}

// 4. BFS/DFS – dependency graphs, permission trees, graph traversal
// Applied in: permission hierarchy, dependency resolution
public List<Integer> topologicalSort(int n, int[][] edges) {
    Map<Integer, List<Integer>> graph = new HashMap<>();
    int[] inDegree = new int[n];
```

```

        for (int[] edge : edges) {
            graph.computeIfAbsent(edge[0], k -> new ArrayList<>()).add(edge[1]);
            inDegree[edge[1]]++;
        }
        Queue<Integer> queue = new LinkedList<>();
        for (int i = 0; i < n; i++) {
            if (inDegree[i] == 0) queue.offer(i);
        }
        List<Integer> result = new ArrayList<>();
        while (!queue.isEmpty()) {
            int node = queue.poll();
            result.add(node);
            for (int neighbor : graph.getOrDefault(node, List.of())) {
                if (--inDegree[neighbor] == 0) queue.offer(neighbor);
            }
        }
        return result.size() == n ? result : List.of(); // Empty = cycle detected
    }

// 5. Binary Search – O(log n) search in sorted data
// Applied in: finding transaction in sorted timeline, search optimization
public int binarySearch(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2; // Avoid overflow
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}

// 6. Dynamic Programming – optimization problems
// Applied in: resource scheduling, rate optimization
public int maxProfit(int[] prices) { // LeetCode 121 – Buy/Sell Stock
    int minPrice = Integer.MAX_VALUE, maxProfit = 0;
    for (int price : prices) {
        minPrice = Math.min(minPrice, price);
        maxProfit = Math.max(maxProfit, price - minPrice);
    }
    return maxProfit;
}

```

Top 20 Patterns — Recognize → Solve

- | | |
|-------------------------|--|
| 1. Two Pointers | → Sorted arrays, palindrome, remove duplicates |
| 2. Sliding Window | → Subarray problems, moving window |
| 3. HashMap/HashSet | → Frequency counting, deduplication, O(1) lookup |
| 4. Stack | → Balanced brackets, next greater element, monotonic |
| 5. Queue/BFS | → Level-order traversal, shortest path |
| 6. DFS/Recursion | → Tree problems, backtracking |
| 7. Binary Search | → Sorted array search, search space reduction |
| 8. Merge Sort / Sort | → Interval merging, K sorted arrays |
| 9. Heap/PriorityQueue | → K largest/smallest, merge K lists |
| 10. Dynamic Programming | → Optimal substructure, memoization |
| 11. Graph (BFS/DFS) | → Connected components, shortest path, cycle detection |
| 12. Union-Find | → Connected components, network connectivity |
| 13. Trie | → Prefix search, word problems |
| 14. Bit Manipulation | → Single number, counting bits, XOR tricks |
| 15. Greedy | → Activity selection, interval scheduling |
| 16. Backtracking | → Permutations, combinations, N-Queens |
| 17. Divide & Conquer | → Merge sort, binary search variants |
| 18. Tree Traversal | → Inorder/preorder/postorder patterns |
| 19. String Manipulation | → Substring, pattern matching, anagram |
| 20. Math | → Modular arithmetic, prime numbers, GCD |

11.2 Java Interview Questions

Core Java

Q: What is the difference between `==` and `.equals()` in Java?

`==` compares references (memory addresses). `.equals()` compares content. For `String`, `Integer` (cached -128 to 127), always use `.equals()`. For `null` check, use `==`.

Q: Explain the contract between `equals()` and `hashCode()`.

If `a.equals(b)` is true, then `a.hashCode()` must equal `b.hashCode()`. The reverse is not required. Violating this breaks `HashMap`/`HashSet` behavior.

Q: What is the difference between `HashMap`, `LinkedHashMap`, and `TreeMap`?

`HashMap`: Unordered, $O(1)$ average. `LinkedHashMap`: Insertion-ordered, $O(1)$. `TreeMap`: Sorted by key, $O(\log n)$. Java 8+ `HashMap` uses trie-based buckets (`TreeMap` structure) for buckets with > 8 entries.

Q: What are the different ways to create a thread in Java? Which is preferred?

`Thread` subclass, `Runnable` lambda, `ExecutorService` (preferred). Raw thread creation in production is an anti-pattern. Use `ThreadPoolExecutor` or Spring's `@Async`.

Q: Explain `volatile` vs `synchronized`.

`volatile`: Guarantees visibility (reads/writes directly to main memory, not CPU cache). Does NOT guarantee atomicity for compound operations. `synchronized`: Guarantees both visibility AND atomicity (mutual exclusion). Use `volatile` for simple flag variables; `synchronized` or `AtomicXxx` for compound operations.

Q: What is `Comparable` vs `Comparator`?

`Comparable`: Object's natural ordering (implements `compareTo`). Used by `TreeMap` / `TreeSet` by default. `Comparator`: External comparison strategy. Used when you need multiple sort orderings or can't modify the class.

11.3 Spring Interview Questions

Q: How does Spring dependency injection work internally?

Spring creates a `BeanFactory` (or `ApplicationContext`). At startup, it scans for `@Component`, `@Bean`, `@Configuration`. It creates `BeanDefinition` objects describing each bean. Then it instantiates beans, resolves constructor/setter dependencies, applies `BeanPostProcessor`s (where AOP proxies are created), calls `@PostConstruct` methods, and marks beans as ready.

Q: What is the difference between `@Component`, `@Service`, `@Repository`, and `@Controller`?

All four are `@Component` specializations — functionally equivalent for DI purposes. The semantic difference: `@Repository` enables exception translation (converts SQL exceptions to `Spring DataAccessException`). `@Service`, `@Controller` are documentation/tooling hints. Use them consistently for code clarity.

Q: Explain the `@Transactional` pitfalls.

- Self-invocation:** Calling `@Transactional` method from same class bypasses proxy — no transaction.
- Visibility:** `@Transactional` on private methods is silently ignored.
- Rollback rules:** By default, only `RuntimeException` rolls back (not checked exceptions).
- Propagation:** `REQUIRES_NEW` suspends outer transaction — completely separate transaction.
- Isolation levels:** Higher isolation = more locking = lower throughput.

Q: What is Spring AOP and how does it work?

Spring AOP creates proxy objects around beans (JDK dynamic proxy for interface-based or CGLIB subclass proxy). Method calls go through the proxy, which checks for applicable advice (Before, After, Around, AfterReturning, AfterThrowing). `@Transactional`, `@Cacheable`, `@Async`, `@Retryable` all use AOP.

Q: How does Spring Boot autoconfiguration work?

Spring Boot reads `META-INF/spring/`
`org.springframework.boot.autoconfigure.AutoConfiguration.imports` (or `spring.factories` in older versions). These are `@Configuration` classes with `@ConditionalOnClass`, `@ConditionalOnMissingBean` conditions. Example: `DataSourceAutoConfiguration` activates only if `DataSource` class is on classpath and no `DataSource` bean is already defined.

11.4 LLD (Low-Level Design) Questions

Design a Thread-Safe LRU Cache

```
// LRU Cache: O(1) get and put
// Data structure: HashMap + Doubly Linked List
public class LRUCache<K, V> {
    private final int capacity;
    private final Map<K, Node<K, V>> map = new LinkedHashMap<>() {
        @Override
        protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
            return size() > capacity;
        }
    };

    // Thread-safe version
    private final Map<K, V> cache = Collections.synchronizedMap(
        new LinkedHashMap<>(16, 0.75f, true) {
            @Override
            protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
                return size() > capacity;
            }
        }
    );

    // Better: Caffeine (production-grade caching library)
    // Caffeine.newBuilder()
    //     .maximumSize(1000)
    //     .expireAfterWrite(5, MINUTES)
    //     .recordStats()
    //     .build()
}
```

Design a Rate Limiter

```
// Token Bucket Algorithm
public class TokenBucketRateLimiter {
    private final long capacity;
    private final long refillRate; // tokens per second
    private AtomicLong tokens;
    private volatile long lastRefillTime;

    public synchronized boolean tryAcquire() {
        refill();
        if (tokens.get() > 0) {
            tokens.decrementAndGet();
            return true;
        }
        return false;
    }

    private void refill() {
        long now = System.currentTimeMillis();
        long elapsed = now - lastRefillTime;
    }
}
```

```

        long tokensToAdd = elapsed * refillRate / 1000;
        if (tokensToAdd > 0) {
            tokens.set(Math.min(capacity, tokens.get() + tokensToAdd));
            lastRefillTime = now;
        }
    }
}

// Sliding Window Counter
public class SlidingWindowRateLimiter {
    private final Deque<Long> timestamps = new ArrayDeque<>();
    private final int maxRequests;
    private final long windowMs;

    public synchronized boolean isAllowed() {
        long now = System.currentTimeMillis();
        // Remove timestamps outside window
        while (!timestamps.isEmpty() && timestamps.peekFirst() <= now - windowMs) {
            timestamps.pollFirst();
        }
        if (timestamps.size() < maxRequests) {
            timestamps.addLast(now);
            return true;
        }
        return false;
    }
}

```

Design Patterns — Must Know for LLD

```

Creational:
Singleton    → Spring beans (ApplicationContext manages)
Factory      → PaymentProcessorFactory.create(type)
Builder      → Request/Response objects, complex configs
Prototype    → Clone-able entities

Structural:
Decorator    → Logging wrapper, cache wrapper around service
Adapter      → Integrate legacy system with new interface
Facade       → Simplify complex subsystem (PaymentFacade → multiple services)
Proxy        → Spring AOP, lazy loading, access control

Behavioral:
Strategy     → PaymentStrategy (Stripe/Braintree/PayPal)
Observer     → Event publishing (ApplicationEventPublisher)
Template Method → BaseService with abstract processBusiness()
Command      → Undo/redo, queued operations
Chain of Responsibility → Filter chain, validation pipeline

```

11.5 HLD (High-Level Design) Questions

Framework for Any Design Question

1. Understand the problem
 - What are we building? Who uses it?
 - Scale: users, requests per second, data volume
 - Consistency requirements: strong vs eventual
 - Availability: 99.9%? 99.99%?
2. Define the API
 - REST endpoints or events
 - Request/Response contracts
3. High-level components
 - Draw the boxes: clients, API gateway, services, databases, queues
 - Draw the data flows between components
4. Data model

- Key entities and relationships
- Which database type? SQL vs NoSQL rationale

5. Scale and deep dive

- Where are the bottlenecks?
- Caching strategy
- Scaling strategy (horizontal, sharding, read replicas)
- Failure scenarios and mitigation

6. Trade-offs

- What did you optimize for? What did you sacrifice?
- What would you do differently at 10x scale?

11.6 Behavioral Interview — STAR Method

FAANG Behavioral Principles (Amazon LPs)

Ownership:	"Tell me about a time you took ownership beyond your role"
Dive Deep:	"Tell me about a production incident you debugged"
Bias for Action:	"When did you make a decision with insufficient data?"
Disagree+Commit:	"When did you push back on a technical decision?"
Customer Obsession:	"How did you build something users actually needed?"
Invent+Simplify:	"When did you simplify a complex system?"
Think Big:	"Describe a long-term architectural vision you drove"
Deliver Results:	"Most impactful technical contribution in your career"

STAR Format

Situation: Set the context (team size, scale, company stage)
Task: What was your specific responsibility?
Action: What did YOU specifically do? (Use "I", not "we")
Result: Quantifiable outcome (latency, uptime, cost, velocity)

Example: "Production incident response"

Situation: Our payment service was dropping 5% of transactions at 3 AM on a Friday. High-severity P0 incident.

Task: I was on-call. My responsibility: diagnose and resolve within our 30-minute SLA.

Action: I pulled thread dumps and found all HTTP handler threads BLOCKED waiting for HikariCP connections.
Checked metrics: DB connection pool 100% utilized.
Found a new query without an index doing full table scans, holding connections for 30+ seconds.
Added the index (online, no lock), deployed in 12 minutes.

Result: Error rate dropped to 0% within 2 minutes of index creation.
Implemented automated slow query alerting to prevent recurrence.
P99 latency improved 40% as bonus from the index.

11.7 Real FAANG-Level Expectations

What "Senior" Means at Google/Amazon/Stripe

Junior → Senior → Staff → Principal

At Senior level (L5 Google, SDE2 Amazon, Senior Stripe):

- Design and implement features independently
- Identify technical risks and mitigation strategies
- Mentor junior engineers
- Make architectural decisions for your service
- On-call: diagnose and resolve incidents

- Code review: spot security, performance, correctness issues

Interview signal they're looking for:

- Do you think beyond the happy path?
- Do you consider scale without being asked?
- Do you ask clarifying questions or make assumptions?
- Can you communicate trade-offs clearly?
- Do you write code like it's going to production?

Code Quality Standards in Interviews

```
// Mediocre (gets functional credit):
public List<Integer> findDups(List<Integer> list) {
    List<Integer> result = new ArrayList<>();
    for (int i = 0; i < list.size(); i++) {
        for (int j = i+1; j < list.size(); j++) {
            if (list.get(i).equals(list.get(j))) {
                result.add(list.get(i));
            }
        }
    }
    return result;
}

// Excellent (gets strong hire):
public Set<Integer> findDuplicates(List<Integer> numbers) {
    // Handle null input
    if (numbers == null || numbers.isEmpty()) return Set.of();

    Set<Integer> seen = new HashSet<>();
    Set<Integer> duplicates = new HashSet<>();

    for (int num : numbers) {
        if (!seen.add(num)) { // add returns false if already present
            duplicates.add(num);
        }
    }
    return Collections.unmodifiableSet(duplicates);
    // O(n) time, O(n) space – explain why this is better than O(n²)
}
```

Section Summary: Interview Strategy

90-Day Interview Sprint Plan:

Month 1: Foundation - LeetCode Easy: 50 problems (all patterns) - LeetCode Medium: 30 problems - Java fundamentals + Spring concepts

Month 2: Depth - LeetCode Medium: 50 more problems - LeetCode Hard: 10 problems - System design: study 5 real designs - Mock interviews: 2/week with peers

Month 3: Polish - Mock interviews: 5/week - Behavioral stories: 10 prepared STAR stories - Revisit weak areas - Company-specific research (tech blog, engineering papers)

Day-of interview tips: 1. Think aloud — interviewers want to see your reasoning 2. Clarify before coding — always 3. Start with brute force, then optimize 4. Test with examples on paper before running 5. Handle edge cases explicitly (null, empty, single element)

Section 12: AI Era Engineering

The New Reality: AI tools can write syntactically correct code faster than any human. The question is not "can you type code?" — it's "do you understand what the code does, why it's wrong, and how to make it production-ready?" Engineering judgment, architecture thinking, and debugging intuition are the moats of the AI era.

12.1 How AI Changes Software Engineering

What AI Does Well (Commoditized)

AI excels at:

- ✓ Generating boilerplate (CRUD controllers, DTOs, tests)
- ✓ Completing patterns it has seen millions of times
- ✓ Translating between languages/frameworks
- ✓ Explaining unfamiliar code
- ✓ Writing unit tests for defined functions
- ✓ Converting between SQL dialects
- ✓ Documentation generation
- ✓ Configuration file generation
- ✓ Regex patterns
- ✓ Standard algorithm implementations

What AI Does Poorly (Your Moat)

AI struggles with:

- ✗ Understanding YOUR specific system's invariants
- ✗ Knowing which trade-off is right for YOUR constraints
- ✗ Debugging distributed system failures from partial logs
- ✗ Knowing when NOT to use a pattern
- ✗ System-level judgment ("this will cause thundering herd at scale")
- ✗ Evaluating correctness of concurrent code
- ✗ Security reasoning ("this allows SQL injection in 3 steps")
- ✗ Architecture decisions with organizational context
- ✗ Diagnosing GC issues from JVM metrics
- ✗ Understanding technical debt and its accumulation cost

The Shift in Skill Hierarchy

Pre-AI era:

1. Syntax fluency → most valued (scarcity of people who could code)
2. Algorithm knowledge
3. Framework knowledge
4. System design
5. Architecture judgment

AI era:

1. Architecture judgment → most valued (AI can't replace)
2. System design
3. Debugging and incident response
4. Code review and correctness judgment
5. Syntax fluency (still needed, but table stakes)

12.2 AI-Assisted Coding Workflow — The Right Way

Using Cursor/Copilot Effectively

```
Level 1 (Novice AI user): Accept completions blindly
  - Dangerous: ships bugs, security issues, wrong patterns

Level 2 (Competent AI user): Review completions critically
  - Better: catches obvious issues, but misses subtle ones

Level 3 (Elite AI user): AI generates first draft, engineer designs architecture
  - AI writes the code within your architectural constraints
  - You define the contract, invariants, error handling, concurrency model
  - AI fills in the implementation
```

Effective AI Prompting for Java/Spring

```
Bad prompt: "Write a payment service"

Good prompt: "Write a Spring Boot service class PaymentService with:
- Constructor injection (not field injection)
- Methods: createPayment(CreatePaymentRequest, String idempotencyKey)
  returning CompletableFuture<PaymentResponse>
- Idempotency check using RedisTemplate
- @Transactional on the DB write portion only
- Custom exception: DuplicatePaymentException for existing idempotencyKey
- Structured logging with MDC for requestId and userId
- The service should be thread-safe for 200 concurrent requests"

Better prompt structure:
1. What it IS (class type, package)
2. What it DOES (specific method contracts)
3. CONSTRAINTS (threading, transactions, error handling)
4. QUALITY REQUIREMENTS (logging, metrics, null safety)
5. What it is NOT (what NOT to include)
```

Code Review Checklist for AI-Generated Code

```
// When reviewing AI-generated Java code, check for:

// 1. Thread safety – is shared state properly synchronized?
// Bad AI pattern: HashMap in @Service (singleton, shared state)
private Map<String, Object> cache = new HashMap<>(); // WRONG – not thread-safe!

// 2. Resource leaks – are connections/streams closed?
// Bad AI pattern: unclosed resources
Connection conn = dataSource.getConnection();
// ... uses conn but never calls conn.close() ...

// 3. Exception handling – does it hide errors?
// Bad AI pattern: swallowing exceptions
try {
    riskyOperation();
} catch (Exception e) {
    // "handled" – WRONG – silent failure, debugging nightmare
}

// 4. Transaction boundaries – are they correct?
// Bad AI pattern: @Transactional on controller (too broad)
// Or: no transaction where money movement happens

// 5. SQL injection – are inputs parameterized?
// Bad AI pattern: string concatenation in JPQL/SQL

// 6. Infinite retry – does retry have a limit and backoff?
// Bad AI pattern: while(true) { retry() }

// 7. Wrong collection choice – ConcurrentHashMap vs HashMap
```

```
// AI often picks HashMap when code is in a bean (singleton)

// 8. Missing null checks – Optional vs null return
// AI often returns null from methods that should return Optional

// 9. Magic numbers – unexplained constants
// 10. Missing input validation – assuming valid input at service layer
```

12.3 Architecture Thinking — The Human Advantage

The Questions AI Cannot Answer for You

"Should we use event sourcing for the payment ledger?"

AI can tell you: what event sourcing is, how to implement it

AI cannot tell you:

- Does your team have operational experience with it?
- Does your audit compliance requirement actually need full replay capability?
- Is the query complexity worth it for your use case?
- What's your team's domain model maturity?

These require: organizational context, risk assessment, team capability evaluation, regulatory nuance – all uniquely human judgment.

Architecture Decision Records (ADR) — The Practice

```
# ADR-042: Use Kafka for Payment Event Distribution

## Status: Accepted

## Context
Payment events need to be consumed by: Notifications, Audit, Analytics, Fraud Detection
Current: direct service calls creating tight coupling and cascading failures.

## Decision
Use Kafka as the event backbone for payment domain events.

## Rationale
- Decouples producers from consumers (services can be deployed independently)
- Events are durable – consumers can replay from any offset
- Consumer groups allow each service to process at their own pace
- Supports adding new consumers without modifying payment service

## Alternatives Considered
- Redis Pub/Sub: No persistence, no replay, unsuitable for audit requirements
- Direct HTTP calls: Tight coupling, cascading failures, no fan-out
- RabbitMQ: Less suited for replay/event sourcing patterns

## Consequences
- Introduces operational complexity (Kafka cluster management)
- Eventual consistency between payment write model and consumers
- Requires idempotent consumers
- Audit requirement met: events persist for 7 years per compliance

## Reviewed By: [Names]
## Date: 2024-01-15
```

12.4 Product Thinking — What Separates L5 from L6

```
L5 (Senior) thinks: "How do I implement this feature correctly?"
L6 (Staff) thinks: "Should we build this feature at all?
                  What problem does it solve?
                  Are there simpler alternatives?"
```

Product thinking for engineers:

1. Understand the "why" before the "what"
2. Quantify the impact: who uses it, how often, what happens without it
3. Consider the cost: not just build time, but ongoing maintenance
4. Think about second-order effects: what does this enable/disable in the future?
5. Propose alternatives: "We could also solve this by..."

12.5 Engineering Judgment Examples

When to Break the "Rules"

Rule: "Always use @Transactional for DB writes"

When to break it: High-throughput logging service where DB failure should not roll back the business operation

Rule: "Use async/event-driven for all inter-service communication"

When to break it: Synchronous call when: you need the response immediately, you can't proceed without the answer, latency is acceptable

Rule: "Don't put business logic in controllers"

When to break it: Simple validation that doesn't belong in the domain layer and would require artificial service method creation

Rule: "Use BigDecimal for all monetary calculations"

When to break it: Analytics/reporting where approximate values are acceptable and performance matters (float/double is 10x faster)

The principle: Rules encode best practices for common cases.

Elite engineers understand WHY the rule exists, which allows them to know when the reason doesn't apply.

12.6 Future-Proof Skills

What Will Still Matter in 5 Years

Timeless engineering skills (AI amplifies, not replaces):

1. First-principles reasoning
 - "Why is this distributed system behaving unexpectedly?"
 - Requires: fundamental CS knowledge + system intuition
2. Trade-off articulation
 - "We're trading consistency for availability here because..."
 - Requires: domain knowledge + architectural experience
3. System behavior under failure
 - "What happens when Redis goes down? When the DB is slow?"
 - Requires: distributed systems knowledge + operational experience
4. Code review judgment
 - "This will cause N+1 queries at scale"
 - "This concurrent code has a race condition under load"
 - Requires: deep language + database + architecture knowledge
5. Incident response
 - Reading metrics, interpreting thread dumps, correlating logs
 - Requires: practical experience + deep system understanding
6. Organizational navigation
 - Knowing when to build vs buy vs reuse
 - Building technical consensus
 - Mentoring others

Learning Strategy for AI Era

Focus MORE on:

- Fundamentals (OS, networks, databases, concurrency)
- Architecture and system design
- Debugging methodologies
- Code reading and critical evaluation
- Domain expertise (banking, payments, etc.)

Focus LESS on:

- Memorizing API signatures (AI knows them)
- Boilerplate code patterns (AI generates them)
- Syntax details of unfamiliar languages (AI translates)
- Tutorial-level framework knowledge (AI explains it)

The learning goal has shifted:

From: "I can write code to do X"

To: "I understand deeply why X works, when it breaks,
and what alternatives exist"

Section Summary: AI Era Engineering Principles

1. **Be the architect, not the typist.** Use AI to generate code within architectural constraints you've defined.
2. **Your debugging intuition is irreplaceable.** No AI understands your system's specific behavior in production.
3. **Trade-off judgment requires context AI doesn't have.** Organizational, regulatory, and team factors are yours alone.
4. **Code review is more important than ever.** AI-generated code needs rigorous review — you are the quality gate.
5. **Invest in fundamentals.** The deeper your foundation (OS, networking, databases, concurrency), the more effectively you can use AI as a tool.
6. **Build a point of view.** Staff engineers are valued for architectural opinions and direction, not just implementation speed.
7. **Learn the "why" behind every pattern.** If you understand why DI exists, you can evaluate whether a non-Spring alternative serves better for a given context.

Section 13: Real Enterprise Engineering

Day 1 at HSBC or Amazon: You join, get access to a 500k-line Java codebase, attend standup, and are expected to contribute a bug fix by week 2. This section prepares you for the real experience — navigating legacy code, production incidents, PR culture, and team collaboration.

13.1 Navigating Enterprise Codebases

How to Onboard to a New Java Codebase

Week 1: Orientation

1. Find the entry points:

- `main()` → `@SpringBootApplication` → understand the bootstrap
- List all `@Controller/@RestController` classes → understand the API surface
- List all `@KafkaListener` classes → understand event consumers
- List all `@Scheduled` methods → understand background jobs

2. Understand the data model:

- Find all `@Entity` classes → this is the domain model
- Examine the Flyway/Liquibase migrations → understand schema evolution history
- Find the main `@Repository` interfaces → understand data access patterns

3. Understand the configuration:

- `application.yml` → core config
- `application-prod.yml` → production overrides
- `@ConfigurationProperties` classes → strongly-typed config beans

4. Find the tests:

- Unit tests → understand service logic
- Integration tests → understand component interactions
- `@SpringBootTest` → understand full-stack behavior

Week 2: First Contribution

- Take a small bug fix or well-defined feature
- Follow existing patterns exactly
- Write tests like the existing tests
- Match the code style (checkstyle/spotless rules)
- Ask questions in PR comments, not in Slack (creates knowledge record)

Reading Unfamiliar Java Code

```
// 1. Read signatures before bodies
public Optional<TransferResult> processInternationalTransfer(
    TransferRequest request,
    SwiftRoutingInfo routing,
    ComplianceContext compliance) throws ComplianceViolationException {
// Tells you: optional return (may fail/not apply), 3 inputs, checked exception
// → Already understand the contract before reading implementation

// 2. Identify the layers
// Controller (HTTP) → Service (business) → Repository (data) → Entity (model)
// Follow the call chain from the HTTP endpoint down to understand one feature fully

// 3. Find the tests first for unfamiliar code
// Tests document expected behavior more clearly than production code

// 4. Use the git log (your best documentation)
git log --follow -p src/main/java/com/bank/payment/PaymentService.java
// Shows WHY each change was made (if commit messages are good)
// Find the ticket/PR that introduced a pattern you don't understand

// 5. IDE structure navigation
// - Find Usages (Cmd+Click on interface → find implementations)
```

```
// - Call Hierarchy (who calls this method?)  
// - Type Hierarchy (what implements this interface?)
```

13.2 Code Review — Giving and Receiving

What Great PR Reviews Look For

```
Security:  
- SQL injection via string concat in queries?  
- Missing input validation (@Valid on request bodies)  
- Sensitive data logged?  
- Missing authorization check (can user X access resource Y?)  
- New library with known CVEs?  
  
Correctness:  
- Race conditions in concurrent code?  
- NullPointerException risk? (unchecked Optional.get())  
- Integer overflow? (int for count that could exceed 2B)  
- Transaction boundaries correct? (@Transactional on right method)  
- Exception handling appropriate? (silently caught?)  
  
Performance:  
- N+1 query? (lazy-loaded collection in loop)  
- Missing index for new query?  
- Unbounded collection? (could grow to millions)  
- Expensive operation in tight loop?  
- Cache miss patterns?  
  
Maintainability:  
- Is this testable? (can it be unit tested without Spring context?)  
- Does it follow existing patterns in the codebase?  
- Are names clear and accurate?  
- Is it unnecessarily complex?  
- Will the next engineer understand this in 6 months?  
  
Architecture:  
- Does this belong in this layer?  
- Does this create unwanted coupling?  
- Will this scale to 10x the current load?  
- Does this violate domain boundaries?
```

Giving Feedback Effectively

```
Nit: Minor style suggestion (not blocking)  
"Nit: Consider extracting this into a named constant for readability"  
  
Question: Seeking understanding  
"Question: Why are we using pessimistic locking here vs optimistic?  
Want to understand the trade-off being made."  
  
Concern: Potential issue (blocking if unaddressed)  
"Concern: This HashMap in a @Service singleton is not thread-safe.  
Concurrent requests could corrupt the state. Use ConcurrentHashMap."  
  
Blocker: Must fix (defect, security issue, violation)  
"Blocker: This query appends user input directly to SQL string –  
SQL injection vulnerability. Use parameterized query."  
  
Praise: Acknowledge good patterns  
"Nice use of the builder pattern here – much cleaner than the factory  
approach in the old code."
```

13.3 Production Incidents — The Real Test

Incident Response Framework

1. DETECT (0-5 minutes)
 - Alert fires (PagerDuty, Datadog)
 - Check dashboards: error rate, latency, traffic
 - Is it gradual degradation or sudden failure?
 - Is it all users or subset? All regions or one?
2. COMMUNICATE (immediately)
 - Post in incident Slack channel: "Investigating elevated error rate on payment API"
 - Tag relevant teams
 - Start incident timer
3. DIAGNOSE (5-20 minutes)
 - Recent deployments? (git log, deployment history)
 - Infrastructure changes? (Terraform logs, k8s events)
 - Check logs: filter by error level + service name
 - Check traces: find failing requests, look at span details
 - Thread dump if CPU spike or thread pool exhaustion
4. MITIGATE (varies)
 - Can you roll back the last deployment?
 - Can you toggle a feature flag to disable new behavior?
 - Can you increase a timeout/pool size?
 - Can you redirect traffic to healthy region?
 - Mitigation BEFORE root cause – stop the bleeding first
5. RESOLVE + COMMUNICATE
 - Confirm metrics return to normal
 - Post resolution in incident channel
 - Keep stakeholders updated
6. POST-MORTEM (within 48 hours)
 - Write blameless RCA
 - Timeline of events
 - Root cause (technical)
 - Contributing factors (process)
 - Action items with owners and deadlines

Common Production Issues — Java Specific

Issue: Memory leak (heap grows continuously)
Symptoms: GC time increasing, OOM eventually
Diagnosis:
 jmap -histo:live <pid> → object histogram (what's growing?)
 Heap dump + MAT analysis → find GC roots holding objects
Common causes: ThreadLocal not cleared, static Map growing, listener not removed

Issue: Thread pool exhaustion
Symptoms: RejectedExecutionException, slow response (threads queueing)
Diagnosis:
 jstack <pid> → thread dump, look for many threads in same state
 Actuator /actuator/metrics/executor.pool.size
 HikariCP metrics → connection pool stats
Common causes: DB slow (connections held), pool too small, processing too slow

Issue: GC pressure causing latency spikes
Symptoms: P99/P999 latency spikes every ~60 seconds
Diagnosis:
 GC logs: -Xlog:gc*:file=/var/log/gc.log
 JFR: java -XX:StartFlightRecording duration=120s
 Grafana: gc.pause metric spikes
Common causes: Heap too small, long-lived objects in old gen, allocation rate too high

Issue: Connection leak
Symptoms: HikariCP connection timeout, "Unable to acquire JDBC Connection"
Diagnosis:
 spring.datasource.hikari.leak-detection-threshold=2000


```
Check for try-with-resources everywhere DB connections used
Common causes: Exception path skips connection.close()
```

13.4 Refactoring Legacy Code

Safe Refactoring Strategy

```
The Strangler Fig Pattern – migrate legacy incrementally

Legacy Monolith → New Microservice (incrementally)

Step 1: Add feature flag
if (featureFlags.isEnabled("new-payment-service", userId)) {
    return newPaymentService.process(request);
} else {
    return legacyPaymentService.process(request);
}

Step 2: Route small % to new service (canary)
5% → new service → validate behavior matches legacy

Step 3: Shadow mode – run both, compare results
newService.process(request);
legacyResult = legacyService.process(request);
if (!resultsMatch(newResult, legacyResult)) {
    alert.warning("Divergence detected: " + requestId);
}
return legacyResult; // Still serving legacy until confident

Step 4: Gradually increase % to new service
5% → 25% → 50% → 100%

Step 5: Remove legacy code (satisfying!)
```

Dealing with Code You Inherited

```
// Don't touch it without tests.
// First, write characterization tests – tests that document WHAT THE CODE DOES.

@Test
public void testLegacyCalculation_documentsBehavior() {
    // You don't know if this is "right" – you document what it does
    LegacyCalculator calc = new LegacyCalculator();
    BigDecimal result = calc.computeFee(new BigDecimal("100.00"), "USD");
    // Whatever this returns IS the expected behavior until someone decides to change it
    assertEquals(new BigDecimal("2.50"), result);
}

// Now you have a safety net. THEN refactor.
// After refactoring: all tests still pass = behavior preserved.
```

13.5 Coding Standards — What Matters at Enterprise Companies

Style and Standards

```
// Checkstyle / SpotBugs / PMD / SonarQube – automated enforcement

// Google Java Style Guide (most companies follow this or similar):
// - 2 or 4 spaces indentation (never tabs)
// - 100-120 character line limit
// - K&R brace style (opening brace on same line)
// - Blank line between class members
// - @Override always present when overriding
```

```
// Lombok usage (reduces boilerplate, common in enterprises)
@Getter
@Builder
@ToString(exclude = {"password", "ssn"})
@EqualsAndHashCode(onlyExplicitlyIncluded = true)
@Entity
public class User {
    @EqualsAndHashCode.Include
    private UUID id;

    private String email;
    private String password;
    @ToString.Exclude // Don't log PII
    private String ssn;
}

// Naming conventions:
// Classes: PascalCase (PaymentService, not paymentService)
// Methods/variables: camelCase (createPayment, paymentId)
// Constants: SCREAMING_SNAKE_CASE (MAX_RETRY_COUNT)
// Packages: lowercase, reverse domain (com.hsbc.payment.service)
```

13.6 Agile Execution — What Senior Engineers Do Differently

Sprint-Level Engineering Excellence

```
Story sizing:
- Senior engineers can identify hidden complexity ("this looks like 3 points
  but the auth integration alone is 8 – let me explain why")
- Know what to spike vs estimate
- Know when to pull in an architect

PR strategy:
- Break large features into small, reviewable PRs
- Each PR should be independently mergeable (not depend on next PR)
- Feature flags for large features deployed incrementally

Estimation honesty:
- "I don't know, I need a day to spike" is the right answer
- Padding for code review, testing, documentation is legitimate
- Always ask: "is there an existing library or pattern for this?"

Technical debt management:
- Log debt immediately (tech debt ticket in backlog)
- Quantify the cost ("this N+1 query adds 200ms to every page load for 100k users")
- Advocate for scheduled debt payment ("we need 1 sprint per quarter for cleanup")
```

Section Summary: Real Enterprise Engineering Mindset

What distinguishes enterprise engineers:

1. **Code review culture:** Every change reviewed. Reviews are about knowledge sharing, not gatekeeping.
2. **Documentation as first-class:** ADRs, runbooks, post-mortems are engineering artifacts.
3. **Testing discipline:** Unit tests, integration tests, contract tests — not optional.
4. **Observability by default:** Every service has health checks, metrics, distributed tracing from day one.
5. **Production ownership:** If you deployed it, you're responsible for its behavior in production.
6. **Blameless culture:** Post-mortems identify systemic issues, not scapegoats. "How did the system allow this mistake?" not "who made this mistake?"

7. **Gradual rollout:** Feature flags, canary deployments, blue-green — never ship directly to 100% of users.
8. **Backward compatibility:** APIs are contracts. Breaking changes need versioning and migration paths.

Section 14: Node.js → Java Complete Mapping

Your Bridge: This is the most important section for your transition. Every concept you understand in Node.js maps to something in Java. Use this as your primary reference for the first 3 months — find the Node.js concept you know, learn the Java equivalent, understand the differences.

14.1 Runtime & Execution Model

Dimension	Node.js	Java
Runtime	V8 Engine	JVM (HotSpot/GraalVM)
Compilation	JIT at runtime	Bytecode → JIT (tiered C1/C2)
Concurrency model	Single thread + event loop	Multi-threaded (OS threads)
I/O model	Non-blocking (libuv)	Blocking (default) or async (NIO/WebFlux)
Memory management	V8 GC	Pluggable GC (G1, ZGC, etc.)
Warm-up	Fast	JIT warm-up (30-60s)
Startup time	~100-500ms	3-8s (JVM) / 50-200ms (Native)
Memory footprint	50-150MB	200-500MB (JVM overhead)

Concurrency Deep Comparison

```
// Node.js: single thread, non-blocking I/O
// 1000 requests handled concurrently by one thread via event loop
app.get('/payments', async (req, res) => {
  const payments = await db.find({}); // Non-blocking – event loop continues
  res.json(payments);
});
```

```
// Java: thread per request (Tomcat default: 200 threads)
// Each request blocks its thread while waiting for DB
@GetMapping("/payments")
public List<Payment> getPayments() {
  return paymentRepo.findAll(); // Blocks thread for DB duration
}

// Java WebFlux: non-blocking like Node.js
@GetMapping("/payments")
public Flux<Payment> getPayments() {
  return paymentRepo.findAll(); // Reactive – doesn't block thread
}
```

Implication: In Java blocking model, 200 concurrent requests = 200 threads. In Node.js, 200 concurrent requests = 1 thread handling all I/O asynchronously.

14.2 Package Management & Build

Dimension	Node.js	Java
Package registry	npm / yarn registry	Maven Central / JFrog Artifactory
Build file	package.json	pom.xml (Maven) / build.gradle (Gradle)

Dimension	Node.js	Java
Build tool	npm, yarn, pnpm	Maven, Gradle
Lock file	package-lock.json	pom.xml is deterministic (with versions)
Script runner	npm scripts	Maven plugins / Gradle tasks
Local packages	node_modules/	~/.m2/repository (global local cache)

```
// package.json (Node.js)
{
  "name": "payment-service",
  "version": "1.0.0",
  "dependencies": {
    "express": "^4.18.0",
    "mongoose": "^7.0.0"
  },
  "scripts": {
    "start": "node index.js",
    "test": "jest"
  }
}
```

```
<!-- pom.xml (Java/Maven) -->
<project>
  <groupId>com.bank</groupId>
  <artifactId>payment-service</artifactId>
  <version>1.0.0</version>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.5</version>
  </parent>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
  </dependencies>
</project>
```

14.3 Web Framework Mapping

Dimension	Node.js	Java
Framework	Express/Fastify/Koa	Spring MVC/WebFlux
Router	express.Router()	@RequestMapping, @GetMapping
Middleware	app.use(middleware)	Filter, Interceptor, AOP
Request object	req	HttpServletRequest
Response object	res	HttpServletResponse
Body parser	body-parser	@RequestBody (auto)
Path variables	req.params.id	@PathVariable
Query params	req.query.page	@RequestParam
Headers	req.headers	@RequestHeader

Dimension	Node.js	Java
Error handling	app.use(errorMiddleware)	@ControllerAdvice

```
// Express.js
const app = express();
app.use(authMiddleware);
app.use('/api/payments', paymentRouter);

const paymentRouter = express.Router();
paymentRouter.post('/', async (req, res) => {
  const { fromAccount, toAccount, amount } = req.body;
  try {
    const result = await paymentService.transfer(fromAccount, toAccount, amount);
    res.status(201).json(result);
  } catch (err) {
    next(err);
  }
});
```

```
// Spring Boot
@RestController
@RequestMapping("/api/payments")
public class PaymentController {

  @PostMapping
  @ResponseStatus(HttpStatus.CREATED)
  public PaymentResponse createPayment(@Valid @RequestBody PaymentRequest request) {
    return paymentService.transfer(request);
  }
}
// Auth handled by Spring Security filter (equivalent to Express middleware)
// Error handling in @ControllerAdvice (equivalent to Express error middleware)
```

14.4 Database & ORM

Dimension	Node.js	Java
ORM	Mongoose, TypeORM, Sequelize	Hibernate, Spring Data JPA
Query builder	Knex, Drizzle	QueryDSL, JPA Criteria API
Connection pool	pg-pool, mongoose pool	HikariCP
Schema	Mongoose schema, TypeORM entity	@Entity, JPA annotations
Migrations	Knex migrate, TypeORM migrate	Flyway, Liquibase
Raw SQL	pg, mysql2	JdbcTemplate, JOOQ

```
// TypeORM (Node.js)
@Entity()
export class Payment {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ type: 'decimal', precision: 19, scale: 4 })
  amount: number;

  @Column()
  @CreateDateColumn()
  createdAt: Date;
}

// Repository
const payments = await paymentRepo.find({
  where: { status: 'PENDING', createdAt: MoreThan(cutoffDate) },
```

```

    order: { createdAt: 'DESC' },
    take: 20
  });

```

```

// Spring Data JPA (Java)
@Entity
@Table(name = "payments")
public class Payment {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @Column(nullable = false, precision = 19, scale = 4)
    private BigDecimal amount;

    @CreatedDate
    @Column(nullable = false, updatable = false)
    private Instant createdAt;
}

// Repository
List<Payment> payments = paymentRepo
    .findByStatusAndCreatedAtAfter(PaymentStatus.PENDING, cutoffDate);
// OR with Pageable:
Page<Payment> page = paymentRepo.findByStatus(status,
    PageRequest.of(0, 20, Sort.by("createdAt").descending()));

```

14.5 Async & Promises → CompletableFuture

```

// Node.js: async/await
async function processPayment(payment) {
    const account = await accountService.findById(payment.fromAccount);
    const fxRate = await fxService.getRate(payment.currency);
    const result = await ledger.record(payment, fxRate);
    return result;
}

// Parallel:
const [account, fxRate] = await Promise.all([
    accountService.findById(payment.fromAccount),
    fxService.getRate(payment.currency)
]);

// Error handling:
const result = await someAsyncOperation()
    .catch(err => handleError(err));

```

```

// Java: CompletableFuture
public CompletableFuture<PaymentResult> processPayment(Payment payment) {
    return accountService.findById(payment.getFromAccount())
        .thenCombine(
            fxService.getRate(payment.getCurrency()),
            (account, fxRate) -> new ProcessingContext(account, fxRate)
        )
        .thenCompose(ctx -> ledger.record(payment, ctx.getFxRate()))
        .exceptionally(ex -> handleError(ex));
}

// Parallel:
CompletableFuture<Account> accountFuture = accountService.findById(fromId);
CompletableFuture<FxRate> rateFuture = fxService.getRate(currency);
CompletableFuture.allOf(accountFuture, rateFuture).join();
Account account = accountFuture.join();
FxRate rate = rateFuture.join();

```

14.6 Environment Config & Secrets

Dimension	Node.js	Java
Env vars	process.env.DB_URL	\${DB_URL} in application.yml
Config file	.env (dotenv)	application.yml / application.properties
Typed config	zod schema, joi	@ConfigurationProperties
Multiple environments	.env.production	application-prod.yml
Secrets	.env.local, AWS SSM	AWS Secrets Manager, Vault, k8s secrets

```
// Node.js .env
DB_URL=postgres://localhost:5432/mydb
JWT_SECRET=supersecret
PORT=3000

// Access:
const dbUrl = process.env.DB_URL;
```

```
# application.yml (Java)
spring:
  datasource:
    url: ${DB_URL:jdbc:postgresql://localhost:5432/mydb}
    password: ${DB_PASSWORD}
  server:
    port: ${PORT:8080}

# Typed config:
# @ConfigurationProperties(prefix = "payment")
# → payment.timeout-ms=5000
```

14.7 Testing

Dimension	Node.js	Java
Unit testing	Jest, Mocha	JUnit 5, TestNG
Mocking	Jest mocks, Sinon	Mockito, MockMvc
Assertions	Jest expect, Chai	AssertJ (fluent)
Integration testing	Supertest	MockMvc, TestRestTemplate, Testcontainers
Test containers	testcontainers-node	Testcontainers
Code coverage	Istanbul/NYC	JaCoCo

```
// Jest unit test
describe('PaymentService', () => {
  let service;
  let mockRepo;

  beforeEach(() => {
    mockRepo = { findById: jest.fn(), save: jest.fn() };
    service = new PaymentService(mockRepo);
  });

  it('should throw when account not found', async () => {
    mockRepo.findById.mockResolvedValue(null);
    await expect(service.transfer('ACC-1', 'ACC-2', 100))
      .rejects.toThrow('Account not found');
```



```
});  
});
```

```
// JUnit 5 + Mockito  
@ExtendWith(MockitoExtension.class)  
class PaymentServiceTest {  
  
    @Mock  
    private AccountRepository accountRepo;  
  
    @InjectMocks  
    private PaymentService paymentService;  
  
    @Test  
    void shouldThrowWhenAccountNotFound() {  
        when(accountRepo.findById(any())).thenReturn(Optional.empty());  
  
        assertThatThrownBy(() ->  
            paymentService.transfer("ACC-1", "ACC-2", new BigDecimal("100")))  
            .isInstanceOf(AccountNotFoundException.class)  
            .hasMessageContaining("Account not found");  
    }  
}  
  
// Integration test with Testcontainers  
@SpringBootTest  
@Testcontainers  
class PaymentIntegrationTest {  
  
    @Container  
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:15")  
        .withDatabaseName("testdb");  
  
    @DynamicPropertySource  
    static void configureProperties(DynamicPropertyRegistry registry) {  
        registry.add("spring.datasource.url", postgres::getJdbcUrl);  
    }  
  
    @Autowired  
    private PaymentService paymentService;  
  
    @Test  
    void shouldPersistPayment() {  
        PaymentResponse response = paymentService.createPayment(testRequest);  
        assertThat(response.getPaymentId()).isNotNull();  
    }  
}
```

14.8 Error Handling

```
// Node.js: Error subclassing  
class PaymentError extends Error {  
    constructor(message, code, statusCode) {  
        super(message);  
        this.name = 'PaymentError';  
        this.code = code;  
        this.statusCode = statusCode;  
    }  
}  
  
// Express error middleware  
app.use((err, req, res, next) => {  
    if (err instanceof PaymentError) {  
        res.status(err.statusCode).json({ code: err.code, message: err.message });  
    } else {  
        res.status(500).json({ error: 'Internal server error' });  
    }  
});
```

```
// Java: Exception hierarchy
public class PaymentException extends RuntimeException {
    private final ErrorCode code;
    public PaymentException(ErrorCode code, String message) {
        super(message);
        this.code = code;
    }
}

// Spring @ControllerAdvice (equivalent to Express error middleware)
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(PaymentException.class)
    public ResponseEntity<ErrorResponse> handlePayment(PaymentException ex) {
        return ResponseEntity.status(ex.getCode().httpStatus())
            .body(new ErrorResponse(ex.getCode().name(), ex.getMessage()));
    }
}
```

14.9 Logging

```
// Node.js: Winston/Pino
const logger = winston.createLogger({
    format: winston.format.json(),
    transports: [new winston.transports.Console()]
});
logger.info('Payment processed', { paymentId, amount, userId });
```

```
// Java: SLF4J + Logback
@Slf4j // Lombok annotation – creates log field
public class PaymentService {
    public void process(Payment payment) {
        log.info("Payment processed: paymentId={} amount={} userId={}",
            payment.getId(), payment.getAmount(), payment.getUserId());
        // MDC.put("traceId", ...) for correlation
    }
}
```

14.10 Streaming / Events

Dimension	Node.js	Java
Event emitter	EventEmitter	ApplicationEventPublisher
Streams	Node.js Streams	Java NIO, Reactive Streams
Reactive	RxJS	Project Reactor (Flux/Mono)
Message queue	BullMQ, bee-queue	Spring AMQP, Spring Kafka
Server-sent events	SSE module	Spring WebFlux SSE
WebSocket	ws, socket.io	Spring WebSocket (STOMP)

14.11 Performance Characteristics Comparison

Scenario	Node.js	Java
I/O-bound (DB, HTTP)	Excellent	Excellent (blocking OK, or WebFlux)
CPU-bound	Single thread → bottleneck	Multiple threads → scales

Scenario	Node.js	Java
Memory per connection	Very low (~KB)	Higher (thread stack = 256KB-1MB each)
Throughput (HTTP)	High (event loop)	High (thread pool, Virtual Threads)
Latency consistency	Very consistent	More variance (GC pauses)
Startup time	Sub-second	3-8s (JVM), ms (Native)
Long-running tasks	Worker threads needed	Natural (just a thread)

14.12 When Java Wins vs When Node.js Wins

Java Advantages

- ✓ CPU-intensive computation (thread parallelism)
- ✓ Complex type systems (catches bugs at compile time)
- ✓ Large team codebases (refactoring safety with types)
- ✓ Ecosystem maturity (Spring, Hibernate, thousands of enterprise libs)
- ✓ JVM performance at scale (JIT optimization over time)
- ✓ Regulated industries (compliance tooling, audit libraries)
- ✓ Long-running processes (JVM overhead amortized)
- ✓ Thread-based parallelism without worker complexity

Node.js Advantages

- ✓ Rapid prototyping (no compilation, fast iteration)
- ✓ I/O-bound microservices (event loop efficiency)
- ✓ Real-time (WebSocket, SSE – natural async)
- ✓ Serverless/edge functions (fast cold start)
- ✓ Full-stack JavaScript teams (same language frontend/backend)
- ✓ Small payload transformations (API gateways, BFF)
- ✓ npm ecosystem for utilities
- ✓ Lower memory for I/O-bound services

Section Summary

The key insight from this comparison: **you already understand the concepts**. DI is DI whether it's done by `nest-di` or Spring. Kafka consumers are Kafka consumers whether you write them in JavaScript or Java. Transactions are transactions.

Your transition challenge is not conceptual — it's about: 1. Learning Java's type system and how to work with it efficiently 2. Understanding Spring's idioms and conventions 3. Learning JVM behavior and tuning 4. Internalizing Java's concurrency model (biggest mindset shift)

Everything else maps directly.

Section 15: Practical Learning — Projects, Architecture & Anti-Patterns

Build to Understand: Reading is passive. Building is active. This section gives you a curated set of projects that, when built, will cement every concept in this handbook. Each project is sized for 2-8 weeks and mirrors real enterprise work.

15.1 Learning Project Portfolio

Project 1: Banking API (Weeks 1-4) — Foundation

Purpose: Learn Spring Boot, JPA, REST API design, security, migrations

Stack: Spring Boot, PostgreSQL, Spring Security (JWT), Flyway, JUnit 5

```
bank-api/
├── src/main/java/com/bank/
│   ├── BankApplication.java
│   ├── config/
│   │   ├── SecurityConfig.java
│   │   └── SwaggerConfig.java
│   ├── account/
│   │   ├── Account.java           (Entity)
│   │   ├── AccountRepository.java
│   │   ├── AccountService.java
│   │   └── AccountController.java
│   ├── payment/
│   │   ├── Payment.java
│   │   ├── PaymentRepository.java
│   │   ├── PaymentService.java
│   │   └── PaymentController.java
│   ├── auth/
│   │   ├── AuthService.java
│   │   └── JwtService.java
│   └── shared/
│       ├── exception/
│       ├── dto/
│       └── config/
├── src/main/resources/
│   ├── application.yml
│   ├── application-test.yml
│   └── db/migration/
│       ├── V1__create_accounts.sql
│       ├── V2__create_payments.sql
│       └── V3__add_indexes.sql
└── src/test/
    ├── unit/                (Mockito unit tests)
    └── integration/         (Testcontainers + @SpringBootTest)
```

What you learn: - Spring Boot project structure - JPA entity design, relationships, migrations - Spring Security JWT authentication - REST API design: pagination, sorting, filtering - Unit testing with Mockito, integration testing with Testcontainers - BigDecimal for monetary values, transaction management

Project 2: Event-Driven Payment Platform (Weeks 5-8) — Intermediate

Purpose: Learn Kafka, microservices, circuit breakers, async processing

Stack: Spring Boot × 3 services, Kafka, Redis, PostgreSQL, Resilience4j

```

payment-platform/
├─ payment-service/      ← Initiates payments
├─ notification-service/ ← Sends emails/SMS
├─ fraud-detection-service/ ← Kafka consumer, ML scoring
├─ docker-compose.yml    ← Local environment
└─ k8s/                  ← Kubernetes manifests

```

What you learn: - Multi-service communication via Kafka - Saga pattern (payment saga across 3 services) - Idempotency (deduplication across services) - Circuit breaker + retry (Resilience4j) - Redis for rate limiting and caching - Docker Compose for local development - Kubernetes deployment basics

Kafka topics to implement:

```

payment.initiated    → Fraud service evaluates
payment.approved     → Payment service debits
payment.completed    → Notification service sends
payment.failed       → Saga compensates
payment.retry.queue   → Retry failed events
payment.DLT          → Dead letter topic

```

Project 3: CQRS Event Store (Weeks 9-12) — Advanced

Purpose: Learn event sourcing, CQRS, projections, temporal queries

Stack: Spring Boot, PostgreSQL (event store), Elasticsearch (read model), Kafka

```

// Account aggregate with full event sourcing
public class Account {
    private UUID id;
    private BigDecimal balance;
    private AccountStatus status;
    private long version;

    // Events: AccountOpened, MoneyDeposited, MoneyWithdrawn,
    //         AccountFrozen, AccountClosed

    // Commands: OpenAccount, DepositMoney, WithdrawMoney
    // Projections: AccountBalanceView, AccountHistoryView
}

```

What you learn: - Event sourcing from scratch (no framework) - CQRS: separate command and query models - Elasticsearch for read projections - Temporal queries: "what was the balance on Jan 15?" - Snapshot optimization for long event streams - Event schema versioning

Project 4: Production-Ready Microservice Template (Ongoing)

Purpose: Reference implementation of enterprise patterns

Stack: Spring Boot, comprehensive observability

```

// This is your "golden template" – production-ready from day one

payment-service-template/
├─ src/main/java/com/template/
│  └─ config/
│     ├── SecurityConfig.java      (JWT + OAuth2)
│     ├── CacheConfig.java        (Redis + Caffeine)
│     ├── KafkaConfig.java
│     ├── DataSourceConfig.java    (HikariCP tuning)
│     ├── ActuatorConfig.java      (health, metrics)
│     └─ AsyncConfig.java          (thread pool)
│  └─ common/
│     ├── exception/              (exception hierarchy)
│     ├── logging/                (MDC filter, audit aspect)
│     └─ metrics/                 (custom Micrometer metrics)

```

```

├── resilience/           (CB, retry, rate limit)
├── validation/          (custom validators)
├── domain/
│   ├── payment/
│   │   ├── api/         (controller, DTOs)
│   │   ├── domain/      (entities, events)
│   │   ├── service/     (business logic)
│   │   └── infrastructure/ (repo, kafka, external clients)
├── src/test/
│   ├── architecture/    (ArchUnit architecture tests)
│   ├── unit/
│   ├── integration/     (Testcontainers)
│   └── contract/        (Pact consumer-driven tests)
├── k8s/                 (complete k8s manifests)
├── helm/                 (Helm chart)
├── .github/workflows/   (CI/CD pipelines)
└── docker-compose.yml

```

15.2 Production-Ready Project Structure

```

src/
├── main/
│   ├── java/com/company/service/
│   │   ├── Application.java      ← @SpringBootApplication entry
│   │   ├── api/                 ← HTTP layer
│   │   │   ├── controller/      ← @RestController
│   │   │   ├── dto/             ← Request/Response records
│   │   │   ├── filter/          ← Servlet filters (logging, rate limit)
│   │   │   └── interceptor/     ← Spring interceptors
│   │   ├── domain/              ← Business logic (pure Java, no Spring)
│   │   │   ├── model/           ← Domain entities, value objects
│   │   │   ├── service/        ← Domain services
│   │   │   ├── event/          ← Domain events
│   │   │   ├── repository/     ← Repository interfaces (ports)
│   │   │   └── exception/      ← Domain exceptions
│   │   ├── infrastructure/      ← Adapters for external systems
│   │   │   ├── persistence/     ← JPA entities, repos (adapters)
│   │   │   │   ├── entity/
│   │   │   │   └── repository/
│   │   │   ├── kafka/          ← Producers, consumers
│   │   │   ├── redis/          ← Cache, rate limiter, locks
│   │   │   └── client/         ← External HTTP clients
│   │   └── config/              ← Spring configuration
│   │       ├── SecurityConfig.java
│   │       ├── KafkaConfig.java
│   │       └── CacheConfig.java
│   ├── resources/
│   │   ├── application.yml
│   │   ├── application-local.yml
│   │   ├── application-prod.yml
│   │   └── db/migration/
├── test/
│   ├── java/com/company/service/
│   │   ├── architecture/        ← ArchUnit layer tests
│   │   ├── domain/              ← Pure unit tests (no Spring)
│   │   ├── api/                 ← MockMvc tests
│   │   ├── integration/         ← @SpringBootTest + Testcontainers
│   │   └── contract/            ← Pact tests
│   ├── resources/
│   │   └── application-test.yml

```

15.3 Anti-Patterns — Learn to Recognize and Avoid

Spring Anti-Patterns

```
// 1. God Service – one service doing everything
@Service
public class PaymentService {
    // 5000 lines, handles: payment, refund, dispute, reporting,
    // notification, fraud check, ledger, audit...
    // Fix: Split by bounded context
}

// 2. Anemic Domain Model – entities with no behavior
@Entity
public class Order {
    private BigDecimal total;
    // Just getters/setters – no business logic
    // Fix: Move business logic INTO the entity
    public void addItem(OrderItem item) {
        // Validate, add, recalculate total here
    }
}

// 3. Constructor injection avoided in favor of field injection
@Service
public class BadService {
    @Autowired
    private SomeDependency dep; // Cannot mock without Spring context
    // Fix: Constructor injection
}

// 4. @Transactional on every method (lazy safety blanket)
@Service
@Transactional // On CLASS – wraps every method including read-only queries
public class ServiceWithUnnecessaryTransactions {
    // Fix: @Transactional(readOnly = true) on reads, @Transactional on writes
}

// 5. Catching Exception everywhere
try { ... }
catch (Exception e) {
    log.error("Error", e);
    // returns null or default – caller has no idea what happened
}
// Fix: let domain exceptions propagate, handle at boundary (@ControllerAdvice)
```

Database Anti-Patterns

```
// 1. N+1 queries
List<Order> orders = orderRepo.findAll();
for (Order order : orders) {
    // Each call fires a SQL query!
    String customerName = order.getCustomer().getName();
}
// Fix: JOIN FETCH or @EntityGraph

// 2. Loading entire table
List<User> allUsers = userRepo.findAll(); // 10 million users → OOM
// Fix: Pageable or streaming

// 3. hibernate.ddl-auto = create-drop in production
# application-prod.yml
spring.jpa.hibernate.ddl-auto: validate # ALWAYS validate in production

// 4. Missing transaction on multi-step operations
public void transfer(from, to, amount) {
    account1.debit(amount); // Committed
    // CRASH HERE → money lost!
    account2.credit(amount); // Never runs
}
```

```
// Fix: @Transactional on the whole method

// 5. Synchronous Hibernate load in loop (variant of N+1)
for (String id : accountIds) {
    Account a = accountRepo.findById(id).get(); // N queries!
}
// Fix: accountRepo.findAllById(accountIds) → single IN query
```

Concurrency Anti-Patterns

```
// 1. Unsynchronized shared state
@Service // Singleton – shared by all requests!
public class StatefulService {
    private Map<String, Object> requestState = new HashMap<>(); // NOT thread-safe
}
// Fix: ConcurrentHashMap, or better: don't store per-request state in singleton

// 2. Incorrect double-checked locking (pre-Java 5)
if (instance == null) {
    synchronized (this) {
        if (instance == null) {
            instance = new Expensive(); // Without volatile, broken!
        }
    }
}
// Fix: volatile instance field, or use enum singleton, or Spring singleton bean

// 3. Blocking in reactive pipeline
Mono.fromCallable(() -> blockingDatabaseCall()) // Blocks reactor thread!
// Fix: subscribeOn(Schedulers.boundedElastic()) or use non-blocking R2DBC

// 4. Thread pool starvation
// All threads waiting for another thread pool to respond → deadlock
@Async("poolA")
public void taskA() {
    poolBService.taskB().get(); // Waits for poolB
}
@Async("poolA") // SAME pool – poolB is also poolA, all threads waiting for each other
public void taskB() { }
// Fix: Use separate thread pools for dependent tasks
```

15.4 Debugging Exercises

Exercise 1: Find the N+1 Query

Given: User complains that the "transactions" page is slow
 Given: The following service code:

```
List<Account> accounts = accountRepo.findById(userId);
for (Account account : accounts) {
    List<Transaction> txns = account.getTransactions(); // ?
    txns.forEach(t -> dashboard.add(t.getSummary()));
}
```

Task:

1. Enable SQL logging: `spring.jpa.show-sql=true`
2. Count the queries executed
3. Fix the N+1 problem
4. Verify query count reduced to 1

Exercise 2: Diagnose Thread Pool Exhaustion

Given: API returns 503 intermittently under load
 Given: Log shows: "Unable to acquire JDBC Connection"

Task:

1. Write HikariCP metrics monitoring code
2. Simulate the problem with high concurrency test
3. Add leak detection: hikari.leak-detection-threshold=2000
4. Find the connection leak (a missing try-with-resources)
5. Fix it and verify the issue is resolved

Exercise 3: Fix a Race Condition

```
// Given: This service intermittently creates duplicate promotions
@Service
public class PromotionService {
    public Promotion createFirstTimePromotion(String userId) {
        if (promotionRepo.existsByUserId(userId)) {
            throw new DuplicatePromotionException();
        }
        // Concurrent requests BOTH pass the check above!
        return promotionRepo.save(new Promotion(userId));
    }
}

// Task:
// 1. Write a concurrent test that reproduces the duplicate creation
// 2. Fix using database unique constraint + exception handling
// 3. Alternative: fix using Redis distributed lock
// 4. Compare the trade-offs of both approaches
```

15.5 Key Libraries to Know

```
<!-- Spring Boot Starters (know these by heart) -->
spring-boot-starter-web           <!-- REST API (Tomcat) -->
spring-boot-starter-webflux       <!-- Reactive API (Netty) -->
spring-boot-starter-data-jpa      <!-- Hibernate + Spring Data -->
spring-boot-starter-security      <!-- Spring Security -->
spring-boot-starter-actuator       <!-- Health, metrics, info -->
spring-boot-starter-validation    <!-- Bean Validation (Jakarta) -->
spring-boot-starter-cache         <!-- Caching abstraction -->
spring-boot-starter-test          <!-- JUnit 5 + Mockito + AssertJ -->

<!-- Production essentials -->
spring-kafka                      <!-- Apache Kafka -->
spring-cloud-starter-circuitbreaker-resilience4j <!-- Circuit breaker -->
micrometer-tracing-bridge-otel    <!-- Distributed tracing -->
micrometer-registry-prometheus    <!-- Prometheus metrics -->
flyway-core                       <!-- DB migrations -->
lombok                           <!-- Reduce boilerplate -->

<!-- Testing -->
testcontainers-postgresql         <!-- Real DB in tests -->
testcontainers-kafka              <!-- Real Kafka in tests -->
rest-assured                     <!-- API integration tests -->
archunit                          <!-- Architecture tests -->

<!-- Utilities -->
jackson-databind                 <!-- JSON serialization -->
mapstruct                        <!-- DTO mapping -->
bucket4j-core                    <!-- Rate limiting -->
redisson                         <!-- Redis distributed locks -->
caffeine                         <!-- In-memory caching -->
```

15.6 6-Month Learning Schedule

Month 1: Java Language Mastery
Week 1-2: Section 1 (Fundamentals) + Section 14 (Node.js mapping)
Week 3-4: Section 2 (JVM) + Start Project 1 (Banking API)
Daily: 2 LeetCode Easy problems

```
Month 2: Spring Mastery
  Week 5-6: Section 3 (Spring ecosystem) – complete Project 1
  Week 7-8: Section 5 (Databases) – add advanced DB features to Project 1
  Daily: 2 LeetCode Medium problems

Month 3: Distributed Systems
  Week 9-10: Section 6 (Kafka, distributed systems) – Start Project 2
  Week 11-12: Section 7 (Concurrency) – complete Project 2
  Daily: 2 LeetCode Medium problems

Month 4: Production Engineering
  Week 13-14: Section 4 (Production patterns) + Section 8 (Cloud/DevOps)
  Week 15-16: Section 9 (Security) + Section 10 (System Design)
  Weekly: 1 system design study

Month 5: Interview Preparation
  Week 17-18: Section 11 (Interview prep) – Start mock interviews
  Week 19-20: Section 12-13 (AI era, enterprise) – Finalize project portfolio
  Daily: 3 LeetCode problems (mix easy/medium/hard)
  Weekly: 2 mock interviews

Month 6: Mastery Sprint
  Target companies research + tailored preparation
  Weekly: 3+ mock interviews
  Build: Project 3 or 4 based on target company focus
  Contributions: Open source Spring projects
```

Section Summary: Build Everything You Can

The fastest path to enterprise Java fluency is not reading — it's building. Each project forces you to confront the gaps between knowing a concept and actually implementing it.

The learning flywheel: 1. Read concept → understand abstractly 2. Build project → encounter real problems 3. Debug and fix → deeply understand behavior 4. Review code with experienced engineers → learn production standards 5. Teach concept to someone else → solidify understanding

Your Node.js advantage on projects: You'll build things that actually work quickly because the architectural patterns are familiar. Use that momentum to dive into Java-specific depth (JVM tuning, Spring AOP internals, JPA optimization) in your second pass.

Java Syntax Cheatsheet

Quick Reference for Node.js Engineers

Variables & Types

```
// Primitives (stack)
int count = 0;
long bigCount = 10_000_000_000L; // L suffix for long
double price = 9.99;
boolean active = true;
char letter = 'A';

// Reference types (heap)
String name = "Alice";
Integer boxed = 42; // Autoboxed int
BigDecimal money = new BigDecimal("100.00"); // ALWAYS for money

// Constants
static final int MAX_RETRIES = 3; // Convention: SCREAMING_SNAKE_CASE

// Type inference (Java 10+)
var list = new ArrayList<String>();
var map = new HashMap<String, Integer>();

// Null safety
String value = null;
Optional<String> safe = Optional.ofNullable(value);
String result = safe.orElse("default");
String orThrow = safe.orElseThrow(() -> new IllegalStateException("missing"));
```

Control Flow

```
// If-else
if (x > 0) {
    // positive
} else if (x < 0) {
    // negative
} else {
    // zero
}

// Switch expression (Java 14+)
String label = switch (status) {
    case PENDING -> "Waiting";
    case ACTIVE -> "Running";
    case COMPLETED -> "Done";
    default -> "Unknown";
};

// Pattern switch (Java 21)
String describe = switch (obj) {
    case Integer i -> "int: " + i;
    case String s -> "string: " + s;
    case null -> "null";
    default -> "other";
};

// Ternary
String type = amount > 1000 ? "large" : "small";

// For loops
for (int i = 0; i < 10; i++) { } // traditional
for (String item : collection) { } // enhanced for (like for...of)
```

```

collection.forEach(item -> process(item)); // functional

// While / Do-while
while (condition) { }
do { } while (condition);

// Try-catch-finally
try {
    riskyOperation();
} catch (SpecificException e) {
    handle(e);
} catch (IOException | SQLException e) { // Multi-catch
    handleMultiple(e);
} finally {
    cleanup(); // Always runs
}

// Try-with-resources (auto-close)
try (InputStream is = new FileInputStream(file);
     BufferedReader br = new BufferedReader(new InputStreamReader(is))) {
    return br.readLine();
}

```

Classes & Objects

```

// Class definition
public class PaymentService {
    // Fields
    private final PaymentRepository repo; // final = assigned once
    private static final Logger log = LoggerFactory.getLogger(PaymentService.class);

    // Constructor
    public PaymentService(PaymentRepository repo) {
        this.repo = repo;
    }

    // Method
    public Payment createPayment(BigDecimal amount) {
        // ...
        return payment;
    }

    // Static method
    public static String generateId() {
        return UUID.randomUUID().toString();
    }
}

// Interfaces
public interface Auditable {
    void audit(String action);

    default String prefix() { // Default method (Java 8+)
        return "[AUDIT]";
    }
}

// Abstract class
public abstract class BaseService {
    protected final Logger log = LoggerFactory.getLogger(getClass());

    protected abstract String getServiceName(); // Must implement

    public void logStart() {
        log.info("Starting {}", getServiceName());
    }
}

// Records (Java 16+) – immutable data carriers
public record TransferRequest(
    String fromId,

```

```

        String toId,
        BigDecimal amount
    ) {}

    // Enums
    public enum Status {
        PENDING("P"), ACTIVE("A"), CLOSED("C");

        private final String code;
        Status(String code) { this.code = code; }
        public String getCode() { return code; }
    }

```

Generics

```

// Generic class
public class Pair<A, B> {
    private final A first;
    private final B second;
    // ...
}

// Generic method
public <T> Optional<T> findFirst(List<T> items, Predicate<T> filter) {
    return items.stream().filter(filter).findFirst();
}

// Bounded types
public <T extends Comparable<T>> T max(List<T> items) {
    return items.stream().max(Comparator.naturalOrder()).orElseThrow();
}

// Wildcards
void processAll(List<? extends Animal> animals) { } // Producer Extends
void addToList(List<? super Cat> list) { }           // Consumer Super

```

Collections

```

// List
List<String> list = new ArrayList<>(); // mutable
List<String> immutable = List.of("a", "b"); // immutable (Java 9+)
list.add("x");
list.get(0);
list.remove("x");
list.size();
list.contains("x");
list.sort(Comparator.naturalOrder());

// Map
Map<String, Integer> map = new HashMap<>();
map.put("key", 1);
map.get("key"); // Returns null if missing
map.getDefault("key", 0); // Safe default
map.putIfAbsent("key", 0);
map.computeIfAbsent("key", k -> new ArrayList<>());
map.entrySet().forEach(e -> process(e.getKey(), e.getValue()));
Map<String, Integer> immutableMap = Map.of("a", 1, "b", 2); // Java 9+

// Set
Set<String> set = new HashSet<>();
set.add("x");
set.contains("x");
set.remove("x");

// Queue / Stack
Deque<String> deque = new ArrayDeque<>();
deque.push("first"); // Stack: push to front

```

```
deque.pop();           // Stack: pop from front
deque.offer("last");   // Queue: add to back
deque.poll();          // Queue: remove from front
```

Streams

```
List<Transaction> txns = getTransactions();

// Filter + Map + Collect
List<String> ids = txns.stream()
    .filter(t -> t.getAmount().compareTo(BigDecimal.ZERO) > 0)
    .map(Transaction::getId)
    .collect(Collectors.toList()); // Or .toList() in Java 16+

// Reduce
BigDecimal total = txns.stream()
    .map(Transaction::getAmount)
    .reduce(BigDecimal.ZERO, BigDecimal::add);

// Group by
Map<String, List<Transaction>> byStatus = txns.stream()
    .collect(Collectors.groupingBy(t -> t.getStatus().name()));

// Count / Sum / Average
long count = txns.stream().filter(t -> t.isHighRisk()).count();
double avg = txns.stream().mapToDouble(t -> t.getAmount().doubleValue()).average().orElse(0);

// FlatMap (flatten nested lists)
List<OrderItem> allItems = orders.stream()
    .flatMap(order -> order.getItems().stream())
    .collect(Collectors.toList());

// Distinct / Sorted / Limit / Skip
List<String> unique = txns.stream()
    .map(Transaction::getCategory)
    .distinct()
    .sorted()
    .limit(10)
    .collect(Collectors.toList());

// anyMatch / allMatch / noneMatch
boolean hasLarge = txns.stream().anyMatch(t -> t.getAmount().compareTo(LargeThreshold) > 0);

// findFirst / findAny
Optional<Transaction> first = txns.stream().filter(Transaction::isPending).findFirst();
```

Functional Interfaces

```
// The core four
Function<String, Integer> parser = Integer::parseInt;           // T → R
Predicate<String> isEmpty = String::isEmpty;                   // T → boolean
Consumer<String> printer = System.out::println;                 // T → void
Supplier<UUID> idGen = UUID::randomUUID;                       // () → T

// Composing
Function<String, Integer> parseAndDouble = parser.andThen(n -> n * 2);
Predicate<String> notBlank = isEmpty.negate().and(s -> !s.isBlank());

// BiFunction, BiPredicate, BiConsumer
BiFunction<String, Integer, String> repeat = String::repeat;
```

Lambdas & Method References

```
// Lambda forms
Runnable r1 = () -> System.out.println("Hello");
Runnable r2 = () -> {
    System.out.println("Line 1");
    System.out.println("Line 2");
};

// Method references
// Static:    ClassName::staticMethod
// Instance: instance::method OR ClassName::instanceMethod
// Constructor: ClassName::new

Function<String, Integer> parse = Integer::parseInt;    // static
Consumer<String> log = logger::info;                  // instance
Supplier<List<String>> factory = ArrayList::new;        // constructor
```

String Operations

```
String s = " Hello, World! ";

s.trim()                // Remove whitespace
s.strip()               // Unicode-aware trim (Java 11+)
s.toLowerCase() / s.toUpperCase()
s.contains("World")
s.startsWith("Hello")
s.endsWith("!")
s.indexOf("World")      // -1 if not found
s.substring(7, 12)     // "World"
s.replace("World", "Java")
s.split(",")           // String[] {" Hello", " World! "}
s.split(",", 2)        // Limit splits
s.isEmpty() / s.isBlank() // "" vs " "
s.formatted("name=%s age=%d", name, age) // Java 15+
s.chars()              // IntStream of char codes

// Join
String joined = String.join(", ", list);
String joined2 = list.stream().collect(Collectors.joining(", "));

// StringBuilder
StringBuilder sb = new StringBuilder();
sb.append("Hello").append(", ").append("World");
String result = sb.toString();

// Text block (Java 15+)
String json = """
    {
        "name": "%s"
    }
    """.formatted(name);
```

Annotations Reference

```
// Core Java
@Override           // Must override superclass method
@Deprecated        // Mark as deprecated (use @since in Javadoc)
@SuppressWarnings  // Suppress compiler warning
@FunctionalInterface // Single abstract method interface

// Spring
@SpringBootApplication // Main app entry point
@Component            // Generic Spring bean
@Service              // Business logic bean
```

```

@Repository // Data access bean (+ exception translation)
@RestController // HTTP controller (includes @Controller + @ResponseBody)
@RequestMapping // Map HTTP requests
@GetMapping @PostMapping @PutMapping @DeleteMapping @PatchMapping
@PathVariable // URL path variable
@RequestParam // Query parameter
@RequestBody // Deserialize request body
@RequestHeader // HTTP header
@ResponseStatus // Set response HTTP status
@Autowired // Inject dependency (prefer constructor injection)
@Value("${prop}") // Inject property value
@Configuration // Spring configuration class
@Bean // Define a Spring bean
@Transactional // Wrap in database transaction
@Cacheable / @CacheEvict / @CachePut // Cache operations
@Async // Execute asynchronously
@Scheduled // Scheduled task
@Valid / @Validated // Trigger validation
@ConditionalOnProperty // Conditional bean creation

// JPA / Hibernate
@Entity // JPA entity (maps to table)
@Table(name = "...") // Specify table name
@Id // Primary key
@GeneratedValue // Auto-generate ID
@Column // Column mapping
@OneToMany / @ManyToOne / @OneToOne / @ManyToMany
@JoinColumn // Foreign key column
@Transient // Not persisted to DB
@Version // Optimistic locking version
@Enumerated(STRING) // Store enum as string
@CreatedDate / @LastModifiedDate // Audit timestamps

// Validation
@NotNull @NotBlank @NotEmpty
@Size(min, max)
@Min @Max @DecimalMin @DecimalMax
@email @Pattern(regexp)
@Positive @PositiveOrZero @Negative
@Past @Future @FutureOrPresent

```


Spring Boot Cheatsheet

Essential Patterns for Enterprise Java

Spring Boot Starter Dependencies

```
<!-- pom.xml essential starters -->

<!-- Web API -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<!-- Security -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<!-- Database -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
</dependency>

<!-- Redis Cache -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>

<!-- Kafka -->
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>

<!-- Actuator (health, metrics) -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<!-- Validation -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<!-- Testing -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>junit-jupiter</artifactId>
  <scope>test</scope>
</dependency>
```

application.yml Reference

```
# Server
server:
  port: 8080
  shutdown: graceful
  servlet:
    context-path: /

# Spring
spring:
  application:
    name: payment-service
  profiles:
    active: ${APP_ENV:local}

# Database
datasource:
  url: ${DB_URL:jdbc:postgresql://localhost:5432/payments}
  username: ${DB_USER:postgres}
  password: ${DB_PASSWORD:secret}
  hikari:
    maximum-pool-size: 10
    minimum-idle: 5
    connection-timeout: 30000
    idle-timeout: 600000
    max-lifetime: 1800000
    pool-name: PaymentPool

# JPA
jpa:
  hibernate:
    ddl-auto: validate          # NEVER create/update in production
    show-sql: false            # true only for debugging
    open-in-view: false        # Disable OSIV – avoid lazy loading in view layer
  properties:
    hibernate:
      dialect: org.hibernate.dialect.PostgreSQLDialect
      format_sql: true
      default_batch_fetch_size: 100  # Batch lazy loads

# Redis
data:
  redis:
    host: ${REDIS_HOST:localhost}
    port: ${REDIS_PORT:6379}
    password: ${REDIS_PASSWORD:}
    timeout: 2000ms

# Cache
cache:
  type: redis
  redis:
    time-to-live: 300000ms      # 5 minutes default

# Lifecycle
lifecycle:
  timeout-per-shutdown-phase: 30s

# Actuator
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus,loggers
  endpoint:
    health:
      show-details: when-authorized
    probes:
      enabled: true
  metrics:
    export:
      prometheus:
```

```

        enabled: true
    tracing:
        sampling:
            probability: 0.1

# Logging
logging:
    level:
        root: INFO
        com.bank: DEBUG
        org.hibernate.SQL: DEBUG          # Enable for query debugging
        org.springframework.web: INFO
    pattern:
        console: "%d{ISO8601} [%thread] %-5level %logger{36} [%X{traceId},%X{spanId}] - %msg%n"

```

REST Controller Patterns

```

@RestController
@RequestMapping("/api/v1/payments")
@Validated
@Slf4j
public class PaymentController {

    private final PaymentService service;

    // Constructor injection – always
    public PaymentController(PaymentService service) {
        this.service = service;
    }

    // GET list with pagination
    @GetMapping
    public Page<PaymentResponse> list(
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "20") int size,
        @RequestParam(required = false) String status,
        @SortDefault(sort = "createdAt", direction = Sort.Direction.DESC)
        Pageable pageable) {
        return service.findAll(status, pageable);
    }

    // GET by ID
    @GetMapping("/{id}")
    public PaymentResponse getById(@PathVariable UUID id) {
        return service.findById(id);
    }

    // POST create
    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public PaymentResponse create(
        @Valid @RequestBody CreatePaymentRequest request,
        @RequestHeader("X-Idempotency-Key") String idempotencyKey) {
        return service.create(request, idempotencyKey);
    }

    // PUT update
    @PutMapping("/{id}")
    public PaymentResponse update(
        @PathVariable UUID id,
        @Valid @RequestBody UpdatePaymentRequest request) {
        return service.update(id, request);
    }

    // DELETE
    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable UUID id) {
        service.delete(id);
    }
}

```

```
// Global exception handler
@RestControllerAdvice
@Slf4j
public class GlobalExceptionHandler {

    @ExceptionHandler(NotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorResponse handleNotFound(NotFoundException ex) {
        return new ErrorResponse("NOT_FOUND", ex.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ValidationErrorResponse handleValidation(MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors()
            .forEach(e -> errors.put(e.getField(), e.getDefaultMessage()));
        return new ValidationErrorResponse("VALIDATION_ERROR", errors);
    }

    @ExceptionHandler(Exception.class)
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
    public ErrorResponse handleGeneral(Exception ex) {
        log.error("Unhandled exception", ex);
        return new ErrorResponse("INTERNAL_ERROR", "An unexpected error occurred");
    }
}
```

Service Layer Patterns

```
@Service
@Transactional(readOnly = true) // Default read-only for all methods
@Slf4j
public class PaymentService {

    private final PaymentRepository paymentRepo;
    private final AccountRepository accountRepo;
    private final ApplicationEventPublisher eventPublisher;

    public PaymentService(PaymentRepository paymentRepo,
                          AccountRepository accountRepo,
                          ApplicationEventPublisher eventPublisher) {
        this.paymentRepo = paymentRepo;
        this.accountRepo = accountRepo;
        this.eventPublisher = eventPublisher;
    }

    public Page<PaymentResponse> findAll(String status, Pageable pageable) {
        Specification<Payment> spec = status != null
            ? (root, q, cb) -> cb.equal(root.get("status"), PaymentStatus.valueOf(status))
            : Specification.where(null);
        return paymentRepo.findAll(spec, pageable).map(PaymentResponse::from);
    }

    public PaymentResponse findById(UUID id) {
        return paymentRepo.findById(id)
            .map(PaymentResponse::from)
            .orElseThrow(() -> new NotFoundException("Payment not found: " + id));
    }

    @Transactional // Override to write transaction
    public PaymentResponse create(CreatePaymentRequest request, String idempotencyKey) {
        // Check idempotency
        paymentRepo.findByIdempotencyKey(idempotencyKey).ifPresent(existing -> {
            throw new DuplicateOperationException("Payment already processed: " + idempotencyKey);
        });

        // Load and validate
        Account fromAccount = accountRepo.findByIdForUpdate(request.getFromAccountId())
            .orElseThrow(() -> new NotFoundException("Account: " + request.getFromAccountId()));
    }
}
```

```

        if (fromAccount.getBalance().compareTo(request.getAmount()) < 0) {
            throw new InsufficientFundsException(request.getAmount(), fromAccount.getBalance());
        }

        // Execute
        fromAccount.debit(request.getAmount());
        Payment payment = paymentRepo.save(Payment.create(request, idempotencyKey));

        // Publish event (after commit - @TransactionalEventListener)
        eventPublisher.publishEvent(new PaymentCreatedEvent(payment));

        log.info("Payment created: id={}" amount={},", payment.getId(), payment.getAmount());
        return PaymentResponse.from(payment);
    }
}

```

JPA Repository Patterns

```

@Repository
public interface PaymentRepository extends JpaRepository<Payment, UUID>,
    JpaSpecificationExecutor<Payment> {

    // Method naming query
    Optional<Payment> findByIdempotencyKey(String key);
    List<Payment> findByStatusAndCreatedAtAfter(PaymentStatus status, Instant cutoff);
    boolean existsByIdempotencyKey(String key);
    long countByStatus(PaymentStatus status);

    // Custom JPQL
    @Query("SELECT p FROM Payment p WHERE p.accountId = :id AND p.amount > :min ORDER BY p.amount DESC")
    List<Payment> findLargeByAccount(@Param("id") String accountId,
        @Param("min") BigDecimal minAmount);

    // Pessimistic lock
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("SELECT p FROM Payment p WHERE p.id = :id")
    Optional<Payment> findByIdForUpdate(@Param("id") UUID id);

    // Projection
    List<PaymentSummary> findByAccountId(String accountId);

    // Streaming (large datasets)
    @QueryHints(@QueryHint(name = "org.hibernate.fetchSize", value = "500"))
    Stream<Payment> findAllByStatus(PaymentStatus status);

    // Native query
    @Query(value = "SELECT * FROM payments WHERE amount > :amount ORDER BY created_at DESC LIMIT 100",
        nativeQuery = true)
    List<Payment> findTopLargePayments(@Param("amount") BigDecimal amount);
}

// Projection interface
public interface PaymentSummary {
    UUID getId();
    BigDecimal getAmount();
    PaymentStatus getStatus();
    Instant getCreatedAt();
}

```

Kafka Producer & Consumer

```

// Producer
@Service
public class PaymentEventProducer {

    private final KafkaTemplate<String, Object> kafkaTemplate;
}

```

```

        public void publishPaymentCreated(Payment payment) {
            PaymentCreatedEvent event = PaymentCreatedEvent.from(payment);
            kafkaTemplate.send("payment.created", payment.getId().toString(), event)
                .whenComplete((result, ex) -> {
                    if (ex != null) {
                        log.error("Failed to publish event: {}", payment.getId(), ex);
                        outboxService.saveForRetry(event);
                    } else {
                        log.debug("Event published: partition={} offset={}",
                            result.getRecordMetadata().partition(),
                            result.getRecordMetadata().offset());
                    }
                });
        }
    }

    // Consumer
    @Component
    @Slf4j
    public class PaymentEventConsumer {

        @KafkaListener(topics = "payment.created", groupId = "notification-service")
        public void handle(
            @Payload PaymentCreatedEvent event,
            @Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
            @Header(KafkaHeaders.OFFSET) long offset,
            Acknowledgment ack) {

            log.info("Processing: id={} partition={} offset={}", event.getPaymentId(), partition, offset);

            try {
                // Idempotency check
                if (processedEventRepo.existsByEventId(event.getEventId())) {
                    log.info("Duplicate, skipping: {}", event.getEventId());
                    ack.acknowledge();
                    return;
                }

                notificationService.notifyPaymentCreated(event);
                processedEventRepo.markProcessed(event.getEventId());
                ack.acknowledge();

            } catch (Exception e) {
                log.error("Failed to process event: {}", event.getEventId(), e);
                throw e; // Trigger retry/DLQ
            }
        }
    }
}

```

Testing Patterns

```

// Unit test
@ExtendWith(MockitoExtension.class)
class PaymentServiceTest {

    @Mock PaymentRepository paymentRepo;
    @Mock AccountRepository accountRepo;
    @Mock ApplicationEventPublisher eventPublisher;
    @InjectMocks PaymentService paymentService;

    @Test
    void createPayment_shouldDebitAccount() {
        Account account = Account.withBalance(new BigDecimal("1000.00"));
        when(accountRepo.findByIdForUpdate(any())).thenReturn(Optional.of(account));
        when(paymentRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));

        paymentService.create(new CreatePaymentRequest("ACC-1", "ACC-2",
            new BigDecimal("100.00"), "IDEM-1");

        assertThat(account.getBalance()).isEqualByComparingTo("900.00");
    }
}

```

```

        verify(eventPublisher).publishEvent(any(PaymentCreatedEvent.class));
    }
}

// Integration test with Testcontainers
@SpringBootTest
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
@Testcontainers
class PaymentIntegrationTest {

    @Container
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:15");

    @Container
    static KafkaContainer kafka = new KafkaContainer(DockerImageName.parse("confluentinc/cp-kafka:7.6.0"));

    @DynamicPropertySource
    static void props(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.kafka.bootstrap-servers", kafka::getBootstrapServers);
    }

    @Autowired MockMvc mockMvc;
    @Autowired ObjectMapper objectMapper;

    @Test
    void createPayment_returnsCreated() throws Exception {
        mockMvc.perform(post("/api/v1/payments")
            .contentType(MediaType.APPLICATION_JSON)
            .header("X-Idempotency-Key", UUID.randomUUID().toString())
            .content(objectMapper.writeValueAsString(testRequest)))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.paymentId").isNotEmpty())
            .andExpect(jsonPath("$.status").value("PENDING"));
    }
}

```

Configuration Patterns

```

// Type-safe configuration
@ConfigurationProperties(prefix = "payment")
@Validated
@Data // Lombok
public class PaymentConfig {

    @NotNull
    @Valid
    private Limits limits;

    @Min(100) @Max(60000)
    private int timeoutMs = 5000;

    @NotBlank
    private String processorUrl;

    @Data
    public static class Limits {
        @NotNull @DecimalMin("0.01")
        private BigDecimal maxSinglePayment;

        @NotNull @DecimalMin("0.01")
        private BigDecimal dailyLimit;
    }
}

// Enable in main class or config
@EnableConfigurationProperties(PaymentConfig.class)
@SpringBootApplication
public class Application { ... }

```

Topic Priority Matrix

What to Learn First — Ranked by Interview Weight and Career Impact

Priority Tiers

Tier 1 — Master These First (Critical Path)

Learn these before any other topic. They appear in every interview.

Topic	Section	Time to Learn	Interview Frequency
Java OOP, Generics, Collections	1	1 week	Every Java interview
Streams & Lambdas	1	3 days	Very high
Spring DI + Bean Lifecycle	3	4 days	Every Spring interview
Spring MVC + REST APIs	3	3 days	Very high
@Transactional internals	3, 5	2 days	Very high
Node.js → Java mapping	14	Ongoing	Your bridge concept
JVM Heap/Stack/GC basics	2	3 days	High
CompletableFuture basics	7	2 days	High
SQL optimization + JPA N+1	5	4 days	Very high
Concurrency basics (sync, volatile, AtomicXxx)	7	1 week	Very high

Tier 2 — Learn Within Month 2 (Differentiators)

These topics set senior candidates apart from junior ones.

Topic	Section	Time to Learn	Interview Frequency
Spring Security + JWT + OAuth2	9	1 week	High (banks)
Transaction isolation levels	5	3 days	High (banks)
Kafka producers/consumers	6	1 week	Very high
Circuit breaker + Retry	4	3 days	High
ThreadPoolExecutor internals	7	3 days	High
GC algorithms + tuning	2	1 week	Medium-High
HikariCP sizing	5	1 day	Medium
Idempotency patterns	4	2 days	High (payments)
Redis patterns (cache-aside, locks)	10	3 days	High
System design fundamentals	10	Ongoing	Very high

Tier 3 — Learn Within Month 3 (Depth)

Needed for senior/staff roles and deep specialization.

Topic	Section	Time to Learn	Interview Frequency
Saga pattern + distributed transactions	6	1 week	High (banks)
Event sourcing + CQRS	6	1 week	Medium-High
CAP theorem + PACELC	6	3 days	Medium
AOP internals	3	3 days	Medium
Spring WebFlux (reactive)	3	1 week	Medium
Flyway/Liquibase migrations	5	2 days	Medium
OWASP + secure coding	9	1 week	High (banks)
JVM profiling + thread dumps	2	1 week	Medium
Deadlock prevention + race conditions	7	3 days	High
K8s deployment for Java	8	3 days	Medium

Tier 4 — Learn Within Month 4-5 (Mastery)

Needed for top-tier companies and principal/staff-level roles.

Topic	Section	Time to Learn	Interview Frequency
Virtual threads (Project Loom)	7	2 days	Growing
GraalVM native images	8	3 days	Low-Medium
Lock-free programming (CAS)	7	1 week	High (FAANG)
JVM bytecode and classloading	2	1 week	Medium (staff)
Spring autoconfiguration internals	3	3 days	Medium
Elasticsearch integration	10	1 week	Medium
API gateway design	4, 10	1 week	Medium
Advanced Kafka (transactions, streams)	6	1 week	Medium-High
DDD + aggregate design	6, 13	2 weeks	High (banks)
Architecture decision records	12, 13	Ongoing	Growing

Company-Specific Priority

FAANG Companies (Google, Amazon, Meta, Apple, Netflix)

MUST master (ranked):

1. DSA — LeetCode Medium/Hard fluency (non-negotiable)
2. System Design HLD — design at scale (millions of users)
3. Behavioral (Amazon: Leadership Principles, Google: Googliness)
4. Java concurrency — thread safety, deadlocks, CompletableFuture
5. JVM internals — GC, heap, performance tuning
6. Distributed systems — Kafka, consistency, CAP

Good to have:

- Spring expertise (assumed, not deeply tested at FAANG)
- Specific database internals
- Cloud-specific tools

Global Banks (HSBC, Goldman, JP Morgan, Morgan Stanley)

MUST master (ranked):

1. Spring ecosystem – deep knowledge (DI, security, data)
2. Transaction management – isolation levels, distributed transactions
3. Security – OAuth2, JWT, OWASP, PCI-DSS awareness
4. Kafka + event-driven – message ordering, idempotency
5. JPA/Hibernate – N+1, caching, pessimistic/optimistic locking
6. Compliance patterns – audit logging, immutable audit trails

Good to have:

- DSA (asked, but medium difficulty is sufficient)
- System design (asked, but financial systems focus)
- Saga/event sourcing (asked in senior roles)

Product Companies (Stripe, Atlassian, Uber, Adobe)

MUST master (ranked):

1. API design quality – REST best practices, versioning
2. Microservices resilience – CB, retry, timeouts
3. Kafka / event-driven architecture
4. Observability – metrics, tracing, logging
5. DSA – Medium level solid, Hard exposure
6. System design – product-specific systems

Good to have:

- Spring expertise
- Database optimization
- Cloud infrastructure (AWS/GCP)

Learning Time Investment Guide

Weeks 1-2: Tier 1 core language (20h/week = 40h total)

Weeks 3-4: Tier 1 Spring + first project (20h/week)

Weeks 5-8: Tier 2 topics + second project (15h/week)

Weeks 9-12: Tier 3 topics + interview prep starts (15h/week)

Weeks 13-20: Interview prep + Tier 4 topics + mock interviews (15h/week)

Total investment: ~300 hours over 5 months for interview-ready proficiency
~500 hours for staff-level depth

Quick Assessment: Are You Ready?

Junior Ready (6-12 month target at mid-tier companies)

- ☐ Can write a Spring Boot CRUD API without help
- ☐ Understand @Transactional and its main pitfalls
- ☐ Can solve LeetCode Easy problems in Java fluently
- ☐ Know the main JPA annotations and can avoid N+1
- ☐ Can explain JWT authentication flow

Senior Ready (Target: HSBC, Atlassian, Stripe)

- ☐ Can design a microservices system from scratch
- ☐ Know Kafka producer/consumer patterns cold
- ☐ Can explain transaction isolation levels and when to use each
- ☐ Proficient in LeetCode Medium problems
- ☐ Can implement circuit breaker and retry patterns
- ☐ Know OAuth2 client credentials flow deeply

Staff Ready (Target: FAANG, Goldman, Stripe senior)

- ☐ Can design a global payment system with 100k TPS
- ☐ Deep JVM knowledge (GC tuning, thread dump diagnosis)
- ☐ Can implement event sourcing and CQRS
- ☐ LeetCode Hard familiarity
- ☐ Can articulate trade-offs for major architectural decisions
- ☐ Can lead technical design discussions with clear, structured thinking

6-Month Java Mastery Roadmap

From Node.js Engineer to Enterprise Java Senior

Overview

```
Month 1: Language & JVM Foundation
Month 2: Spring Ecosystem Mastery
Month 3: Distributed Systems & Concurrency
Month 4: Production Engineering & Security
Month 5: Interview Preparation Intensive
Month 6: Specialization & Targeting
```

Month 1: Language & JVM Foundation

Goals

- Write Java code without referring to syntax documentation
- Understand JVM well enough to explain memory and GC
- Build your first Spring Boot API

Week 1: Java Language (20 hours)

```
Day 1-2: Section 14 (Node.js Mapping) – establish mental model
Day 3-4: Section 1 (Java Fundamentals Part 1)
→ Variables, types, OOP, interfaces vs abstract classes
Day 5-7: Section 1 (Java Fundamentals Part 2)
→ Collections, Streams, Generics, Lambdas

Daily practice:
→ Rewrite 2 Node.js utilities in Java
→ Solve 2 LeetCode Easy in Java
```

Week 2: JVM + Environment Setup (20 hours)

```
Day 1-2: Section 2 (JVM Architecture)
→ Heap, stack, metaspace, GC basics
Day 3-4: Section 2 (GC + JIT + Performance)
Day 5: Setup development environment:
→ IntelliJ IDEA (Community or Ultimate)
→ Java 21 (Temurin distribution)
→ Maven or Gradle
→ Docker Desktop
Day 6-7: Hands-on: JVM monitoring with VisualVM
→ Profile a simple app, observe GC, heap growth

Daily practice:
→ 2 LeetCode Easy problems
→ Write and profile JVM code
```

Week 3-4: First Project — Banking API (30 hours)

```
Build: Spring Boot REST API with:
→ @RestController + @Service + @Repository layers
→ PostgreSQL + JPA entities + Flyway migrations
→ Spring Security + JWT auth
→ Bean Validation on requests
→ @ControllerAdvice error handling
```

→ JUnit 5 + Mockito unit tests

Reference: Section 3 (Spring), Section 5 (DB basics), Section 15 (project guide)

Daily practice:

→ 2 LeetCode Easy/Medium problems

Month 1 Milestones

- ☐ Banking API deployed and tested locally
- ☐ Can explain JVM heap/stack/GC without notes
- ☐ 30 LeetCode problems solved in Java
- ☐ Comfortable reading Spring Boot code

Month 2: Spring Ecosystem Mastery

Goals

- Deep Spring knowledge (DI, AOP, Security, Data)
- Understand `@Transactional` completely
- Build production-quality service layer

Week 5-6: Spring Deep Dive (25 hours)

Study: Section 3 (Enterprise Java Ecosystem)

- Bean lifecycle (`BeanPostProcessor`, `@PostConstruct`)
- AOP proxy model (`@Transactional` pitfalls)
- Spring Security filter chain
- `@ConfigurationProperties`
- Spring Events (`@TransactionalEventListener`)

Hands-on: Add to Banking API:

- Spring Security + OAuth2 Resource Server
- AOP aspect for audit logging
- `@Cacheable` with Redis
- `@Async` for notification sending

Daily practice:

→ 2 LeetCode Medium problems

Week 7-8: Database Excellence (20 hours)

Study: Section 5 (Databases & Persistence)

- Transaction isolation levels (write code to demonstrate each)
- Optimistic vs pessimistic locking
- N+1 problem: diagnose and fix
- HikariCP configuration
- Flyway migration best practices

Hands-on:

- Add `@Version` (optimistic locking) to Payment entity
- Fix all N+1 queries (enable SQL logging, count queries)
- Add custom indexes, run `EXPLAIN ANALYZE`
- Simulate connection pool exhaustion, add leak detection

Daily practice:

→ 2 LeetCode Medium problems (focus on HashMap, sorting patterns)

Month 2 Milestones

- ☐ Can explain Spring bean lifecycle completely
- ☐ Can identify and fix `@Transactional` pitfalls
- ☐ Banking API has production-quality error handling, validation, security

- ☐ 30 LeetCode Medium problems solved
 - ☐ Can explain N+1 and fix it 3 different ways
-

Month 3: Distributed Systems & Concurrency

Goals

- Master Kafka patterns
- Understand Java concurrency model deeply
- Build multi-service project

Week 9-10: Kafka + Distributed Systems (25 hours)

```
Study: Section 6 (Distributed Systems)
→ Kafka architecture (partitions, offsets, consumer groups)
→ Delivery guarantees (at-least-once, exactly-once)
→ DLQ pattern implementation
→ Saga pattern (choreography first, then orchestration)
→ CAP theorem with real examples

Hands-on: Start Project 2 (Event-Driven Payment Platform)
→ Payment service publishes events
→ Notification service consumes events
→ Idempotent consumer implementation
→ DLQ handler

Daily practice:
→ 2 LeetCode Medium problems (graphs, BFS/DFS patterns)
```

Week 11-12: Concurrency (25 hours)

```
Study: Section 7 (Concurrency & Multithreading)
→ synchronized vs volatile vs AtomicXxx
→ ThreadPoolExecutor configuration
→ CompletableFuture chains
→ Deadlock detection and prevention
→ ConcurrentHashMap patterns
→ Virtual threads (Java 21)

Hands-on:
→ Implement thread-safe rate limiter
→ Create race condition, diagnose with jstack, fix
→ Implement parallel payment enrichment with CompletableFuture.allOf()
→ Configure custom thread pool for @Async

Daily practice:
→ 2 LeetCode Medium-Hard (concurrency thinking)
```

Month 3 Milestones

- ☐ Event-Driven Payment Platform running locally with Docker Compose
 - ☐ Can implement Kafka producer/consumer with DLQ from scratch
 - ☐ Can identify and fix race conditions in code review
 - ☐ Can explain all Java concurrency primitives and when to use each
 - ☐ 30 LeetCode Medium problems solved
-

Month 4: Production Engineering & Security

Goals

- Production-ready observability

- Resilience patterns (CB, retry, rate limiting)
- Security deep knowledge

Week 13-14: Production Patterns (20 hours)

Study: Section 4 (Production Backend Engineering)
 Section 8 (Cloud & DevOps)

- Circuit breaker + retry (Resilience4j)
- Rate limiting (Bucket4j)
- Distributed tracing (Micrometer + Zipkin)
- Prometheus metrics + Grafana dashboards
- Kubernetes deployment with proper probes
- Docker multi-stage builds

Hands-on:

- Add Resilience4j to external calls
- Add Prometheus metrics + custom business metrics
- Configure liveness/readiness probes
- Deploy to local k8s (minikube)

Daily practice:

- 2 LeetCode Medium-Hard
- 1 system design study (using template from Section 10)

Week 15-16: Security (20 hours)

Study: Section 9 (Security)

- OAuth2 flows in depth (authorization code + PKCE, client credentials)
- JWT validation (all checks, vulnerability patterns)
- Spring Security configuration
- OWASP Top 10 mitigations
- Secrets management (Vault, AWS Secrets Manager)

Hands-on:

- Implement full OAuth2 with Keycloak (local)
- Service-to-service client credentials flow
- Intentionally introduce SQL injection → fix it
- Add rate limiting to auth endpoints

Daily practice:

- 2 LeetCode problems
- 1 system design study

Month 4 Milestones

- ☐ Payment Platform has full observability (traces, metrics, logs)
- ☐ Can configure Spring Security + JWT + OAuth2 from scratch
- ☐ Can explain OWASP Top 10 with Java-specific examples
- ☐ Can deploy Spring Boot service to k8s with proper configuration
- ☐ 20 system design scenarios studied

Month 5: Interview Preparation Intensive

Goals

- Interview-ready performance on all dimensions
- 5+ mock interviews completed
- Behavioral stories prepared

Week 17-18: DSA Intensive (30 hours)

Study: Section 11 (Interview Preparation)

- Review all 20 algorithm patterns

- LeetCode: 3 problems/day (mix Medium and Hard)
- Focus: Graph algorithms, Dynamic Programming, Heap

Practice routine:

- Morning: 1 LeetCode problem (45-minute mock)
- Evening: 1 LeetCode problem (review optimal solution)
- Weekend: 2 full mock coding interviews (with peers/Pramp)

Week 19-20: System Design + Behavioral (25 hours)

System Design:

- Practice: payment system, notification system, rate limiter
- Template: use the 45-minute framework from Section 10
- Daily: 1 system design whiteboard session

Behavioral:

- Prepare 10 STAR stories covering all leadership principles
- Record yourself giving answers, review for clarity
- Mock interviews: 1 behavioral + 1 system design per week

Java/Spring deep questions:

- Practice answering all questions from Section 11
- Explain to yourself, then to another person

Month 5 Milestones

- ☐ 150+ LeetCode problems solved total
- ☐ 5+ mock interviews completed
- ☐ 10 STAR behavioral stories prepared and rehearsed
- ☐ Can design any of the 10 key systems in 45 minutes
- ☐ All Spring interview questions answered confidently

Month 6: Specialization & Targeting

Week 21-22: Company Research + Targeting

For each target company:

- Read engineering blog (Netflix Tech Blog, Stripe Blog, etc.)
- Study the specific systems they build
- Find Glassdoor/LeetCode interview reports
- Tailor examples to their domain

FAANG path:

- LeetCode company-tagged problems (Amazon, Google, Meta)
- Leadership principles (Amazon: all 16)
- System design at scale

Banks path:

- Section 6 advanced (event sourcing, CQRS, saga)
- Compliance and audit patterns
- Domain-driven design concepts

Week 23-24: Final Sprint

- 3+ mock interviews per week
- Revisit weak topics from mock feedback
- Complete Project 3 (CQRS Event Store) if targeting senior/staff at banks
- GitHub portfolio: clean READMEs, architecture diagrams

Final checklist:

- Resume updated with Java projects
- GitHub shows Java code quality
- Can explain every bullet point on resume in depth

- Confident with top 10 system designs
- Confident with top 50 Java/Spring interview questions

Daily Practice Habits (Throughout All 6 Months)

Morning (30 min):

- 1 LeetCode problem in Java (timed: 25 min)
- Review optimal solution if needed

Evening (30 min):

- Read one section subsection
- Take notes in your own words (active recall)

Weekend (2-3 hours):

- Build feature for current project
- 1 full mock interview session
- 1 system design practice

Tools & Resources

IDE: IntelliJ IDEA Ultimate (request student/OSS license)
Java: Eclipse Temurin 21 (adoptium.net)
Build: Maven (start), then Gradle (bonus)
DB: PostgreSQL 15 (Docker)
Kafka: Confluent Platform (Docker Compose)
Redis: Redis Stack (Docker)
k8s: minikube or k3s (local)
Monitoring: Grafana + Prometheus + Zipkin (Docker Compose)

LeetCode: paid subscription (company tags, premium problems)
Excalidraw: system design whiteboarding
Pramp: free peer mock interviews
interviewing.io: paid mock interviews with engineers from top companies

Java vs Node.js — Complete Comparison Table

Technology Stack Mapping

Category	Node.js	Java
Runtime	V8 (Google)	JVM (Oracle/OpenJDK)
Language	JavaScript / TypeScript	Java / Kotlin / Scala
Type system	Dynamic (JS) / Static (TS)	Statically typed, compiled
Package manager	npm / yarn / pnpm	Maven / Gradle
Package registry	npm registry	Maven Central / JCenter
HTTP framework	Express / Fastify / Koa	Spring MVC / Quarkus / Micronaut
Reactive framework	RxJS / custom	Project Reactor / RxJava
ORM	TypeORM / Prisma / Mongoose	Hibernate / Spring Data JPA
Validation	Joi / Zod / class-validator	Jakarta Validation / Spring Validation
Testing	Jest / Mocha / Vitest	JUnit 5 / TestNG
Mocking	Jest mocks / Sinon	Mockito / EasyMock
API testing	Supertest	MockMvc / REST Assured
DB container tests	testcontainers-node	Testcontainers
Logging	Winston / Pino	Logback / Log4j2 (via SLF4J)
HTTP client	Axios / node-fetch	RestTemplate / WebClient / Feign
Scheduler	node-cron / agenda	Quartz / Spring @Scheduled
Process manager	PM2	JVM (self-managed)
Hot reload	nodemon	Spring DevTools

Architecture Patterns

Pattern	Node.js	Java
Dependency injection	NestJS IoC / manual	Spring IoC Container
AOP / Middleware	Express middleware	Spring AOP (proxy-based)
Config management	dotenv / convict	application.yml / @ConfigurationProperties
Feature flags	LaunchDarkly SDK	LaunchDarkly SDK / Unleash
Rate limiting	express-rate-limit / ioredis	Bucket4j / Resilience4j
Circuit breaker	opossum	Resilience4j
Retry	async-retry	Resilience4j / Spring-Retry
Caching	ioRedis / node-cache	Spring Cache / Caffeine / Redis
Message queue	BullMQ / bee-queue	Spring AMQP (RabbitMQ)
Event streaming	kafkajs	Spring Kafka / Apache Kafka Client

Pattern	Node.js	Java
Service discovery	Consul (client)	Eureka / Consul / K8s DNS
API gateway	Kong / custom	Spring Cloud Gateway
Tracing	OpenTelemetry Node	Micrometer Tracing / Spring Sleuth
Metrics	prom-client	Micrometer + Prometheus

Performance Characteristics

Dimension	Node.js	Java
Startup time	100-500ms	2-10s (JVM) / 50-200ms (GraalVM Native)
Memory (idle)	50-100MB	150-300MB (JVM overhead)
Memory (at load)	Grows linearly	Configured heap (Xmx)
CPU efficiency (I/O)	Excellent	Excellent
CPU efficiency (compute)	Single thread	Multi-thread (excellent)
Concurrent connections	Hundreds of thousands (event loop)	Hundreds (threads) / Millions (virtual threads)
Latency consistency	Very consistent	Variance from GC pauses
Warm-up behavior	Fast (seconds)	Slow (30-60s for JIT optimization)
Peak throughput	Good	Excellent (JIT + multi-thread)
Memory leaks	Common (closures, listeners)	Less common (GC) but possible

Development Experience

Dimension	Node.js	Java
Syntax verbosity	Low (JS) / Medium (TS)	High (more explicit)
Type safety	Optional (TS)	Mandatory
Compile time	None (JS) / Fast (TS)	Slower (Maven/Gradle)
IDE support	VS Code / WebStorm	IntelliJ IDEA (best-in-class)
Refactoring safety	Low (JS) / Medium (TS)	Very high (compiler-enforced)
Test speed	Fast	Slower (Spring context startup)
Debugging	Chrome DevTools / VS Code	IntelliJ debugger (excellent)
Code generation	AI tools	Lombok + AI tools
Boilerplate	Low	Higher (reduced by Lombok/records)
Learning curve	Low (JS)	Steeper

Production Operations

Dimension	Node.js	Java
GC tuning needed	No (V8 manages)	Yes (JVM flags)

Dimension	Node.js	Java
Memory tuning	Usually not	Yes (-Xms, -Xmx, -XX:+...)
Thread dump analysis	Not needed	Yes (jstack, VisualVM)
Heap dump analysis	Node heapdump	jmap + MAT
Profiling tools	clinic.js, 0x	async-profiler, JFR, VisualVM
Container overhead	Low	Higher (JVM startup + memory)
Health checks	Custom express routes	Spring Actuator (built-in)
Graceful shutdown	manual	Spring (server.shutdown=graceful)
Clustering	PM2 cluster / worker_threads	JVM threads (built-in)
Zero-downtime deploy	Rolling k8s update	Rolling k8s update (same)

Ecosystem & Community

Dimension	Node.js	Java
Package count (registry)	2M+ (npm)	500k+ (Maven Central)
Package quality	Variable	Generally mature
Breaking changes	Common (semver not always respected)	Rare (strong backward compatibility)
LTS support	2-3 years	8+ years (Java LTS versions)
Enterprise adoption	Growing	Dominant
Banking/finance usage	Growing	Dominant for decades
Job market	Large	Very large
Salary premium	Moderate	High (enterprise Java)
Open source activity	Very active	Active (Spring, Apache ecosystem)

When to Choose Which

Choose Java/Spring When:

- ✓ Building banking/financial systems (compliance, audit, type safety)
- ✓ Large team (50+ engineers) on same codebase
- ✓ CPU-intensive workloads (data processing, ML inference)
- ✓ Long-lived services needing 5+ year maintainability
- ✓ Strict SLA requirements (JVM GC tunable for latency goals)
- ✓ Enterprise integrations (SAP, Oracle, SWIFT, FIX protocol)
- ✓ Strong typing required by domain complexity or team size
- ✓ Existing Java organization (hiring, tooling, knowledge transfer)

Choose Node.js/TypeScript When:

- ✓ API gateway / BFF (Backend for Frontend) – fast I/O, no CPU
- ✓ Real-time applications (WebSocket, SSE) – event loop natural fit
- ✓ Serverless/edge functions – fast cold start critical
- ✓ Small team, rapid iteration needed
- ✓ Full-stack JavaScript team (shared types/models)
- ✓ Prototype → production pipeline (fast iteration)

- ✓ Heavy JSON/REST API transformation without business logic
- ✓ Green-field startup with < 10 engineers

Key Mental Model Shifts

Node.js Thinking	→	Java Thinking
"Functions and callbacks"	→	"Classes and interfaces"
"prototype chain"	→	"class hierarchy + generics"
"async everything"	→	"synchronous, with async option"
"JSON is first-class"	→	"Objects are first-class"
"duck typing"	→	"explicit interface contracts"
"npm install anything"	→	"evaluate library maturity/security"
"fix the error at runtime"	→	"fix the error at compile time"
"event loop = concurrency"	→	"thread pool = concurrency"
"process.env.PORT"	→	"\${server.port:8080}"
"module.exports = ..."	→	"@Service / @Bean"
"jest.mock('./service')"	→	"Mockito.mock(Service.class)"
"const obj = {...spread}"	→	"new Builder().field(val).build()"
"any type as escape hatch"	→	"Object / wildcard<?>"
"1 process handles all"	→	"200 threads, each handles 1"

Common Mistakes to Avoid

Java & Spring Anti-Patterns That Will Hurt You in Production and Interviews

Java Language Mistakes

1. Using `==` for String/Object Comparison

```
// WRONG
if (status == "ACTIVE") { } // Always false for non-literal strings!
if (user1 == user2) { }     // Compares references, not content

// CORRECT
if ("ACTIVE".equals(status)) { } // Null-safe: literal on left
if (user1.equals(user2)) { }
if (Objects.equals(user1, user2)) { } // Null-safe both sides
```

2. Float/Double for Money

```
// WRONG – floating point imprecision
double price = 0.1 + 0.2; // 0.30000000000000004
double fee = 100.0 * 0.029; // 2.8999999999999996 (not 2.90!)

// CORRECT
BigDecimal price = new BigDecimal("0.1").add(new BigDecimal("0.2")); // 0.3 exactly
BigDecimal fee = new BigDecimal("100.00")
    .multiply(new BigDecimal("0.029"))
    .setScale(2, RoundingMode.HALF_UP); // 2.90
```

3. Modifying Collection While Iterating

```
// WRONG – ConcurrentModificationException
for (String item : list) {
    if (shouldRemove(item)) list.remove(item);
}

// CORRECT
list.removeIf(this::shouldRemove); // Or use Iterator.remove()
// OR
List<String> toRemove = list.stream()
    .filter(this::shouldRemove)
    .collect(Collectors.toList());
list.removeAll(toRemove);
```

4. Not Closing Resources

```
// WRONG – connection never closed if exception thrown
Connection conn = dataSource.getConnection();
conn.prepareStatement(sql).execute();
conn.close(); // Skipped if exception above!

// CORRECT
try (Connection conn = dataSource.getConnection();
     PreparedStatement stmt = conn.prepareStatement(sql)) {
    stmt.execute();
} // Auto-closed
```

5. Null Return Instead of Optional

```
// WRONG – forces callers to do null checks everywhere
public User findUser(String id) {
    return userRepo.findById(id); // Returns null if not found
}

// CORRECT – explicit in signature that value may be absent
public Optional<User> findUser(String id) {
    return userRepo.findById(id);
}
// Callers: service.findUser(id).orElseThrow(() -> new UserNotFoundException(id))
```

6. String Concatenation in Loops

```
// WRONG – creates N intermediate String objects
String result = "";
for (String item : items) {
    result += item + ", "; // O(n²) memory allocation!
}

// CORRECT
StringBuilder sb = new StringBuilder();
for (String item : items) sb.append(item).append(", ");
// OR: String.join(", ", items)
// OR: items.stream().collect(Collectors.joining(", "))
```

Spring Mistakes

7. Field Injection Instead of Constructor Injection

```
// WRONG – cannot test without Spring context, hides dependencies
@Service
public class PaymentService {
    @Autowired
    private PaymentRepository repo; // Hidden dependency
}

// CORRECT – explicit, testable, immutable
@Service
public class PaymentService {
    private final PaymentRepository repo;

    public PaymentService(PaymentRepository repo) {
        this.repo = repo;
    }
}
```

8. @Transactional Self-Invocation

```
// WRONG – self-invocation bypasses proxy, no transaction!
@Service
public class OrderService {
    public void processOrder(Order order) {
        this.placeOrder(order); // Calls own method – NO transaction!
    }

    @Transactional
    public void placeOrder(Order order) { ... }
}

// CORRECT – inject self via ApplicationContext, or refactor
@Service
public class OrderService {
    private final OrderService self; // Self-injection via Spring
```

```
// OR: extract placeOrder to separate @Service
}
```

9. @Transactional on Private Methods

```
// WRONG – Spring proxy cannot intercept private methods
@Service
public class PaymentService {
    @Transactional
    private void updateBalance(String id, BigDecimal amount) {
        // @Transactional is IGNORED – no proxy for private!
    }
}

// CORRECT – must be public (or at minimum protected)
@Transactional
public void updateBalance(String id, BigDecimal amount) { ... }
```

10. Checked Exceptions Not Rolling Back @Transactional

```
// WRONG – checked exceptions don't roll back by default
@Transactional
public void processPayment(Payment payment) throws PaymentException {
    accountRepo.debit(payment.getFromAccount(), payment.getAmount());
    // EXCEPTION THROWN HERE → transaction COMMITS debit!
    if (!externalGateway.charge(payment)) {
        throw new PaymentException("Gateway declined"); // Checked!
    }
}

// CORRECT
@Transactional(rollbackFor = Exception.class) // Roll back on any exception
public void processPayment(Payment payment) throws PaymentException { ... }

// OR: use unchecked exception
public class PaymentException extends RuntimeException { } // Rolls back automatically
```

JPA/Database Mistakes

11. N+1 Query Problem

```
// WRONG – fires 1 + N queries
List<Order> orders = orderRepo.findAll(); // 1 query
for (Order order : orders) {
    String name = order.getCustomer().getName(); // N queries!
}

// CORRECT – 1 query with JOIN FETCH
@Query("SELECT o FROM Order o JOIN FETCH o.customer")
List<Order> findAllWithCustomer();
```

12. FetchType.EAGER on Collections

```
// WRONG – loads ALL items every time you load an Order
@OneToMany(fetch = FetchType.EAGER) // NEVER on collections
private List<OrderItem> items;

// CORRECT – lazy load, fetch when needed
@OneToMany(fetch = FetchType.LAZY) // Default for collections – keep it!
private List<OrderItem> items;
// Use @EntityGraph or JOIN FETCH to load when needed
```


13. Missing @Version for Concurrent Updates

```
// WRONG – concurrent updates: last-write-wins (silently)
@Entity
public class Account {
    private BigDecimal balance; // Two threads update: one update lost!
}

// CORRECT – optimistic locking
@Entity
public class Account {
    private BigDecimal balance;
    @Version
    private Long version; // Throws OptimisticLockException on conflict
}
```

14. hibernate.ddl-auto = update in Production

```
# WRONG – Hibernate might drop columns with data!
spring.jpa.hibernate.ddl-auto: update

# CORRECT – validate schema matches entities, use Flyway for migrations
spring.jpa.hibernate.ddl-auto: validate
```

15. Loading All Records Without Pagination

```
// WRONG – 10 million records → OOM
List<User> users = userRepo.findAll();

// CORRECT – paginate or stream
Page<User> page = userRepo.findAll(PageRequest.of(0, 100));
// OR for batch processing:
userRepo.findAllAsStream().forEach(this::process); // Stream
```

Concurrency Mistakes

16. HashMap in Singleton Service

```
// WRONG – HashMap not thread-safe, multiple HTTP threads share this!
@Service // Singleton
public class CacheService {
    private Map<String, Object> cache = new HashMap<>(); // RACE CONDITION!
}

// CORRECT
private Map<String, Object> cache = new ConcurrentHashMap<>();
```

17. Raw Thread Creation in Production

```
// WRONG – uncontrolled, unbounded thread creation
new Thread(() -> processPayment(payment)).start(); // Can create millions!

// CORRECT – bounded thread pool
@Async("paymentExecutor")
public void processPaymentAsync(Payment payment) { ... }
```

18. Blocking in Reactive Pipeline

```
// WRONG – blocks reactor thread, defeats reactive purpose
Mono.fromCallable(() -> {
    return jdbcTemplate.queryForObject(sql, String.class); // BLOCKING!
})
```

```
});

// CORRECT
Mono.fromCallable(() -> jdbcTemplate.queryForObject(sql, String.class))
    .subscribeOn(Schedulers.boundedElastic()); // Offload to bounded thread pool
```

19. ThreadLocal Leak in Thread Pool

```
// WRONG – thread reused from pool, ThreadLocal from previous request still there
MDC.put("userId", userId);
// ... handle request ...
// Missing: MDC.clear()!

// CORRECT – always clean up in finally
try {
    MDC.put("userId", userId);
    handleRequest();
} finally {
    MDC.clear(); // Prevent leaking to next request on this thread
}
```

Security Mistakes

20. Dynamic SQL with User Input

```
// WRONG – SQL injection vulnerability
String sql = "SELECT * FROM users WHERE name = '" + userInput + "'";
// userInput = "' OR '1'='1" → returns all users!

// CORRECT
@Query("SELECT u FROM User u WHERE u.name = :name")
Optional<User> findByName(@Param("name") String name);
```

21. Logging Sensitive Data

```
// WRONG – PII/credentials in logs
log.info("Processing card: {}", cardDetails); // Logs full card number!
log.debug("User password: {}", password); // NEVER!

// CORRECT
log.info("Processing card ending in: {}", cardNumber.substring(cardNumber.length() - 4));
log.debug("Authentication attempt for user: {}", userId); // Log ID, not credentials
```

22. Returning Internal Exception Details

```
// WRONG – reveals stack trace/schema to attackers
@ExceptionHandler(Exception.class)
public ResponseEntity<String> handleError(Exception e) {
    return ResponseEntity.status(500).body(e.getMessage()); // Internal detail!
}

// CORRECT – log internally, return safe message
@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorResponse> handleError(Exception e) {
    log.error("Internal error", e); // Full detail for internal analysis
    return ResponseEntity.status(500)
        .body(new ErrorResponse("INTERNAL_ERROR", "An unexpected error occurred"));
}
```

Production/Operations Mistakes

23. No Heap Limits in Container

```
# WRONG - JVM will try to use all container/host memory
# - Kubernetes OOMKilled without mercy

# CORRECT
resources:
  limits:
    memory: "1Gi"

# AND in Java args:
JAVA_OPTS: "-XX:+UseContainerSupport -XX:MaxRAMPercentage=75.0"
```

24. Ignoring Connection Timeouts

```
// WRONG - hangs indefinitely on slow external service
RestTemplate restTemplate = new RestTemplate();

// CORRECT - always configure timeouts
HttpComponentsClientHttpRequestFactory factory = new HttpComponentsClientHttpRequestFactory();
factory.setConnectTimeout(5000); // 5s to establish connection
factory.setReadTimeout(30000); // 30s to read response
RestTemplate restTemplate = new RestTemplate(factory);
```

25. No Graceful Shutdown

```
# WRONG - abrupt kill on pod restart -> in-flight requests fail
# (no configuration = default abrupt shutdown)

# CORRECT
server:
  shutdown: graceful
spring:
  lifecycle:
    timeout-per-shutdown-phase: 30s
```

Interview Red Flags (Don't Say These)

```
x "I put @Transactional on everything to be safe"
x "I use HashMap because it's the most common"
x "I don't worry about N+1, the database is fast"
x "I always use double for numeric calculations"
x "I catch Exception at the top level to handle everything"
x "We test in production" (without clarifying intent)
x "The API validates the input so I don't need to in the service"
x "I use FetchType.EAGER so I always have the data"
x "I restart the service when there's a memory leak"
x "I disable CSRF because it's annoying"
```

Every mistake in this list represents a real production incident, security breach, or performance degradation that has affected real systems at real companies. Learn from others' mistakes.

Enterprise Engineering Checklist

Production-Ready Java Service Standards

API Layer Checklist

- ☐ REST endpoints follow naming conventions (`/api/v{n}/resources/{id}`)
- ☐ HTTP methods used correctly (GET=read, POST=create, PUT=replace, PATCH=update, DELETE=remove)
- ☐ Response status codes correct (201 for created, 204 for no content, 422 for validation errors)
- ☐ Input validation with `@Valid` on all request bodies
- ☐ Custom validation for business rules
- ☐ Paginated responses for list endpoints (Pageable, Page)
- ☐ Idempotency key header for mutating operations
- ☐ Request ID / trace ID propagated through service calls
- ☐ API versioning strategy defined (URI versioning recommended)
- ☐ `@ControllerAdvice` for global exception handling
- ☐ No internal exception details in API responses
- ☐ API documentation (Springdoc/OpenAPI configured)

Security Checklist

- ☐ Authentication implemented (JWT, OAuth2, or API key)
- ☐ Authorization checked for every endpoint (not just "is authenticated")
- ☐ Row-level security: users can only access their own resources
- ☐ Password hashing with BCrypt (cost factor ≥ 10)
- ☐ Account lockout after failed login attempts
- ☐ Rate limiting on authentication endpoints
- ☐ CORS configured (not `allowedOrigins("")` in production)
- ☐ CSRF protection enabled (or disabled only for stateless JWT APIs)
- ☐ Security headers configured (HSTS, X-Frame-Options, CSP)
- ☐ Sensitive data excluded from logs and error responses
- ☐ Secrets in environment variables or secrets manager (never in code)
- ☐ Dependencies scanned for CVEs (OWASP dependency check in CI)
- ☐ Input validation prevents SQL injection, XSS
- ☐ Audit logging for sensitive operations (financial, admin, PII)

Database Checklist

- ☐ Database migrations use Flyway/Liquibase (not DDL auto)
- ☐ `ddl-auto: validate` in production
- ☐ All migrations tested on staging with production-like data
- ☐ Proper indexes for all frequent query patterns
- ☐ N+1 queries eliminated (SQL logging enabled in dev, counted)
- ☐ `FetchType.LAZY` on all `@OneToMany` and `@ManyToMany`
- ☐ `@Version` on entities with concurrent update scenarios
- ☐ `BigDecimal` for all monetary/financial calculations
- ☐ Transaction isolation level explicitly chosen (not default)
- ☐ `@Transactional(readOnly = true)` on read operations
- ☐ Connection pool sized appropriately (HikariCP config present)
- ☐ Connection pool metrics monitored
- ☐ Database credentials in secrets management, not application.yml

Resilience Checklist

- ☐ Circuit breaker on all external service calls
- ☐ Retry with exponential backoff and jitter
- ☐ Timeout configured on all HTTP clients
- ☐ Timeout configured on all database queries
- ☐ Bulkhead configured (separate thread pools for critical paths)
- ☐ DLQ (Dead Letter Queue) for all async message processing
- ☐ Idempotent message consumers (deduplication)
- ☐ Graceful degradation strategy documented for each downstream dependency
- ☐ Rate limiting on public endpoints

Observability Checklist

- ☐ Structured JSON logging in production
- ☐ MDC populated with: `traceId`, `requestId`, `userId` on every request
- ☐ MDC cleared after each request (prevent thread pool leakage)
- ☐ Distributed tracing configured (Micrometer Tracing / OTel)
- ☐ Custom business metrics registered (payments processed, failures, durations)
- ☐ Prometheus endpoint enabled (`/actuator/prometheus`)
- ☐ Dashboards exist for: error rates, latency (P50/P95/P99), throughput, saturation
- ☐ Alerts configured for: error rate spikes, latency degradation, pod restarts
- ☐ Health endpoints: `/actuator/health/liveness` and `/actuator/health/readiness`
- ☐ Custom `HealthIndicator` for critical dependencies (DB, external APIs)
- ☐ Log level configurable without restart (`/actuator/loggers`)

Performance Checklist

- ☐ JVM heap sized appropriately (`-Xms` = `-Xmx` to avoid resize GC)
- ☐ GC algorithm chosen for workload (G1GC default, ZGC for low-latency)
- ☐ Container support enabled (`-XX:+UseContainerSupport`)
- ☐ Maximum RAM percentage set (`-XX:MaxRAMPercentage=75.0`)
- ☐ Thread pool sizes configured (not using defaults)
- ☐ Redis or local cache for frequently-read reference data
- ☐ Database queries optimized (EXPLAIN ANALYZE reviewed for new queries)
- ☐ Pagination for all list operations (no unbounded queries)
- ☐ Lazy loading used for collections (not EAGER)
- ☐ Heap dump on OOM configured (`-XX:+HeapDumpOnOutOfMemoryError`)
- ☐ GC logging enabled in production (`-Xlog:gc*`)

Kubernetes Deployment Checklist

- ☐ Liveness probe configured with appropriate `initialDelaySeconds`
- ☐ Readiness probe configured (traffic only when truly ready)
- ☐ Resource requests AND limits defined (memory and CPU)
- ☐ `preStop` hook with sleep for graceful load balancer deregistration
- ☐ `terminationGracePeriodSeconds` $\geq$ application shutdown timeout
- ☐ `server.shutdown: graceful` in Spring Boot config
- ☐ Secrets stored in k8s Secrets or external secrets manager
- ☐ ConfigMap for non-sensitive configuration
- ☐ HPA configured (CPU-based or custom metrics)
- ☐ Pod disruption budget (ensure minimum availability during rolling updates)
- ☐ Pod anti-affinity (prevent all pods on same node)

- ☐ Non-root user in Dockerfile
 - ☐ Read-only filesystem where possible
-

Testing Checklist

- ☐ Unit tests for all business logic (no Spring context needed)
 - ☐ Integration tests with Testcontainers (real DB, real Kafka)
 - ☐ MockMvc tests for API layer (request/response validation)
 - ☐ Test coverage threshold configured (e.g., JaCoCo $\geq 80\%$)
 - ☐ Architecture tests with ArchUnit (layer dependency rules enforced)
 - ☐ Contract tests for inter-service APIs (Pact or Spring Cloud Contract)
 - ☐ Performance/load tests for critical paths (JMeter / Gatling)
 - ☐ Security tests (OWASP ZAP in CI pipeline)
 - ☐ Tests run in CI before any deployment
-

Team Engineering Checklist

- ☐ Code review process defined (who reviews, what to look for)
 - ☐ PR template exists (description, testing steps, checklist)
 - ☐ Branching strategy defined (main, develop, feature/*)
 - ☐ Code style enforced (Checkstyle or Google Java Style in CI)
 - ☐ Architecture Decision Records (ADRs) for significant decisions
 - ☐ Runbook for common operational tasks
 - ☐ On-call rotation and escalation path defined
 - ☐ Post-mortem process for production incidents
 - ☐ Technical debt tracked (backlog, not ignored)
 - ☐ API changelog maintained for external consumers
-

Pre-Production Release Checklist

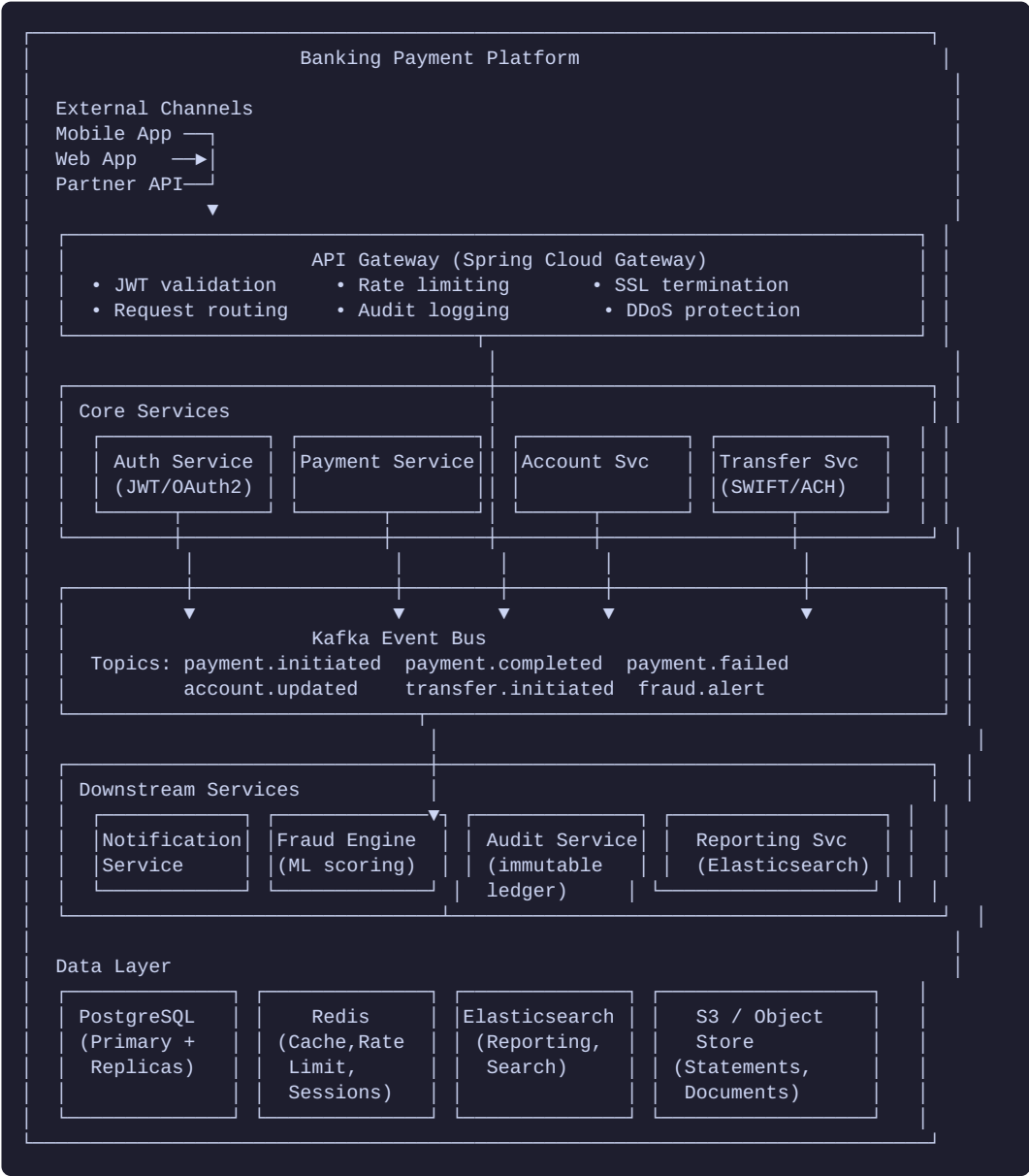
- ☐ All automated tests pass in CI
- ☐ Security scan (OWASP, SonarQube) passed
- ☐ Performance tested against production-like load
- ☐ Database migration reviewed (backward-compatible, tested on staging)
- ☐ Rollback plan documented (can we roll back? In how many minutes?)
- ☐ Feature flag configured (if gradual rollout needed)
- ☐ Monitoring dashboards + alerts ready for the new feature
- ☐ Team notified of deployment window
- ☐ On-call engineer designated for immediate post-deploy monitoring
- ☐ Canary deployment if significant change (route 5% traffic first)

Production Architecture Examples

Real-World Enterprise Java System Designs

Example 1: Banking Payment Processing Platform

Architecture Overview



Service Details

Payment Service — Core Domain

Tech: Spring Boot 3.2, Java 21
Database: PostgreSQL (primary) + read replica
Cache: Redis (idempotency keys, 24h TTL; rate limits)
Kafka: Producer (payment.initiated, payment.failed)
Resilience: Circuit breaker on external gateway, retry on transient failures

Throughput: 50,000 TPS peak
Latency: P99 < 500ms

Fraud Engine — ML-Powered

Tech: Spring Boot + Python microservice (via gRPC)
Input: Kafka consumer (payment.initiated)
Features: velocity, device fingerprint, behavioral anomaly
Output: APPROVE / REVIEW / DECLINE with score
Latency budget: 100ms synchronous (real-time blocking) or 500ms async

Example 2: Microservices Communication Patterns

Synchronous vs Asynchronous Decision Matrix

```
Communication Pattern Decision

Query (GET data)                → REST/gRPC (synchronous)
Command needing immediate response → REST/gRPC (synchronous)
Command with fire-and-forget     → Kafka (asynchronous)
Cross-domain event propagation  → Kafka (asynchronous)
Real-time streaming              → Kafka / WebSocket
Bulk data transfer               → Batch job / S3

Decision factors:
• Does caller need immediate result? → Sync
• Can caller proceed without result? → Async
• Multiple consumers of event?       → Kafka (fan-out)
• Need event replay capability?      → Kafka
• Need strong consistency?          → Sync (same DB transaction ideal)
```

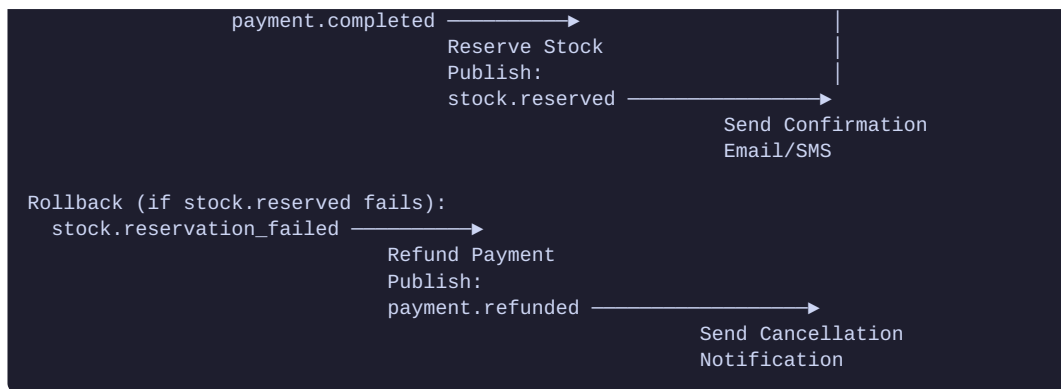
Service Mesh Configuration (Istio)

```
# Service-to-service retry policy
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: payment-service
spec:
  http:
    - route:
        - destination:
            host: payment-service
      timeout: 10s
    retries:
      attempts: 3
      perTryTimeout: 3s
      retryOn: gateway-error,connect-failure,retriable-4xx
```

Example 3: Event-Driven Order Processing

Choreography Saga

Order Service	Payment Service	Inventory Service	Notification Service
Create Order Publish: order.created	Process Payment Publish:		



Spring Implementation

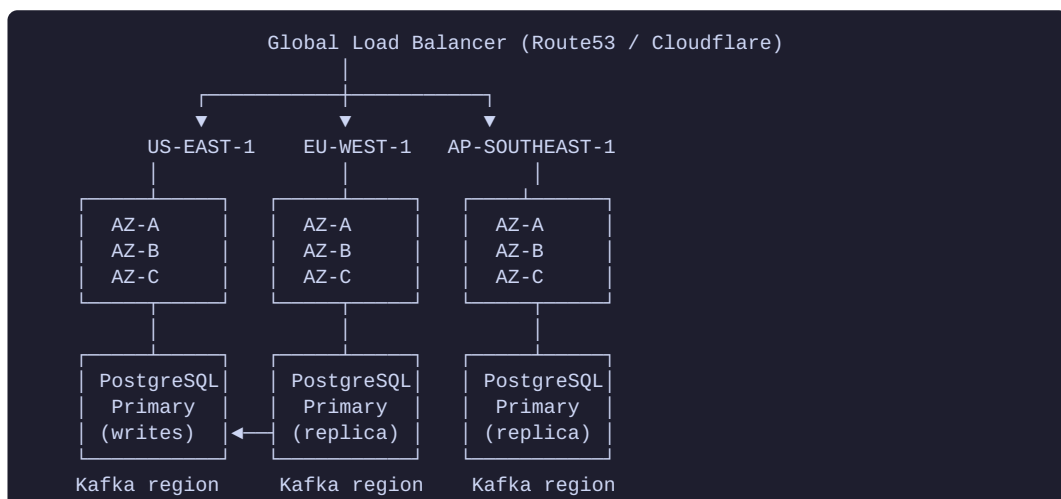
```
// Each service listens to relevant events
@Component
public class PaymentSagaParticipant {

    @KafkaListener(topics = "order.created")
    @Transactional
    public void onOrderCreated(OrderCreatedEvent event) {
        try {
            PaymentResult result = paymentGateway.charge(
                event.getCustomerId(),
                event.getTotalAmount(),
                event.getOrderId() // Idempotency key
            );

            if (result.isSuccessful()) {
                kafkaTemplate.send("payment.completed",
                    new PaymentCompletedEvent(event.getOrderId(), result.getTransactionId()));
            } else {
                kafkaTemplate.send("payment.failed",
                    new PaymentFailedEvent(event.getOrderId(), result.getDeclineReason()));
            }
        } catch (Exception e) {
            // Compensating action: publish failure event
            kafkaTemplate.send("payment.failed",
                new PaymentFailedEvent(event.getOrderId(), "INTERNAL_ERROR"));
        }
    }
}
```

Example 4: High-Availability Architecture

Multi-Region Active-Active



```
+ MirrorMaker2 + MirrorMaker2 + MirrorMaker2
(cross-region replication)
```

Java/Spring considerations for multi-region:

```
# application.yml – region-aware configuration
spring:
  datasource:
    # Writes to primary (current region primary)
    url: ${DB_PRIMARY_URL}
    # Reads from replica (local region for latency)
  jpa:
    properties:
      hibernate:
        read-only.url: ${DB_REPLICA_URL}

# Kafka: produce to local cluster, MirrorMaker2 replicates cross-region
spring:
  kafka:
    bootstrap-servers: ${KAFKA_LOCAL_BOOTSTRAP_SERVERS}
    consumer:
      group-id: ${SERVICE_NAME}-${AWS_REGION} # Region-specific consumer group
```

Example 5: CQRS + Event Sourcing for Account Management

Data Flow

```
Command Flow (Write):
CreateAccountCommand → AccountCommandHandler → Account.openAccount()
                                                    → AccountOpenedEvent → EventStore
                                                    → EventPublisher → Kafka

Query Flow (Read):
GET /accounts/{id} → AccountQueryHandler → AccountReadModel (Elasticsearch)
                                                    ← Populated by: AccountProjector (Kafka consumer)

AccountProjector (Kafka consumer):
AccountOpenedEvent → create AccountDocument in Elasticsearch
MoneyDepositedEvent → update balance in AccountDocument
AccountFrozenEvent → update status in AccountDocument
```

Event Store Schema

```
CREATE TABLE event_store (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  stream_id   VARCHAR(100) NOT NULL, -- e.g., "account-ACC-123"
  stream_type VARCHAR(50) NOT NULL,  -- e.g., "Account"
  version     BIGINT NOT NULL,
  event_type  VARCHAR(100) NOT NULL, -- "AccountOpened", "MoneyDeposited"
  payload     JSONB NOT NULL,
  metadata    JSONB,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  UNIQUE (stream_id, version),          -- Optimistic concurrency
  CHECK (version >= 0)
);

CREATE INDEX idx_event_store_stream ON event_store(stream_id, version);
CREATE INDEX idx_event_store_type_time ON event_store(event_type, created_at);
```

Example 6: Real-Time Risk Management System

Architecture for Financial Risk (Sub-100ms requirement)



Performance techniques for sub-100ms risk:

```
// Pre-computed position cache (Redis, updated by position service)
// Loaded at startup into JVM heap for zero-latency reads
@Service
public class PositionCache {
    private final ConcurrentHashMap<String, Position> positions =
        new ConcurrentHashMap<>(100_000);

    @PostConstruct
    public void loadFromRedis() {
        // Load all active positions into memory at startup
        redisTemplate.opsForHash().entries("positions:active")
            .forEach((k, v) -> positions.put((String) k, (Position) v));
    }

    // O(1) lookup - no network round trip
    public Position get(String instrumentId) {
        return positions.get(instrumentId);
    }
}

// Parallel risk computation using ForkJoinPool
public RiskReport computePortfolioRisk(List<Position> positions) {
    ForkJoinPool pool = new ForkJoinPool(Runtime.getRuntime().availableProcessors());
    return pool.invoke(new RiskComputationTask(positions, 0, positions.size()));
}
```

These architecture examples represent patterns used at global banks, fintech companies, and large-scale platforms. Understanding them deeply and being able to discuss trade-offs, scaling strategies, and failure modes is what separates senior from staff-level engineers.

Interview Preparation Sequence

Week-by-Week Sprint to Interview Readiness

Phase 1: Foundation Fluency (Weeks 1-4)

Week 1: Java Language Confidence

Day 1: Core types, strings, collections (HashMap, ArrayList, HashSet)
Day 2: OOP – classes, interfaces, abstract classes, enums
Day 3: Generics, wildcards, type erasure
Day 4: Streams, lambdas, functional interfaces
Day 5: Exception handling, try-with-resources
Day 6-7: Code katas – rewrite 5 Node.js functions in Java

Self-test: Can you code these without IDE help?

- ☐ Two Sum using HashMap
- ☐ Group a list by a property using Streams
- ☐ Custom exception with error code
- ☐ Generic Pair<A, B> class
- ☐ Fibonacci with memoization

Week 2: Spring Foundations

Day 1: DI container, bean lifecycle, scopes
Day 2: Spring MVC – request lifecycle, @Controller, @RestController
Day 3: Spring Data JPA – entities, repositories, queries
Day 4: Spring Security – filter chain, JWT
Day 5: @Transactional – propagation, isolation, pitfalls
Day 6-7: Build mini CRUD API (no reference code)

Self-test:

- ☐ Explain @Transactional pitfalls (3 of them)
- ☐ Explain N+1 problem and 3 ways to fix it
- ☐ Implement JWT filter from scratch
- ☐ Write JPA repository with custom query

Week 3: JVM + Concurrency

Day 1: Heap, stack, GC fundamentals
Day 2: GC algorithms, GC tuning flags
Day 3: Thread model, synchronized, volatile, AtomicXxx
Day 4: ThreadPoolExecutor, CompletableFuture
Day 5: Race conditions, deadlocks, prevention
Day 6-7: Concurrent programming exercises

Self-test:

- ☐ What JVM flags would you set for a 2GB container?
- ☐ Implement thread-safe counter 3 different ways
- ☐ Find the deadlock in given code
- ☐ Write CompletableFuture chain with parallel fetches

Week 4: First Mock Interview

Format: 45-minute coding mock (use Pramp or peer)
Topics: LeetCode Medium problem in Java
Focus: Clean code, edge cases, time complexity explanation

Phase 2: Depth Building (Weeks 5-8)

Week 5-6: Distributed Systems

Topics:

- Kafka: producers, consumers, partitions, offsets
- Delivery guarantees: at-most, at-least, exactly-once
- DLQ pattern
- CAP theorem with examples
- Circuit breaker

Practice:

- ☐ Implement Kafka producer with retry
- ☐ Implement idempotent Kafka consumer
- ☐ Explain CAP theorem for: PostgreSQL, Redis, Kafka
- ☐ Design DLQ handling strategy

Week 7-8: System Design Fundamentals

Study 5 designs:

1. URL shortener (LLD + HLD)
2. Rate limiter (token bucket + sliding window)
3. Notification system
4. Payment processing
5. Cache design with Redis

Practice: whiteboard each design in 45 minutes

Self-test:

- ☐ Design URL shortener (45 min, no notes)
- ☐ Design rate limiter at API gateway level
- ☐ Explain trade-offs between synchronous and async for payments

Phase 3: Interview Simulation (Weeks 9-12)

Weekly Mock Interview Schedule

Monday: Coding interview (1 LeetCode Medium/Hard, 45 min)
Wednesday: System Design mock (45 min, one of the 10 key designs)
Friday: Behavioral mock (STAR stories, 30 min)
Weekend: Review feedback, study weak areas

LeetCode Target Progress

Week 9: Total solved: 80+ (Easy: 40, Medium: 35, Hard: 5)
Week 10: Total solved: 100+ (Easy: 40, Medium: 50, Hard: 10)
Week 11: Total solved: 120+ (Medium: 60, Hard: 15)
Week 12: Total solved: 140+ (Medium: 70, Hard: 20)

Behavioral Stories Bank (Prepare All 10)

1. Most impactful technical contribution
 - STAR: Situation (scale/context), Task (your role), Action (what YOU did), Result (measurable: latency, uptime, cost, velocity)
2. Production incident response
 - Demonstrate: systematic diagnosis, calm under pressure, root cause analysis
3. Technical disagreement with peer/manager
 - Demonstrate: data-driven argument, listening, commit after decision
4. Simplified a complex system

- Demonstrate: architectural thinking, reduced complexity, measurable benefit
- 5. Worked across teams to deliver something
 - Demonstrate: collaboration, alignment, communication
- 6. Took on something outside your scope
 - Demonstrate: ownership, initiative, leadership potential
- 7. Mentored or helped a junior engineer
 - Demonstrate: communication, patience, knowledge transfer
- 8. Made decision with incomplete information
 - Demonstrate: risk management, bias for action, reversibility thinking
- 9. Failed and what you learned
 - Demonstrate: self-awareness, learning mindset, constructive framing
- 10. Long-term technical vision you drove
 - Demonstrate: strategic thinking, influence, architectural judgment

Phase 4: Company-Specific Preparation (Weeks 13-16)

For Amazon

- Focus:
- 14 Leadership Principles (know all, story for each)
 - LeetCode: Amazon-tagged problems (Graph, DP heavy)
 - System design: design at Amazon scale (millions of users, global)
- Amazon-specific questions:
- ☐ "Tell me about a time you disagreed with your manager" (Backbone)
 - ☐ "Describe a complex problem you solved" (Dive Deep)
 - ☐ "How did you handle a customer complaint?" (Customer Obsession)
 - ☐ "What's the most innovative thing you've built?" (Invent and Simplify)

For Google

- Focus:
- Algorithm and data structure depth (harder problems)
 - System design: scalability, reliability, observability
 - Code quality: clean, readable, optimal
- Google-specific:
- ☐ Expect 2-3 coding rounds + 1-2 system design
 - ☐ Googlyness round: collaboration, intellectual humility
 - ☐ Optimal solutions expected – brute force then optimize

For Goldman Sachs / JP Morgan

- Focus:
- Spring ecosystem depth (DI, AOP, Security, Data)
 - Transaction management (isolation levels, distributed transactions)
 - Security (OAuth2, JWT, PCI-DSS awareness)
 - Financial domain knowledge (FX, payments, settlements)
- Bank-specific questions:
- ☐ "How would you design a system to prevent duplicate payments?"
 - ☐ "Explain database isolation levels with a banking example"
 - ☐ "How do you audit all sensitive operations?"
 - ☐ "How would you handle a SWIFT transfer that partially fails?"
 - ☐ "Explain optimistic vs pessimistic locking for concurrent balance updates"

For Stripe

Focus:

- API design excellence (REST best practices, idempotency)
- Payment system knowledge
- Kafka and event-driven architecture
- Resilience engineering (retry, CB, idempotency)

Stripe-specific:

- ☐ "Design Stripe's idempotency key system"
- ☐ "How do you ensure exactly-once payment processing?"
- ☐ "Design the Stripe webhook delivery system"
- ☐ "How would you scale to 100x current payment volume?"

Daily Interview Prep Routine (During Active Job Search)

Morning (1 hour):

- 1 LeetCode problem, timed (25 min)
- Review solution and alternatives (10 min)
- Practice explaining solution out loud (10 min)
- Read 1 section from handbook (20 min)

Evening (30 min):

- Review 5 behavioral questions (rotate through bank of stories)
- Practice answering 1 system design question to yourself
- Read engineering blog from target company

Weekend (4 hours):

- Full mock interview (coding + system design) with peer/recording
- Review and improve weak behavioral stories
- Study 1 new system design from architecture examples
- Cold coding: pick any hard problem, code without looking at solutions

Quick Interview Reference Cards

Coding Interview Checklist

Before coding:

- ☐ Clarify requirements (1-2 minutes)
- ☐ Discuss examples (happy path + edge cases)
- ☐ Propose approach BEFORE coding
- ☐ Agree on approach with interviewer

During coding:

- ☐ Think aloud continuously
- ☐ Write clean variable names (not a, b, temp)
- ☐ Handle null/empty inputs
- ☐ Modularize with helper methods

After coding:

- ☐ Trace through with example
- ☐ State time and space complexity
- ☐ Discuss alternative approaches
- ☐ Mention test cases you'd write

System Design Checklist

- ☐ Requirements clarification (5 min)
- ☐ Scale estimation (5 min)
- ☐ API design (5 min)
- ☐ High-level diagram: boxes and arrows (10 min)
- ☐ Data model (5 min)
- ☐ Deep dive on critical component (10 min)

- Scaling, bottlenecks, trade-offs (5 min)
- Failure scenarios and mitigation (5 min)

Java/Spring Quick Answers

@Transactional pitfall → self-invocation bypasses AOP proxy
N+1 fix → JOIN FETCH or @EntityGraph
Thread-safe collection → ConcurrentHashMap (not HashMap in singleton)
Connection pool → HikariCP: size = core_count * 2 + spindles
GC pause spikes → Check GC logs, tune MaxGCPauseMillis, add heap
JWT vulnerability → alg:none, missing exp check, missing iss/aud check
Idempotency → Redis with setIfAbsent + DB unique constraint
Circuit breaker → Resilience4j: failureRateThreshold, waitDurationInOpenState